

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO ASSIGNMENT 05 MÔN HỌC PHÁT TRIỂN
CÁC HỆ THỐNG THÔNG MINH

Giảng viên hướng dẫn: PGS.TS. Trần Đình Quế

Họ và tên: Nguyễn Chí Hùng

Mã sinh viên: B23DCCN359

Lớp: D23CNPM04

Nhóm lớp: 05

Hà Nội, 2026

MỤC LỤC

Phần I: Cơ sở lý thuyết	5
1. Tổng quan về RNN	5
2. Tại sao cần RNN?	5
3. Dữ liệu tuần tự trong RNN	6
4. Cấu trúc cơ bản của RNN	7
5. Hidden State – trạng thái ẩn	8
6. Công thức toán học của RNN	9
7. Hàm kích hoạt tanh	9
8. Tại sao RNN có thể "nhớ"?	10
9. Unrolling RNN	11
10. Weight Sharing	12
11. Các dạng bài toán của RNN	12
12. Sequence Length	13
13. Ba kích thước quan trọng trong RNN	14
14. RNN trong PyTorch	14
15. Ý nghĩa các tham số của nn.RNN	15
16. Ví dụ RNN đơn giản nhất	17
17. Phân biệt output và hidden	17
18. RNN cho bài toán phân loại	18
19. Code RNN hoàn chỉnh cho Classification	20
20. Tạo dữ liệu giả	21
21. Forward Pass	21
22. Loss Function	22
23. Optimizer	22
24. Training Loop cơ bản	22
25. Ý nghĩa của backward()	24
26. Vì sao RNN có Backpropagation Through Time?	24
27. Vanishing Gradient	24
28. Ví dụ về Long-Term Dependency	25
29. Exploding Gradient	25

30. Gradient Clipping	25
31. RNN nhiều tầng	26
32. Dropout trong RNN	27
33. Bidirectional RNN	27
34. Các hàm RNN quan trọng trong PyTorch	28
35. RNN và LSTM	28
36. RNN và GRU	29
37. So sánh RNN, LSTM và GRU	29
38. RNN cho dự báo chuỗi thời gian	30
39. RNN cho Regression	31
40. RNN cho Classification	32
41. Cách chuẩn bị dữ liệu cho RNN	32
42. Chuyển dữ liệu sang Tensor	33
43. Dataset và DataLoader	34
44. Training Loop hoàn chỉnh	34
45. Evaluation	35
46. Dự đoán lớp	36
47. Accuracy	36
48. Softmax	37
49. Một mô hình RNN hoàn chỉnh	37
50. Luồng hoạt động của toàn bộ RNN	43
51. Sơ đồ kiến thức RNN cần ghi nhớ	44
52. Kết luận về RNN	45
Phần II. Phân tích dataset	46
1. DJIA 30 Stock Time Series	46
1.1. Nguồn gốc và bản chất	46
1.2. Cấu trúc và mô tả thuộc tính	46
1.3. Đặc điểm của chuỗi dữ liệu	47
1.4. Đánh giá mức độ phù hợp với bài toán RNN	48
1.5. Thiết kế bài toán	48
1.6. Những rủi ro cần kiểm soát	49

2. E-commerce Customer Data For Behavior Analysis	49
2.1. Nguồn gốc và bản chất	49
2.2. Cấu trúc và mô tả thuộc tính	50
2.3. Đặc điểm và chất lượng dữ liệu	50
2.4. Đánh giá mức độ phù hợp với bài toán RNN	51
2.5. Quy trình tiền xử lý đề xuất	52
2.6. Phương án mô hình và đánh giá	53
2.7. Hạn chế	53
3. So sánh hai bộ dữ liệu	53

Phần I: Cơ sở lý thuyết

1. Tổng quan về RNN

1.1. RNN là gì?

RNN (Recurrent Neural Network) là một loại mạng nơ-ron nhân tạo được thiết kế để xử lý **dữ liệu tuần tự (sequential data)**, trong đó thứ tự của các phần tử có ý nghĩa.

Một số dữ liệu phù hợp với RNN:

- Văn bản: câu, đoạn văn, chuỗi từ.
- Dữ liệu thời gian: nhiệt độ, giá cổ phiếu, doanh số.
- Âm thanh: chuỗi tín hiệu âm thanh.
- Chuỗi cảm biến IoT.
- Dữ liệu hành vi theo thời gian.
- Chuỗi DNA.
- Dự báo chuỗi thời gian.

Điểm khác biệt quan trọng của RNN so với mạng nơ-ron truyền thống là RNN có khả năng **duy trì trạng thái từ các bước trước**.

Ví dụ, xét câu:

"Hôm nay trời rất ..."

Để dự đoán từ tiếp theo là **"đẹp"**, mô hình cần sử dụng thông tin từ những từ trước đó.

RNN giải quyết vấn đề này bằng cách duy trì một **hidden state** – trạng thái ẩn.

2. Tại sao cần RNN?

2.1. Hạn chế của mạng nơ-ron thông thường

Giả sử ta có dữ liệu:

Ngày 1: 25°C

Ngày 2: 26°C

Ngày 3: 27°C

Ngày 4: 28°C

Nếu muốn dự đoán nhiệt độ ngày 5, thông tin của những ngày trước đó rất quan trọng.

Một mạng Neural Network thông thường xử lý từng mẫu độc lập:

Input $\rightarrow$ Neural Network $\rightarrow$ Output

Nó không có cơ chế tự nhiên để nhớ:

Ngày 1 $\rightarrow$ Ngày 2 $\rightarrow$ Ngày 3 $\rightarrow$ Ngày 4

Trong khi đó RNN có thể duy trì thông tin:

$x_1 \rightarrow \text{RNN} \rightarrow h_1$

$\downarrow$

$x_2 \rightarrow \text{RNN} \rightarrow h_2$

$\downarrow$

$x_3 \rightarrow \text{RNN} \rightarrow h_3$

$\downarrow$

$x_4 \rightarrow \text{RNN} \rightarrow h_4$

$\downarrow$

Output

Trong đó:

- x_t : dữ liệu tại thời điểm t .
- h_t : trạng thái ẩn tại thời điểm t .
- h_{t-1} : thông tin được truyền từ thời điểm trước.

3. Dữ liệu tuần tự trong RNN

Một chuỗi dữ liệu có thể được biểu diễn:

$$X = (x_1, x_2, x_3, \dots, x_T)$$

Trong đó:

- T : độ dài chuỗi.
- x_1 : phần tử đầu tiên.
- x_2 : phần tử thứ hai.
- ...

- x_t : phần tử tại thời điểm t .

Ví dụ chuỗi nhiệt độ:

[25, 26, 27, 28, 29]

Có:

$$x_1 = 25$$

$$x_2 = 26$$

$$x_3 = 27$$

$$x_4 = 28$$

$$x_5 = 29$$

RNN sẽ xử lý lần lượt:

$$x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow x_4 \rightarrow x_5$$

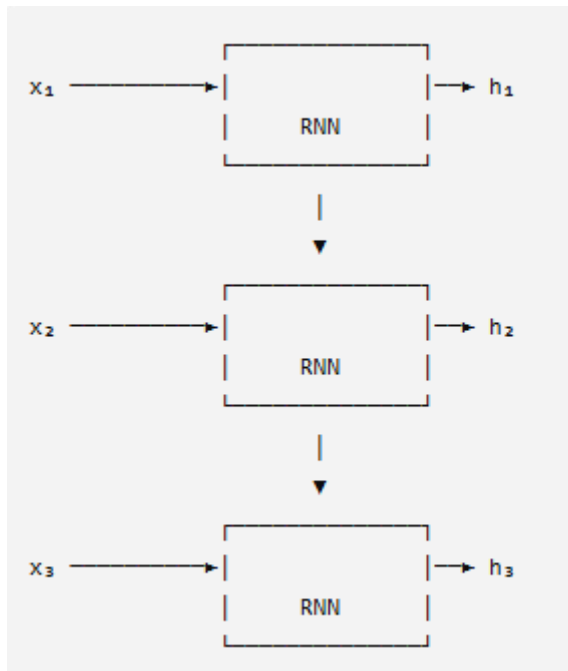
chứ không xem toàn bộ chuỗi như những mẫu hoàn toàn độc lập.

4. Cấu trúc cơ bản của RNN

Một RNN cơ bản gồm:

1. Input x_t .
2. Hidden state h_t .
3. Output y_t .
4. Các trọng số được **chia sẻ giữa các time step**.

Có thể hình dung:



Điều quan trọng là các khối RNN trên thực tế **sử dụng chung một bộ trọng số**.

5. Hidden State – trạng thái ẩn

5.1. Hidden state là gì?

Hidden state là thành phần quan trọng nhất của RNN.

Nó có nhiệm vụ lưu trữ một phần thông tin từ các bước trước.

Tại thời điểm t :

$$h_t = f(x_t, h_{t-1})$$

Điều này có nghĩa:

Trạng thái hiện tại phụ thuộc vào dữ liệu hiện tại và trạng thái của bước trước.

Ví dụ:

h_0

↓

$x_1 \rightarrow h_1$

↓

$x_2 \rightarrow h_2$

↓

$x_3 \rightarrow h_3$

h_3 có thể chứa thông tin được tích lũy từ x_1 , x_2 và x_3 .

6. Công thức toán học của RNN

Đây là công thức quan trọng nhất cần hiểu.

RNN cơ bản thường sử dụng:

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

Trong đó:

- x_t : input tại thời điểm t .
- h_{t-1} : hidden state của thời điểm trước.
- W_{xh} : trọng số từ input đến hidden.
- W_{hh} : trọng số từ hidden trước đến hidden hiện tại.
- b_h : bias.
- $\tanh$: hàm kích hoạt.
- h_t : hidden state hiện tại.

Sau đó output được tính:

$$y_t = W_{hy}h_t + b_y$$

Trong đó:

- W_{hy} : trọng số từ hidden đến output.
- b_y : bias output.
- y_t : output.

7. Hàm kích hoạt tanh

RNN truyền thống thường sử dụng hàm:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Giá trị của tanh nằm trong khoảng:

$$-1 < \tanh(x) < 1$$

Ví dụ:

```
</> Python

import torch

x = torch.tensor([-2.0, -1.0, 0.0, 1.0, 2.0])

y = torch.tanh(x)

print(y)
```

print(y)

Kết quả có dạng:

tensor([-0.9640, -0.7616, 0.0000, 0.7616, 0.9640])

Tanh giúp mô hình đưa giá trị hidden state về khoảng tương đối ổn định.

8. Tại sao RNN có thể "nhớ"?

Giả sử:

$x_1 = \text{"Tôi"}$

$x_2 = \text{"đang"}$

$x_3 = \text{"học"}$

$x_4 = \text{"trí"}$

$x_5 = \text{"tuệ"}$

RNN xử lý:

$x_1 \rightarrow h_1$

↓

$x_2 \rightarrow h_2$

↓

$x_3 \rightarrow h_3$

↓

$x_4 \rightarrow h_4$

↓

$x_5 \rightarrow h_5$

Ta có:

$$h_1 = f(x_1, h_0)$$

$$h_2 = f(x_2, h_1)$$

$$h_3 = f(x_3, h_2)$$

$$h_4 = f(x_4, h_3)$$

$$h_5 = f(x_5, h_4)$$

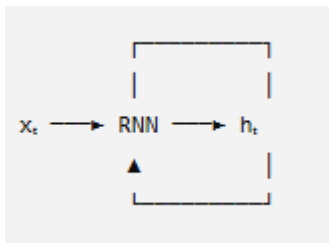
Do đó h_5 gián tiếp phụ thuộc vào các trạng thái trước đó.

Đây chính là cơ chế **memory** của RNN.

9. Unrolling RNN

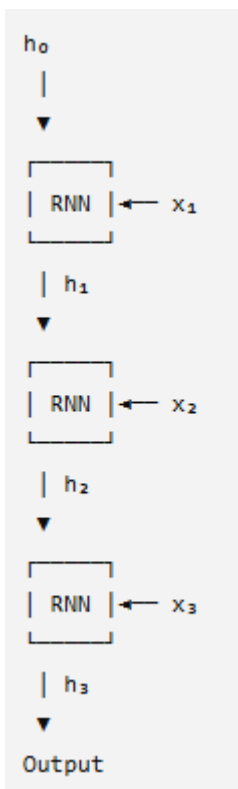
Một RNN có thể được biểu diễn theo hai cách.

9.1. Dạng vòng lặp



9.2. Dạng trải theo thời gian

Để dễ hiểu, ta "mở" vòng lặp:



Output

Quá trình này được gọi là:

Unrolling through time

10. Weight Sharing

Một đặc điểm rất quan trọng của RNN là **các time step dùng chung trọng số**.

Ví dụ:

$$x_1 \rightarrow \text{RNN}(W) \rightarrow h_1$$

$$x_2 \rightarrow \text{RNN}(W) \rightarrow h_2$$

$$x_3 \rightarrow \text{RNN}(W) \rightarrow h_3$$

$$x_4 \rightarrow \text{RNN}(W) \rightarrow h_4$$

Không phải:

$$x_1 \rightarrow \text{RNN}(W_1)$$

$$x_2 \rightarrow \text{RNN}(W_2)$$

$$x_3 \rightarrow \text{RNN}(W_3)$$

mà là:

$$x_1 \rightarrow \text{RNN}(W)$$

$$x_2 \rightarrow \text{RNN}(W)$$

$$x_3 \rightarrow \text{RNN}(W)$$

Điều này giúp số lượng tham số không tăng theo độ dài chuỗi.

11. Các dạng bài toán của RNN

RNN có thể được sử dụng cho nhiều kiểu input/output.

11.1. One-to-One

Input $\rightarrow$ Output

Ví dụ:

Ảnh $\rightarrow$ Nhãn

Trường hợp này thường không cần RNN.

11.2. One-to-Many

Input

↓

$\text{Output}_1 \rightarrow \text{Output}_2 \rightarrow \text{Output}_3$

Ví dụ:

Ảnh $\rightarrow$ Caption

11.3. Many-to-One

$x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow x_4$

↓

Output

Ví dụ:

Câu văn $\rightarrow$ Sentiment

Input:

"I really love this movie"

Output:

Positive

Đây là dạng rất phổ biến cho bài toán phân loại chuỗi.

11.4. Many-to-Many

$x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow x_4$

↓

↓

↓

↓

y_1

y_2

y_3

y_4

Ví dụ:

Machine Translation

hoặc:

Part-of-Speech Tagging

12. Sequence Length

Giả sử một batch có:

`batch_size = 32`

`sequence_length = 10`

`input_size = 5`

thì dữ liệu có thể có kích thước:

`[32, 10, 5]`

Trong đó:

- 32: số mẫu trong batch.
- 10: mỗi mẫu có 10 time step.
- 5: mỗi time step có 5 feature.

Ví dụ dữ liệu cảm biến:

10 thời điểm

×

5 thông số cảm biến

13. Ba kích thước quan trọng trong RNN

Khi sử dụng PyTorch, cần đặc biệt hiểu:

`batch_size`

`sequence_length`

`input_size`

Ví dụ:

`x.shape`

cho:

`[32, 10, 5]`

có nghĩa:

`32 = batch_size`

`10 = sequence_length`

`5 = input_size`

Đây là một trong những phần dễ gây nhầm lẫn nhất khi mới học RNN.

14. RNN trong PyTorch

PyTorch cung cấp sẵn:

nn.RNN

Cú pháp cơ bản:

```
nn.RNN(  
    input_size,  
    hidden_size,  
    num_layers=1,  
    batch_first=True  
)
```

Ví dụ:

```
</> Python  
  
import torch  
import torch.nn as nn  
  
rnn = nn.RNN(  
    input_size=5,  
    hidden_size=16,  
    num_layers=1,  
    batch_first=True  
)
```

15. Ý nghĩa các tham số của nn.RNN

15.1. input_size

Số feature tại mỗi time step.

Ví dụ:

Mỗi ngày:

temperature

humidity

light

thì:

input_size = 3

15.2. hidden_size

Kích thước của hidden state.

Ví dụ:

`hidden_size = 64`

thì:

`h_t.shape = [batch_size, 64]`

Hidden size càng lớn thì mô hình có khả năng biểu diễn nhiều thông tin hơn nhưng số lượng tham số cũng tăng.

15.3. num_layers

Số tầng RNN xếp chồng.

Ví dụ:

`num_layers=2`

có nghĩa:

Layer 1

↓

Layer 2

↓

Output

Không nên nhầm `num_layers` với `sequence_length`.

15.4. batch_first=True

Nếu:

`batch_first=True`

input có dạng:

`[batch, sequence, feature]`

Ví dụ:

`[32, 10, 5]`

Nếu không dùng `batch_first=True`, PyTorch mặc định sử dụng:

`[sequence, batch, feature]`

tức:

[10, 32, 5]

Đối với người mới học, nên dùng:

`batch_first=True`

để dễ quản lý dữ liệu.

16. Ví dụ RNN đơn giản nhất

```
</> Python

import torch
import torch.nn as nn

# Batch = 4
# Sequence Length = 10
# Feature = 5

x = torch.randn(4, 10, 5)

rnn = nn.RNN(
    input_size=5,
    hidden_size=16,
    batch_first=True
)

output, hidden = rnn(x)

print("Input:", x.shape)
print("Output:", output.shape)
print("Hidden:", hidden.shape)
```

Kết quả:

Input: `torch.Size([4, 10, 5])`

Output: `torch.Size([4, 10, 16])`

Hidden: `torch.Size([1, 4, 16])`

17. Phân biệt output và hidden

Đây là điểm rất quan trọng.

Với:

`output, hidden = rnn(x)`

output

Chứa hidden state của **tất cả các time step**.

[batch, sequence, hidden]

Ví dụ:

[4, 10, 16]

Tức:

h_1

h_2

h_3

...

h_{10}

hidden

Chỉ chứa hidden state cuối cùng của mỗi layer.

[num_layers, batch, hidden]

Ví dụ:

[1, 4, 16]

Có nghĩa:

h_{10}

18. RNN cho bài toán phân loại

Giả sử muốn phân loại một chuỗi thành 3 lớp:

Class 0

Class 1

Class 2

Ta có thể lấy hidden state cuối:

`</>` Python

```
output, hidden = self.rnn(x)

last_hidden = hidden[-1]

out = self.fc(last_hidden)
```

Sau đó:

`</>` Python

```
self.fc = nn.Linear(hidden_size, 3)
```

Kiến trúc:

Input Sequence



RNN



Last Hidden State



Linear



3 Classes

19. Code RNN hoàn chỉnh cho Classification

```
</> Python

import torch
import torch.nn as nn

class RNNClassifier(nn.Module):

    def __init__(
        self,
        input_size,
        hidden_size,
        num_layers,
        num_classes
    ):
        super().__init__()

        self.rnn = nn.RNN(
            input_size=input_size,
            hidden_size=hidden_size,
            num_layers=num_layers,
            batch_first=True
        )

        self.fc = nn.Linear(
            hidden_size,
            num_classes
        )

    def forward(self, x):

        output, hidden = self.rnn(x)

        # Lấy hidden state của layer cuối
        last_hidden = hidden[-1]

        # Classification
        out = self.fc(last_hidden)

        return out
```

Khởi tạo:

Python

```
model = RNNClassifier(  
    input_size=5,  
    hidden_size=64,  
    num_layers=2,  
    num_classes=3  
)
```

20. Tạo dữ liệu giả

Python

```
X = torch.randn(100, 10, 5)  
y = torch.randint(0, 3, (100,))
```

Ở đây:

100 = số mẫu

10 = sequence length

5 = số feature

Còn:

y = 0, 1 hoặc 2

21. Forward Pass

Python

```
output = model(X)  
  
print(output.shape)
```

Kết quả:

torch.Size([100, 3])

Mỗi mẫu nhận được 3 giá trị:

```
[score_class_0,  
 score_class_1,  
 score_class_2]
```

Các giá trị này thường được gọi là **logits**.

22. Loss Function

Đối với bài toán classification nhiều lớp, thường sử dụng:

`nn.CrossEntropyLoss()`

Ví dụ:

```
</> Python

criterion = nn.CrossEntropyLoss()

loss = criterion(output, y)
```

Lưu ý:

`CrossEntropyLoss` nhận **logits**, không cần tự gọi `softmax` trước.

Không nên viết:

```
output = torch.softmax(output, dim=1)
```

```
loss = criterion(output, y)
```

trong trường hợp sử dụng trực tiếp `CrossEntropyLoss`.

23. Optimizer

Optimizer phổ biến:

`torch.optim.Adam`

Ví dụ:

```
</> Python

optimizer = torch.optim.Adam(
    model.parameters(),
    lr=0.001
)
```

Trong đó:

`lr` = learning rate

Learning rate quyết định mức độ thay đổi của trọng số sau mỗi lần cập nhật.

24. Training Loop cơ bản

Một vòng huấn luyện RNN:

`</>` Python

```
for epoch in range(20):

    model.train()

    optimizer.zero_grad()

    output = model(X)

    loss = criterion(output, y)

    loss.backward()

    optimizer.step()

    print(
        f"Epoch {epoch+1}, "
        f"Loss: {loss.item():.4f}"
    )
```

Quy trình:

Input



RNN



Output



Loss



Backward



Gradient



Optimizer



Update weights

25. Ý nghĩa của backward()

loss.backward()

PyTorch sẽ tính gradient:

$$\frac{\partial L}{\partial W}$$

đối với các trọng số của mạng.

Sau đó optimizer sử dụng gradient để cập nhật:

$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial W}$$

Trong đó:

- W: trọng số.
- L: loss.
- η : learning rate.

26. Vì sao RNN có Backpropagation Through Time?

Vì RNN xử lý dữ liệu theo thời gian:

$$x_1 \rightarrow h_1 \rightarrow h_2 \rightarrow h_3 \rightarrow h_4$$

nên khi tính gradient, gradient phải truyền ngược qua nhiều time step:

$$h_4 \leftarrow h_3 \leftarrow h_2 \leftarrow h_1$$

Quá trình này gọi là:

Backpropagation Through Time (BPTT).

27. Vanishing Gradient

Một vấn đề nổi tiếng của RNN là:

Vanishing Gradient – mất dần gradient.

Trong quá trình backpropagation qua nhiều time step, gradient có thể liên tục được nhân với những giá trị nhỏ hơn 1:

$$\frac{\partial L}{\partial h_t} \approx \prod_{i=t}^T \frac{\partial h_i}{\partial h_{i-1}}$$

Nếu các giá trị này nhỏ, tích có thể tiến rất gần:

Khi đó gradient trở nên cực nhỏ.

Kết quả:

RNN khó học được các mối quan hệ phụ thuộc dài hạn.

28. Ví dụ về Long-Term Dependency

Xét câu:

"Tôi sinh ra ở Việt Nam và tôi nói tiếng Việt."

Để dự đoán từ cuối, thông tin "Việt Nam" xuất hiện khá xa phía trước.

Nếu chuỗi rất dài:

$X_1 \rightarrow X_2 \rightarrow X_3 \rightarrow \dots \rightarrow X_{100}$

thông tin từ x_1 có thể rất khó truyền tới x_{100} .

Đây chính là vấn đề **long-term dependency**.

29. Exploding Gradient

Ngược lại, gradient có thể trở nên quá lớn.

Ví dụ:

$1.5 \times 1.5 \times 1.5 \times \dots$

sau nhiều bước có thể tăng rất nhanh.

Đây gọi là:

Exploding Gradient.

Một kỹ thuật thường dùng để xử lý là:

Gradient Clipping.

30. Gradient Clipping

Trong PyTorch:

```
</> Python
torch.nn.utils.clip_grad_norm_(
    model.parameters(),
    max_norm=1.0
)
```

Ví dụ:

Python

```
loss.backward()

torch.nn.utils.clip_grad_norm_(
    model.parameters(),
    max_norm=1.0
)

optimizer.step()
```

Mục đích:

Giới hạn độ lớn của gradient để tránh gradient tăng quá lớn.

31. RNN nhiều tầng

Ta có thể sử dụng:

num_layers=3

Kiến trúc:

Input

↓

RNN Layer 1

↓

RNN Layer 2

↓

RNN Layer 3

↓

Output

Code:

Python

```
rnn = nn.RNN(
    input_size=5,
    hidden_size=64,
    num_layers=3,
    batch_first=True
)
```

32. Dropout trong RNN

Để giảm overfitting:

```
</> Python

rnn = nn.RNN(
    input_size=5,
    hidden_size=64,
    num_layers=2,
    dropout=0.2,
    batch_first=True
)
```

Lưu ý quan trọng:

dropout của nn.RNN chỉ được áp dụng giữa các RNN layer khi:

`num_layers > 1`

33. Bidirectional RNN

RNN thông thường chỉ xử lý theo một hướng:

$X_1 \rightarrow X_2 \rightarrow X_3 \rightarrow X_4$

Bidirectional RNN xử lý theo hai hướng:

Forward:

$X_1 \rightarrow X_2 \rightarrow X_3 \rightarrow X_4$

Backward:

$X_4 \rightarrow X_3 \rightarrow X_2 \rightarrow X_1$

Code:

```
</> Python

rnn = nn.RNN(
    input_size=5,
    hidden_size=64,
    bidirectional=True,
    batch_first=True
)
```

Khi đó số feature đầu ra sẽ là:

$$64 \times 2 = 128$$

Do đó nếu đưa vào Linear:

```
</> Python  
self.fc = nn.Linear(64 * 2, num_classes)
```

34. Các hàm RNN quan trọng trong PyTorch

Những hàm/lớp nên biết:

nn.RNN

nn.LSTM

nn.GRU

nn.Linear

nn.CrossEntropyLoss

nn.MSELoss

torch.optim.Adam

torch.optim.AdamW

torch.nn.utils.clip_grad_norm_

35. RNN và LSTM

RNN cơ bản:

Input

↓

RNN

↓

Hidden State

LSTM bổ sung cơ chế **gates** và **cell state** để kiểm soát thông tin được giữ lại hoặc loại bỏ.

LSTM có:

- Forget gate.
- Input gate.
- Output gate.

- Cell state.
- Hidden state.

Do đó LSTM thường được sử dụng khi chuỗi có sự phụ thuộc dài hạn.

Code:

```
</> Python

lstm = nn.LSTM(
    input_size=5,
    hidden_size=64,
    num_layers=2,
    batch_first=True
)
```

36. RNN và GRU

GRU – **Gated Recurrent Unit** – cũng được thiết kế để giải quyết vấn đề của RNN truyền thống.

GRU có cấu trúc đơn giản hơn LSTM.

Code:

```
</> Python

gru = nn.GRU(
    input_size=5,
    hidden_size=64,
    num_layers=2,
    batch_first=True
)
```

37. So sánh RNN, LSTM và GRU

Đặc điểm	RNN	LSTM	GRU
Cấu trúc	Đơn giản	Phức tạp	Trung bình
Hidden state	Có	Có	Có
Cell state	Không	Có	Không
Gate	Không	3 gate	2 gate
Long-term dependency	Hạn chế	Tốt hơn	Tốt

Đặc điểm	RNN	LSTM	GRU
Số tham số	Ít	Nhiều	Trung bình
Tốc độ	Nhanh	Chậm hơn	Thường nhanh hơn LSTM

Có thể hiểu đơn giản:

RNN



RNN cơ bản, dễ hiểu nhưng khó nhớ dài hạn

LSTM



RNN có cơ chế kiểm soát bộ nhớ mạnh

GRU



Phiên bản gated đơn giản hơn LSTM

38. RNN cho dự báo chuỗi thời gian

RNN đặc biệt phù hợp với dữ liệu thời gian.

Ví dụ:

Ngày 1: 20

Ngày 2: 22

Ngày 3: 21

Ngày 4: 24

Ngày 5: 25

Sử dụng:

5 ngày trước



RNN

↓

Dự đoán ngày 6

Ví dụ input:

X.shape = [32, 5, 1]

Trong đó:

32 = batch

5 = 5 ngày

1 = nhiệt độ

Output:

[32, 1]

39. RNN cho Regression

Ví dụ dự đoán nhiệt độ.

Model:

```
</> Python

class RNNRegressor(nn.Module):

    def __init__(self, input_size, hidden_size):
        super().__init__()

        self.rnn = nn.RNN(
            input_size,
            hidden_size,
            batch_first=True
        )

        self.fc = nn.Linear(
            hidden_size,
            1
        )

    def forward(self, x):

        output, hidden = self.rnn(x)

        last_hidden = hidden[-1]

        return self.fc(last_hidden)
```

Loss:

```
</> Python  
criterion = nn.MSELoss()
```

40. RNN cho Classification

Ví dụ phân loại:

Chuỗi cảm biến



RNN



Hidden cuối



Linear



Class

Loss:

```
criterion = nn.CrossEntropyLoss()
```

41. Cách chuẩn bị dữ liệu cho RNN

Giả sử có dữ liệu:

Temperature

25

26

27

28

29

30

31

32

Ta muốn dùng 3 giá trị trước để dự đoán giá trị tiếp theo.

Tạo:

[25,26,27] → 28

[26,27,28] → 29

[27,28,29] → 30

[28,29,30] → 31

[29,30,31] → 32

Code:

```
</> Python

def create_sequences(data, seq_length):

    X = []
    y = []

    for i in range(len(data) - seq_length):
        X.append(data[i:i + seq_length])
        y.append(data[i + seq_length])

    return X, y
```

Sử dụng:

```
</> Python

data = [25,26,27,28,29,30,31,32]

X, y = create_sequences(data, 3)

print(X)
print(y)
```

42. Chuyển dữ liệu sang Tensor

```
</> Python

X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32)
```

Nếu mỗi time step chỉ có một feature:

[batch, sequence, feature]

có thể cần reshape:

```
</> Python
```

```
X = X.unsqueeze(-1)
```

Ví dụ:

[6, 3]

thành:

[6, 3, 1]

43. Dataset và DataLoader

Trong dự án thực tế, không nên đưa toàn bộ dữ liệu vào model cùng một lúc.

Sử dụng:

```
</> Python
```

```
from torch.utils.data import TensorDataset, DataLoader

dataset = TensorDataset(X, y)

loader = DataLoader(
    dataset,
    batch_size=32,
    shuffle=True
)
```

Sau đó:

```
</> Python
```

```
for batch_X, batch_y in loader:

    output = model(batch_X)

    loss = criterion(
        output,
        batch_y
    )
```

44. Training Loop hoàn chỉnh

Một training loop cơ bản:

`</>` Python

```
for epoch in range(20):

    model.train()

    total_loss = 0

    for batch_X, batch_y in loader:

        optimizer.zero_grad()

        output = model(batch_X)

        loss = criterion(
            output,
            batch_y
        )

        loss.backward()

        torch.nn.utils.clip_grad_norm_(
            model.parameters(),
            max_norm=1.0
        )

        optimizer.step()

        total_loss += loss.item()

    avg_loss = total_loss / len(loader)

    print(
        f"Epoch {epoch+1}/20 "
        f"- Loss: {avg_loss:.4f}"
    )
```

45. Evaluation

Sau khi huấn luyện:

`</>` Python

```
model.eval()

with torch.no_grad():

    output = model(X_test)
```

`model.eval()` chuyển mô hình sang chế độ đánh giá.

`torch.no_grad()` giúp không tính gradient trong quá trình test.

Điều này giúp tiết kiệm bộ nhớ và tăng tốc quá trình đánh giá.

46. Dự đoán lớp

Với classification:

```
</> Python

logits = model(X_test)

pred = torch.argmax(
    logits,
    dim=1
)
```

Ví dụ:

logits:

[1.2, 3.4, 0.5]

Lớp được dự đoán:

Class 1

vì:

3.4

là giá trị lớn nhất.

47. Accuracy

Công thức:

$$Accuracy = \frac{\text{Number of correct predictions}}{\text{Total predictions}}$$

Code:

```
</> Python

correct = (pred == y_test).sum().item()

accuracy = correct / len(y_test)

print("Accuracy:", accuracy)
```


48. Softmax

Đối với classification nhiều lớp, logits có thể chuyển thành xác suất bằng:

$$P(y = i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

PyTorch:

```
</> Python

prob = torch.softmax(
    logits,
    dim=1
)
```

Ví dụ:

[1.2, 3.4, 0.5]

có thể chuyển thành:

[0.09, 0.86, 0.05]

49. Một mô hình RNN hoàn chỉnh

Dưới đây là code tương đối đầy đủ để làm nền cho các bài thực hành.

</> Python

```
import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader

# =====
# 1. MODEL
# =====

class RNNClassifier(nn.Module):

    def __init__(
        self,
        input_size,
        hidden_size,
        num_layers,
        num_classes,
        dropout=0.0
    ):
        super().__init__()

        self.rnn = nn.RNN(
            input_size=input_size,
            hidden_size=hidden_size,
            num_layers=num_layers,
            batch_first=True,
            dropout=dropout if num_layers > 1 else 0
        )

        self.fc = nn.Linear(
            hidden_size,
            num_classes
        )
```

```
def forward(self, x):  
  
    # x:  
    # [batch, sequence, feature]  
  
    output, hidden = self.rnn(x)  
  
    # hidden:  
    # [num_layers, batch, hidden_size]  
  
    last_hidden = hidden[-1]  
  
    # [batch, hidden_size]  
    logits = self.fc(last_hidden)  
  
    # [batch, num_classes]  
    return logits
```

```
# =====  
# 2. CREATE DATA  
# =====  
  
X = torch.randn(  
    1000,  
    20,  
    5  
)  
  
y = torch.randint(  
    0,  
    3,  
    (1000,)  
)  
  
# =====  
# 3. DATALOADER  
# =====  
  
dataset = TensorDataset(X, y)  
  
loader = DataLoader(  
    dataset,  
    batch_size=32,  
    shuffle=True  
)
```

```
# =====  
# 4. MODEL  
# =====  
  
model = RNNClassifier(  
    input_size=5,  
    hidden_size=64,  
    num_layers=2,  
    num_classes=3,  
    dropout=0.2  
)  
  
# =====  
# 5. LOSS  
# =====  
  
criterion = nn.CrossEntropyLoss()  
  
# =====  
# 6. OPTIMIZER  
# =====  
  
optimizer = torch.optim.Adam(  
    model.parameters(),  
    lr=0.001  
)
```

```

# =====
# 7. TRAINING
# =====

epochs = 20

for epoch in range(epochs):

    model.train()

    total_loss = 0
    correct = 0
    total = 0

    for batch_X, batch_y in loader:

        # Clear gradient
        optimizer.zero_grad()

        # Forward
        logits = model(batch_X)

        # Loss
        loss = criterion(
            logits,
            batch_y
        )

        # Backward
        loss.backward()

        # Prevent exploding gradient
        torch.nn.utils.clip_grad_norm_(
            model.parameters(),
            max_norm=1.0
        )

        # Update
        optimizer.step()

        # Statistics
        total_loss += loss.item()

        pred = torch.argmax(
            logits,
            dim=1
        )

        correct += (
            pred == batch_y
        ).sum().item()

        total += batch_y.size(0)

    avg_loss = (
        total_loss / len(loader)
    )

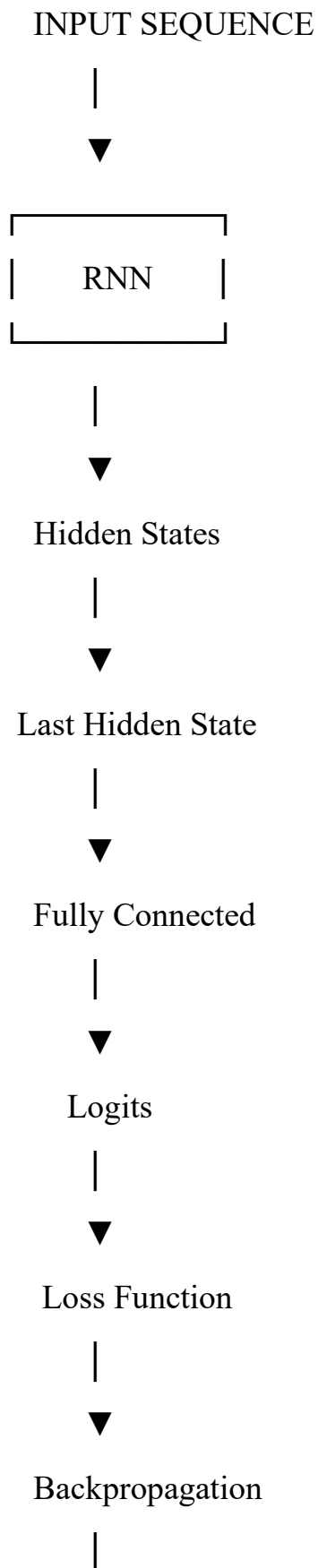
    accuracy = (
        correct / total
    )

    print(
        f"Epoch [{epoch+1}/{epochs}] "
        f"Loss: {avg_loss:.4f} "
        f"Accuracy: {accuracy:.4f}"
    )

```

50. Luồng hoạt động của toàn bộ RNN

Có thể tóm tắt toàn bộ quá trình như sau:





Gradient Clipping



Optimizer

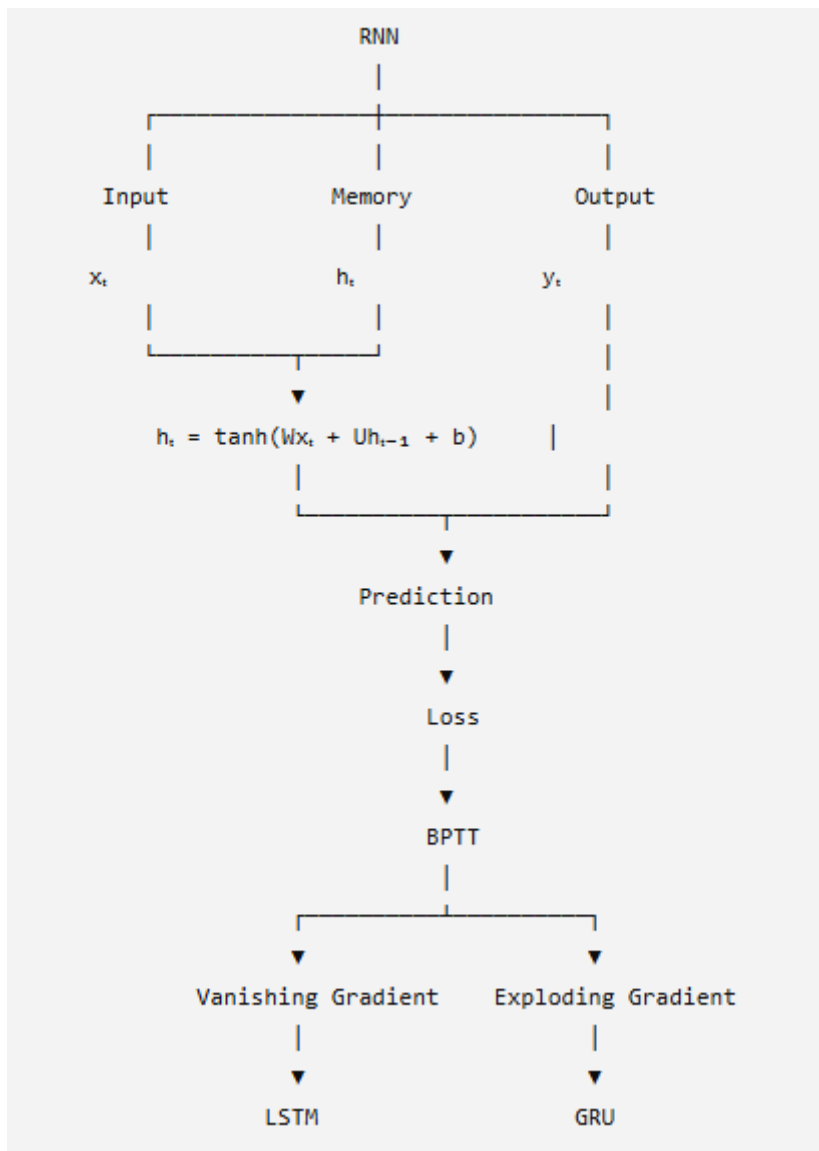


Update Weights

31..

51. Sơ đồ kiến trúc RNN cần ghi nhớ

Có thể dùng sơ đồ sau để hệ thống hóa kiến thức:



52. Kết luận về RNN

RNN là một kiến trúc mạng nơ-ron được thiết kế để xử lý dữ liệu có tính tuần tự bằng cách duy trì **hidden state** qua các bước thời gian. Tại mỗi thời điểm, RNN kết hợp dữ liệu đầu vào hiện tại với hidden state của bước trước để tạo ra hidden state mới. Nhờ cơ chế này, mô hình có thể khai thác thông tin từ các phần tử trước đó của chuỗi thay vì xử lý từng phần tử hoàn toàn độc lập.

Công thức cốt lõi của RNN là:

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

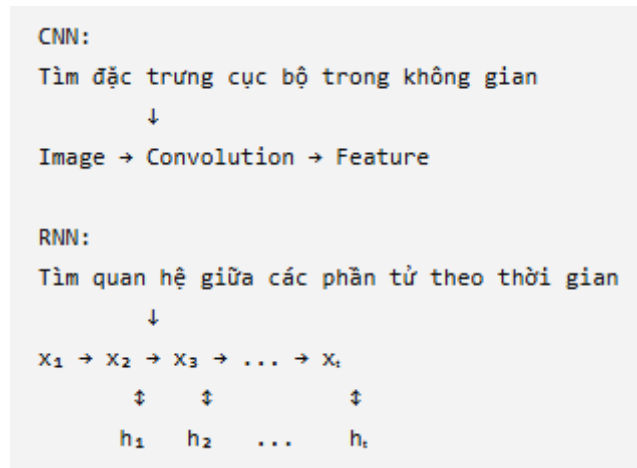
và:

$$y_t = W_{hy}h_t + b_y$$

Trong quá trình huấn luyện, RNN sử dụng **Backpropagation Through Time (BPTT)** để tính gradient qua các bước thời gian. Tuy nhiên, RNN truyền thống

có hạn chế về **vanishing gradient**, **exploding gradient** và khả năng ghi nhớ các phụ thuộc dài hạn. Những hạn chế này là động lực dẫn đến sự phát triển của các kiến trúc cải tiến như **LSTM** và **GRU**.

Nếu đặt RNN cạnh CNN mà bạn đang học, có thể nhớ bản chất rất ngắn gọn:



Phần II. Phân tích dataset

1. DJIA 30 Stock Time Series

Link: [Learn by example RNN/LSTM/GRU time series](#)

1.1. Nguồn gốc và bản chất

Bộ dữ liệu DJIA 30 Stock Time Series trên Kaggle chứa dữ liệu lịch sử của 30 công ty thuộc chỉ số Dow Jones Industrial Average trong giai đoạn từ ngày 01/01/2006 đến 01/01/2018. Đây là **dữ liệu thị trường thực**, khác với bộ dữ liệu 1. Tập `all_stocks_2006-01-01_to_2018-01-01.csv` gộp dữ liệu của cả 30 mã trong một bảng, và bộ dữ liệu còn có thêm tập riêng cho năm 2017. Notebook “Learn by example RNN/LSTM/GRU time series” sử dụng dữ liệu này để minh họa cách xây dựng mô hình RNN, LSTM và GRU cho dự báo chuỗi thời gian.

1.2. Cấu trúc và mô tả thuộc tính

Mỗi dòng ứng với một phiên giao dịch của một mã cổ phiếu. Tập gộp có bảy cột và khoảng 93.600 bản ghi, tương ứng 30 mã với khoảng 3.000 phiên mỗi mã. Một số mã có ít phiên hơn do niêm yết muộn trong giai đoạn khảo sát.

Thuộc tính	Ý nghĩa	Kiểu dữ liệu	Nhận xét
Date	Ngày giao dịch	Ngày	Trục thời gian; chỉ gồm ngày làm việc của thị trường
Open	Giá mở cửa	Số thực	Tương quan rất cao với các cột giá còn lại
High	Giá cao nhất trong phiên	Số thực	Phản ánh biên độ dao động trong ngày
Low	Giá thấp nhất trong phiên	Số thực	Phản ánh biên độ dao động trong ngày
Close	Giá đóng cửa	Số thực	Biến mục tiêu phổ biến nhất
Volume	Khối lượng giao dịch	Số nguyên	Phân phối lệch phải, cần biến đổi logarit
Name	Mã cổ phiếu (ví dụ AAPL, IBM, MSFT)	Phân loại	Dùng để tách thành 30 chuỗi riêng biệt

1.3. Đặc điểm của chuỗi dữ liệu

- **Tính không dừng.** Giá cổ phiếu có xu hướng dài hạn và phương sai thay đổi theo thời gian. Việc huấn luyện trực tiếp trên giá thô khiến mô hình học xu hướng thay vì quan hệ động, và gây khó khăn khi giá ở giai đoạn kiểm thử vượt ra ngoài khoảng giá của giai đoạn huấn luyện. Giải pháp thường dùng là dự báo lợi suất logarit $r_t = \ln(P_t / P_{t-1})$ hoặc sai phân giá.
- **Tự tương quan rất mạnh ở mức giá.** Giá hôm nay gần như bằng giá hôm qua, do đó mô hình cơ sở “giá ngày mai bằng giá hôm nay” đã cho sai số rất nhỏ. Đây là mớic tham chiếu bắt buộc khi đánh giá RNN.
- **Tự tương quan yếu ở mức lợi suất.** Chuỗi lợi suất hàng ngày gần với nhiễu ngẫu nhiên, nên việc dự báo chính xác là rất khó. Ngược lại, độ lớn của lợi suất (biến động) thể hiện hiện tượng tụ cụm, tức các giai đoạn biến động mạnh thường đi liền nhau.
- **Thay đổi chế độ thị trường.** Giai đoạn 2008–2009 chứa cuộc khủng hoảng tài chính, trong khi giai đoạn 2013–2017 có xu hướng tăng ổn định với biến

động thấp. Khi chia theo thời gian, tập kiểm thử sẽ có phân phối khác tập huấn luyện.

- **Đa cộng tuyến giữa Open, High, Low, Close.** Bốn cột giá này gần như dư thừa thông tin của nhau. Khi dùng nhiều đặc trưng nên bổ sung các thông tin có nội dung khác như Volume, biên độ (High – Low) hoặc các chỉ báo kỹ thuật.
- **Chất lượng dữ liệu.** Dữ liệu thực ít giá trị thiếu. Cần kiểm tra các mã có số phiên ít hơn, các ngày thiếu và các giá trị bất thường; chỉ nên điền giá trị bằng phương pháp lan truyền tiến (forward fill) khi thật sự cần.

1.4. Đánh giá mức độ phù hợp với bài toán RNN

Đây là bộ dữ liệu có cấu trúc chuỗi thời gian tự nhiên: quan sát đều đặn theo ngày giao dịch, có thứ tự thời gian rõ ràng, nhiều đặc trưng đồng thời và 30 chuỗi song song để so sánh. Vì vậy nó rất phù hợp để minh họa và so sánh RNN, LSTM và GRU. Cần lưu ý rằng việc phù hợp về mặt cấu trúc không đồng nghĩa với khả năng dự báo tốt: thị trường tài chính có nhiễu cao và theo giả thuyết thị trường hiệu quả, thông tin quá khứ chứa rất ít khả năng dự báo lợi suất tương lai. Do đó, mục tiêu hợp lý của bài toán là so sánh các kiến trúc, quan sát hành vi huấn luyện và đánh giá trung thực, không phải xây dựng hệ thống giao dịch.

1.5. Thiết kế bài toán

Nội dung	Đề xuất
Bài toán	Dự báo giá đóng cửa (hoặc lợi suất) của phiên kế tiếp; mở rộng dự báo nhiều bước hoặc phân loại hướng tăng giảm
Phạm vi	Giai đoạn đầu chọn một mã (ví dụ IBM hoặc AAPL); giai đoạn sau huấn luyện chung trên nhiều mã
Đặc trưng	Đơn biến: Close. Đa biến: Open, High, Low, Close, Volume (Volume nên lấy logarit)
Cửa sổ đầu vào	Ví dụ 60 phiên gần nhất, mỗi mẫu là tensor $60 \times$ số đặc trưng
Chia dữ liệu	Theo thời gian: khoảng 70% đầu để huấn luyện, 15% tiếp theo để kiểm định, 15% cuối để kiểm thử
Chuẩn hóa	MinMaxScaler hoặc chuẩn hóa z-score, chỉ ước lượng trên tập huấn luyện

Nội dung	Đề xuất
Mô hình	SimpleRNN, LSTM, GRU cùng số đơn vị ẩn (ví dụ 50–64), dropout 0,2, tối ưu Adam, hàm mất mát MSE, dừng sớm theo tập kiểm định
Mô hình cơ sở	Giá liền trước (naive); trung bình trượt; có thể thêm ARIMA
Thước đo	RMSE, MAE, MAPE và độ chính xác hướng (directional accuracy)

1.6. Những rủi ro cần kiểm soát

1. **Rò rỉ tương lai:** không xáo trộn dữ liệu khi chia tập; không ước lượng tham số chuẩn hóa trên toàn bộ dữ liệu.
2. **Ảo giác về độ chính xác:** đồ thị giá dự báo thường trông rất khớp vì mô hình chủ yếu lặp lại giá hôm trước. Cần so sánh với mô hình cơ sở và xem xét độ chính xác hướng.
3. **Vượt khoảng giá huấn luyện:** khi dùng giá thô, giá kiểm thử có thể vượt giá trị lớn nhất của tập huấn luyện và làm mô hình mất khả năng dự báo. Dự báo lợi suất giúp giảm vấn đề này.
4. **Phụ thuộc vào khởi tạo:** kết quả của mạng nơ-ron thay đổi theo hạt giống ngẫu nhiên; nên lặp lại nhiều lần và báo cáo giá trị trung bình cùng độ lệch chuẩn.

Không nên suy diễn sang giao dịch thực: kết quả chưa tính chi phí giao dịch, trượt giá và thay đổi chế độ thị trường.

2. E-commerce Customer Data For Behavior Analysis

Link: [🛒 E-commerce Customer Data For Behavior Analysis](#)

2.1. Nguồn gốc và bản chất

Bộ dữ liệu do tác giả Shriyash Jagtap công bố trên Kaggle vào tháng 9 năm 2023 với mô tả khám phá thói quen mua sắm, hiện tượng rời bỏ và mẫu hình mua hàng của khách hàng. Theo mô tả thuộc tính, các trường tên, tuổi và giới tính của khách hàng được sinh tự động bằng thư viện Faker. Vì vậy đây là **dữ liệu tổng hợp**: các mối quan hệ trong dữ liệu là kết quả của quy trình sinh chứ không phản ánh hành vi tiêu dùng thực tế. Bộ dữ liệu phù hợp để học tập và thử nghiệm kỹ thuật, nhưng các kết luận về hành vi khách hàng thật cần được xem xét thận trọng.

2.2. Cấu trúc và mô tả thuộc tính

Đơn vị quan sát là **một giao dịch**, không phải một khách hàng: cùng một mã khách hàng có thể xuất hiện ở nhiều dòng với các ngày mua khác nhau. Bộ dữ liệu có quy mô khoảng 250.000 bản ghi, trải dài khoảng ba năm rưỡi kể từ đầu năm 2020. Các thuộc tính được tổng hợp trong Bảng 1.

Thuộc tính	Ý nghĩa	Kiểu dữ liệu	Vai trò đối với bài toán RNN
Customer ID	Mã định danh khách hàng	Định danh	Khóa để nhóm các giao dịch thành chuỗi theo khách
Purchase Date	Ngày phát sinh giao dịch	Ngày giờ	Trục thời gian, dùng để sắp xếp và tổng hợp
Product Category	Nhóm sản phẩm được mua	Phân loại	Đặc trưng đầu vào (mã hóa one-hot hoặc embedding)
Product Price	Đơn giá sản phẩm	Số thực	Đặc trưng số
Quantity	Số lượng sản phẩm	Số nguyên	Đặc trưng số
Total Purchase Amount	Tổng tiền của giao dịch	Số thực	Đặc trưng; cơ sở để tính doanh thu khi tổng hợp
Payment Method	Phương thức thanh toán	Phân loại	Đặc trưng đầu vào
Returns	Giao dịch có bị trả hàng hay không (0/1)	Nhị phân	Đặc trưng; có thể có giá trị thiếu
Customer Age (và Age)	Tuổi của khách hàng	Số nguyên	Đặc trưng tĩnh; hai cột gần như trùng nhau
Gender	Giới tính	Phân loại	Đặc trưng tĩnh
Customer Name	Tên khách hàng (do Faker sinh)	Văn bản	Không mang thông tin, loại bỏ khi mô hình hóa
Churn	Khách hàng đã rời bỏ (1) hay còn hoạt động (0)	Nhị phân	Nhãn mục tiêu cho bài toán phân loại

2.3. Đặc điểm và chất lượng dữ liệu

Trước khi mô hình hóa cần lưu ý các đặc điểm sau, mỗi đặc điểm đều cần được kiểm tra bằng mã ở Phụ lục A:

- **Nhiều dòng trên mỗi khách hàng.** Nếu chia tập dữ liệu ngẫu nhiên theo dòng, các giao dịch của cùng một khách hàng sẽ nằm ở cả tập huấn luyện lẫn kiểm thử, gây rò rỉ thông tin và làm kết quả bị lạc quan giả tạo.
- **Nhãn Churn ở cấp khách hàng nhưng lưu ở cấp giao dịch.** Cần kiểm tra nhãn có nhất quán giữa các giao dịch của cùng một khách hay không. Nếu không nhất quán, phải đặt quy tắc gộp nhãn rõ ràng. Ngoài ra, tiêu chí gán nhãn rời bỏ không được tác giả mô tả rõ, nên ý nghĩa nghiệp vụ của nhãn còn hạn chế.
- **Mất cân bằng lớp.** Tỷ lệ khách hàng rời bỏ thường thấp hơn đáng kể so với khách hàng còn hoạt động; cần dùng các thước đo như AUC, Precision, Recall, F1 thay vì chỉ dùng độ chính xác.
- **Giá trị thiếu và cột dư thừa.** Cột Returns có thể chứa giá trị thiếu, và có hai cột tuổi gần như trùng nhau. Cần thống nhất cách xử lý (điền giá trị, tạo cờ thiếu hoặc loại bỏ cột).
- **Tính nhất quán của số liệu tiền tệ.** Cần đối chiếu Total Purchase Amount với tích của Product Price và Quantity để xác định cột nào đáng tin cậy làm cơ sở tính doanh thu.
- **Tính nhân tạo của dữ liệu.** Do được sinh ngẫu nhiên, các đặc trưng có thể ít liên hệ với nhãn và với thời gian. Đây là điều cần kiểm chứng bằng thực nghiệm, không nên giả định.

2.4. Đánh giá mức độ phù hợp với bài toán RNN

Về bản chất đây là bộ dữ liệu dạng bảng, không phải chuỗi thời gian có sẵn. Tuy nhiên, sự hiện diện của cột Purchase Date cho phép biến đổi dữ liệu thành dạng tuần tự theo hai hướng.

Hướng A. Chuỗi thời gian tổng hợp theo ngày (khuyến nghị làm hướng chính)

Gộp toàn bộ giao dịch theo ngày để tạo chuỗi đa biến gồm các đại lượng như tổng doanh thu, số giao dịch, giá trị đơn hàng trung bình và tỷ lệ trả hàng. Bài toán là dự báo doanh thu của ngày (hoặc tuần) kế tiếp dựa trên cửa sổ các ngày trước đó, ví dụ cửa sổ 30 ngày. Đây là bài toán hồi quy chuỗi thời gian đa biến (many-to-

one) rất phù hợp với LSTM và GRU. Điều kiện tiên quyết là chuỗi phải có cấu trúc thời gian, do đó cần phân rã chuỗi và vẽ hàm tự tương quan (ACF, PACF) để kiểm tra xu hướng và tính mùa vụ theo tuần, tháng. Nếu chuỗi gần với nhiễu ngẫu nhiên, mô hình RNN sẽ không vượt được mô hình cơ sở; khi đó kết quả này vẫn là một phát hiện hợp lệ cần được báo cáo trung thực.

Hướng B. Chuỗi giao dịch theo từng khách hàng

Với mỗi khách hàng, sắp xếp các giao dịch theo thời gian để tạo một chuỗi có độ dài thay đổi, mỗi phần tử gồm các đặc trưng như nhóm sản phẩm, giá, số lượng, phương thức thanh toán, cở trả hàng và khoảng cách ngày so với giao dịch trước. Nhãn Churn của khách hàng là đầu ra, tạo thành bài toán phân loại chuỗi (many-to-one). Cần dùng padding kết hợp masking (hoặc đóng gói chuỗi) để xử lý độ dài khác nhau. Hướng này có ý nghĩa nghiệp vụ hơn nhưng đòi hỏi nhãn nhất quán và mỗi khách hàng phải có đủ số giao dịch; nếu phần lớn chuỗi rất ngắn thì lợi thế của RNN so với mô hình trên đặc trưng tổng hợp là không rõ ràng.

2.5. Quy trình tiền xử lý đề xuất

1. Đọc dữ liệu, chuyển Purchase Date sang kiểu ngày giờ, loại bỏ bản ghi trùng lặp và cột không mang thông tin.
2. Kiểm tra và xử lý giá trị thiếu; đổi chiều tính nhất quán của các cột tiền tệ và cột tuổi.
3. Hướng A: tổng hợp theo ngày, bổ sung các ngày không có giao dịch bằng giá trị 0 để chuỗi đều đặn; tạo các đặc trưng lịch (thứ trong tuần, tháng).
4. Hướng B: nhóm theo Customer ID, sắp xếp theo thời gian, mã hóa các biến phân loại, tính khoảng cách giữa các lần mua.
5. Chia dữ liệu: Hướng A chia theo trình tự thời gian; Hướng B chia theo khách hàng (GroupKFold hoặc chia theo tập mã khách hàng) và giữ nguyên tỷ lệ nhãn.
6. Chuẩn hóa đặc trưng số bằng tham số ước lượng trên tập huấn luyện; tạo các cửa sổ trượt hoặc chuỗi đệm.

2.6. Phương án mô hình và đánh giá

Nội dung	Hướng A: dự báo doanh thu	Hướng B: dự đoán churn
Loại bài toán	Hồi quy chuỗi thời gian đa biến	Phân loại chuỗi nhị phân
Đầu vào	Cửa sổ 30 ngày × các đặc trưng theo ngày	Chuỗi giao dịch của một khách (đã đệm)
Đầu ra	Doanh thu ngày kế tiếp	Xác suất rời bỏ
Mô hình	RNN, LSTM, GRU cùng kích thước để so sánh công bằng	LSTM hoặc GRU, có lớp Masking
Mô hình cơ sở	Giá trị ngày liền trước; trung bình trượt 7 ngày	Logistic Regression trên đặc trưng tổng hợp
Thước đo	RMSE, MAE, MAPE	AUC, Precision, Recall, F1

2.7. Hạn chế

Hạn chế lớn nhất là dữ liệu tổng hợp, có thể thiếu cấu trúc thời gian thực chất. Bên cạnh đó, nhãn Churn chưa có định nghĩa rõ ràng, và bộ dữ liệu không chứa các tín hiệu hành vi phong phú như lượt truy cập hay thời gian trên ứng dụng. Do đó bộ dữ liệu này phù hợp hơn cho mục đích rèn luyện quy trình xây dựng dữ liệu tuần tự từ dữ liệu giao dịch so với việc rút ra kết luận kinh doanh.

3. So sánh hai bộ dữ liệu

Tiêu chí	E-commerce Customer Data	DJIA 30 Stock Time Series
Nguồn gốc	Tổng hợp nhân tạo	Dữ liệu thị trường thực
Cấu trúc gốc	Bảng giao dịch, một dòng một giao dịch	Chuỗi thời gian, một dòng một phiên giao dịch
Quy mô	Khoảng 250.000 bản ghi	Khoảng 93.600 bản ghi, 30 mã
Tính tuần tự	Không có sẵn, phải xây dựng bằng tổng hợp theo ngày hoặc theo khách	Có sẵn, đều đặn theo ngày giao dịch
Bài toán RNN	Dự báo doanh thu; phân loại churn theo chuỗi	Dự báo giá hoặc lợi suất; phân loại hướng

Tiêu chí	E-commerce Customer Data	DJIA 30 Stock Time Series
Mức độ sẵn sàng cho RNN	Cần nhiều bước biến đổi dữ liệu	Gần như sẵn sàng, chủ yếu cần chuẩn hóa và tạo cửa sổ
Rủi ro chính	Thiếu cấu trúc thời gian; nhãn chưa rõ; rò rỉ giữa các dòng cùng khách	Không dừng; rò rỉ tương lai; ảo giác do tự tương quan
Cách chia dữ liệu	Theo thời gian (hướng A) hoặc theo khách hàng (hướng B)	Theo thời gian
Giá trị học tập	Rèn luyện kỹ năng xây dựng chuỗi từ dữ liệu bảng	So sánh RNN, LSTM, GRU trên chuỗi thời gian thực

Phần III. Thực hành

1. BÀI TOÁN 1 – DỰ BÁO DOANH THU E-COMMERCE

Source code: [Hunguyenobugs/Phat-trien-cac-he-thong-thong-minh](https://github.com/Hunguyenobugs/Phat-trien-cac-he-thong-thong-minh)

1.1. Phiên bản Keras (TensorFlow)

Dữ liệu: **E-commerce Customer Data For Behavior Analysis** (Kaggle) – bảng giao dịch, mỗi dòng là một giao dịch.

Ý tưởng: dữ liệu gốc là dạng bảng nên trước hết **tổng hợp theo ngày** thành chuỗi thời gian đa biến (doanh thu, số đơn, giá trị đơn trung bình, tỷ lệ trả hàng, đặc trưng lịch). Sau đó dùng cửa sổ trượt WINDOW ngày để dự đoán các đại lượng của **ngày kế tiếp**, và so sánh ba kiến trúc RNN, LSTM, GRU với hai mô hình cơ sở (Naive, trung bình 7 ngày).

Quy trình: 1) Nạp và kiểm tra dữ liệu → 2) Phân tích khám phá (EDA) → 3) Tạo chuỗi thời gian, kiểm tra cấu trúc → 4) Chia theo thời gian, chuẩn hóa, tạo cửa sổ → 5) Baseline → 6) Huấn luyện 3 mô hình → 7) Đánh giá và trực quan hóa → 8) Phân tích đặc trưng → 9) Dự báo tương lai → 10) Kết luận.

a) Thư viện và cấu hình

Cell 2: Nhập thư viện và thiết lập hiển thị

Code

```
import os, glob, math, random, warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display
from sklearn.preprocessing import StandardScaler

warnings.filterwarnings("ignore")
plt.rcParams.update({
    "figure.dpi": 100, "figure.figsize": (11, 4),
    "axes.grid": True, "grid.alpha": 0.3,
    "axes.spines.top": False, "axes.spines.right": False,
    "axes.titlesize": 12, "axes.titleweight": "bold",
})
pd.set_option("display.float_format", lambda v: f"{v:,.4f}")
pd.set_option("display.max_columns", 30)
```

Cell này không tạo output hiển thị.

Giải thích code

Nhập các thư viện nền tảng và đặt cấu hình chung cho toàn bộ notebook.

os, glob, math, random, warnings là thư viện chuẩn (đường dẫn, tìm tệp, toán học, số ngẫu nhiên, tắt cảnh báo). numpy và pandas xử lý mảng/bảng, matplotlib vẽ biểu đồ, display giúp hiển thị DataFrame đẹp trong Jupyter. StandardScaler của scikit-learn dùng để chuẩn hóa dữ liệu về trung bình 0 và độ lệch chuẩn 1.

plt.rcParams.update(...) đặt style chung cho mọi biểu đồ: 100 dpi, kích thước mặc định 11×4 inch, bật lưới mờ, bỏ viền trên và phải, tiêu đề đậm cỡ 12. pd.set_option quy định số thực hiển thị có dấu phân cách hàng nghìn và 4 chữ số thập phân, đồng thời cho phép hiển thị tối đa 30 cột; đó là lý do các bảng về sau có dạng 2,725.3852.

Cell 3: Khai báo tham số cấu hình

Code

```
# ===== CẤU HÌNH (có thể chỉnh) =====
DATA_PATH = "../data/ecommerce_customer_data_large.csv" # <-- đường dẫn tới tệp CSV đã tải về
WINDOW    = 30      # số ngày lịch sử đưa vào RNN
HORIZON    = 14     # số ngày dự báo tương lai
TRAIN_RATIO = 0.70   # tỷ lệ tập huấn luyện (theo thời gian)
VAL_RATIO  = 0.15    # tỷ lệ tập kiểm định; phần còn lại là tập kiểm thử

UNITS      = 64      # số đơn vị ẩn mỗi lớp
DROPOUT    = 0.2     # tỷ lệ bỏ ngẫu nhiên
LR          = 1e-3    # tốc độ học
BATCH_SIZE = 32
EPOCHS     = int(os.environ.get("NB_EPOCHS", 100)) # tối đa; có dừng sớm nên thường ít hơn
PATIENCE   = 12
SEED       = 42
```

Cell này không tạo output hiển thị.

Giải thích code

Gom toàn bộ siêu tham số vào một chỗ để dễ chỉnh sửa.

DATA_PATH là đường dẫn tới tệp CSV. WINDOW = 30 nghĩa là mỗi mẫu đưa vào mạng gồm 30 ngày lịch sử liên tiếp. HORIZON = 14 là số ngày dự báo tương lai ở phần 9. TRAIN_RATIO = 0,70 và VAL_RATIO = 0,15 nên tập kiểm thử chiếm 15% còn lại (chia theo thứ tự thời gian).

UNITS = 64 là số nơ-ron ẩn mỗi lớp RNN; DROPOUT = 0,2 là tỷ lệ bỏ ngẫu nhiên nơ-ron để chống quá khớp; LR = 1e-3 là tốc độ học của Adam; BATCH_SIZE = 32. EPOCHS đọc từ biến môi trường NB_EPOCHS (mặc định 100) nhưng thực tế có dừng sớm sau PATIENCE = 12 epoch không cải thiện. SEED = 42 giúp kết quả lặp lại được.

Cell 4: Nạp TensorFlow/Keras, cố định seed, kiểm tra thiết bị

Code

```
os.environ.setdefault("TF_CPP_MIN_LOG_LEVEL", "2")
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

random.seed(SEED); np.random.seed(SEED); tf.random.set_seed(SEED)
print("TensorFlow:", tf.__version__, "| Keras:", keras.__version__)
print("Thiết bị:", [d.name for d in tf.config.list_physical_devices()])
```

Output

```
TensorFlow: 2.21.0 | Keras: 3.15.1
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUD
A/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
Thiết bị: ['/physical_device:CPU:0']
```

Giải thích code

Nạp TensorFlow và Keras, cố định hạt giống ngẫu nhiên và in phiên bản cùng thiết bị tính toán.

`os.environ.setdefault("TF_CPP_MIN_LOG_LEVEL", "2")` giảm bớt thông điệp log của TensorFlow. `random.seed`, `np.random.seed` và `tf.random.set_seed` cùng đặt SEED để kết quả tái lập. Dòng print cuối liệt kê các thiết bị mà TensorFlow nhìn thấy.

Giải thích output

Máy chạy TensorFlow 2.21.0 và Keras 3.15.1.

Dòng cảnh báo cho biết từ TensorFlow 2.11 trở đi, bản chạy native trên Windows không hỗ trợ GPU (muốn dùng GPU phải chạy WSL2 hoặc DirectML). Vì vậy danh sách thiết bị chỉ có CPU:0, toàn bộ huấn luyện Keras của Bài 1 chạy trên CPU. Đây là lý do thời gian mỗi epoch của Keras dài hơn bản PyTorch chạy trên CUDA.

Cell 5: Hàm tiện ích: tìm tệp dữ liệu và tự tương quan (ACF)

Code

```
def find_data_file(path, pattern):
    """Trả về đường dẫn tệp dữ liệu. Nếu DATA_PATH không tồn tại thì tự tìm theo mẫu tên trong thư mục hiện tại."""
    if os.path.exists(path):
        return path
    found = glob.glob(pattern) + glob.glob(os.path.join("...", pattern), recursive=True)
    found = sorted(set(found), key=lambda p: ("large" not in p.lower(), len(p), p))
    if not found:
        raise FileNotFoundError(
            f"Không tìm thấy '{path}'. Hãy sửa biến DATA_PATH ở ô cấu hình thành đường dẫn tới tệp CSV đã tải về.")
    print(f"Không thấy '{path}', tự động dùng: {found[0]}")
    return found[0]

def acf(x, nlags):
    """Hàm tự tương quan (ACF) tính trực tiếp bằng numpy."""
    x = np.asarray(x, dtype=float)
    x = x - x.mean()
    d = np.dot(x, x)
    return np.array([np.dot(x[:len(x) - k], x[k:]) / d for k in range(nlags + 1)])

def plot_acf(ax, x, nlags, title):
    a = acf(x, nlags)
    conf = 1.96 / np.sqrt(len(x))
    ax.bar(range(nlags + 1), a, width=0.3, color="#2F5496")
    ax.axhline(conf, ls="--", c="r", lw=1); ax.axhline(-conf, ls="--", c="r", lw=1)
    ax.axhline(0, c="k", lw=0.8)
    ax.set_title(title); ax.set_xlabel("Độ trễ (lag)"); ax.set_ylabel("ACF")
    return a, conf
```

Cell này không tạo output hiển thị.

Giải thích code

Định nghĩa ba hàm dùng lại nhiều lần: `find_data_file`, `acf` và `plot_acf`.

`find_data_file`: nếu `DATA_PATH` không tồn tại thì tìm các tệp khớp mẫu (ví dụ `ecommerce*.csv`) trong thư mục hiện tại và các thư mục con, ưu tiên tệp có chữ 'large' và tên ngắn; nếu không thấy thì báo lỗi rõ ràng.

`acf`: tính hàm tự tương quan bằng numpy. Trừ trung bình, rồi với mỗi độ trễ `k` tính tích vô hướng giữa chuỗi và chính nó dịch `k` bước, chia cho tổng bình phương (phương sai). Kết quả lag 0 luôn bằng 1.

`plot_acf`: vẽ ACF dạng cột và hai đường đứt đỏ là ngưỡng ý nghĩa $\pm 1.96/\sqrt{n}$ (khoảng tin cậy 95% nếu chuỗi là nhiễu trắng). Cột nào vượt đường đỏ mới được coi là tự tương quan có ý nghĩa thống kê. Hàm trả về mảng ACF và ngưỡng để dùng ở cell 14.

b) Nạp dữ liệu và kiểm tra chất lượng

Cell 7: Nạp dữ liệu và kiểm tra ban đầu

Code

```
DATA_PATH = find_data_file(DATA_PATH, "ecommerce*.csv")
df = pd.read_csv(DATA_PATH)
df.columns = df.columns.str.strip()

required = ["Purchase Date", "Total Purchase Amount"]
missing_cols = [c for c in required if c not in df.columns]
assert not missing_cols, f"Thiếu cột bắt buộc: {missing_cols}. Các cột hiện có: {list(df.columns)}"

df["Purchase Date"] = pd.to_datetime(df["Purchase Date"], errors="coerce")
df["Total Purchase Amount"] = pd.to_numeric(df["Total Purchase Amount"], errors="coerce")
if "Returns" in df.columns:
    df["Returns"] = pd.to_numeric(df["Returns"], errors="coerce")
n_bad = df["Purchase Date"].isna().sum() + df["Total Purchase Amount"].isna().sum()
df = df.dropna(subset=["Purchase Date", "Total Purchase Amount"]).reset_index(drop=True)

print(f"Số dòng (giao dịch): {len(df):,} | Số cột: {df.shape[1]}")
print(f"Khoảng thời gian: {df['Purchase Date'].min().date()} -> {df['Purchase Date'].max().date()}")
if "Customer ID" in df.columns:
    print(f"Số khách hàng duy nhất: {df['Customer ID'].nunique():,}")
print(f"Dòng bị loại vì thiếu ngày/số tiền: {n_bad}")
display(df.head())
display(df.dtypes.to_frame("Kiểu dữ liệu").T)
```

Output

Số dòng (giao dịch): 250,000 | Số cột: 13
Khoảng thời gian: 2020-01-01 -> 2023-09-13
Số khách hàng duy nhất: 49,661
Dòng bị loại vì thiếu ngày/số tiền: 0

	Customer ID	Purchase Date	Product Category	Product Price	Quantity	\
0	44605	2023-05-03 21:30:02	Home	177	1	
1	44605	2021-05-16 13:57:44	Electronics	174	3	
2	44605	2020-07-13 06:16:57	Books	413	1	
3	44605	2023-01-17 13:14:36	Electronics	396	3	
4	44605	2021-05-01 11:29:27	Books	259	4	

	Total Purchase Amount	Payment Method	Customer Age	Returns	Customer Name	\
0	2427	PayPal	31	1.0000	John Rivera	
1	2448	PayPal	31	1.0000	John Rivera	
2	2345	Credit Card	31	1.0000	John Rivera	
3	937	Cash	31	0.0000	John Rivera	
4	2598	PayPal	31	1.0000	John Rivera	

	Age	Gender	Churn
0	31	Female	0
1	31	Female	0
2	31	Female	0
3	31	Female	0
4	31	Female	0

	Customer ID	Purchase Date	Product Category	Product Price	\
Kiểu dữ liệu	int64	datetime64[us]	str	int64	

	Quantity	Total Purchase Amount	Payment Method	Customer Age	\
Kiểu dữ liệu	int64	int64	str	int64	

	Returns	Customer Name	Age	Gender	Churn
Kiểu dữ liệu	float64	str	int64	str	int64

Giải thích code

Đọc CSV, kiểm tra cột bắt buộc, ép kiểu và loại các dòng lỗi.

find_data_file rồi pd.read_csv đọc dữ liệu; df.columns.str.strip() bỏ khoảng trắng thừa ở tên cột. assert bảo đảm có hai cột bắt buộc là Purchase Date và Total Purchase Amount. pd.to_datetime(..., errors="coerce") và pd.to_numeric(..., errors="coerce") ép về kiểu ngày/số, giá trị lỗi thành NaN. Cột Returns (0/1) cũng được ép sang số.

n_bad đếm số ô lỗi, sau đó dropna loại các dòng thiếu ngày hoặc số tiền. Cuối cell in số dòng, số cột, khoảng thời gian, số khách hàng duy nhất; display(df.head()) xem 5 dòng đầu và display(df.dtypes...) xem kiểu dữ liệu từng cột.

Giải thích output

Bảng dữ liệu có 250.000 giao dịch và 13 cột, trải từ 01/01/2020 đến 13/09/2023, thuộc 49.661 khách hàng. Không có dòng nào bị loại (0), tức cột ngày và số tiền sạch hoàn toàn.

Năm dòng đầu đều là khách 44605 (Customer ID) với các đơn ở nhiều ngày khác nhau, cho thấy dữ liệu chưa được sắp xếp theo thời gian, mỗi khách có nhiều giao dịch. Bảng kiểu dữ liệu xác nhận Purchase Date đã là datetime, các cột số là int64/float64 (Returns là float64 vì có giá trị thiếu), các cột văn bản hiển thị là str.

Cell 8: Kiểm tra chất lượng dữ liệu

Code

```
# --- Chất lượng dữ liệu ---
na = df.isna().mean().mul(100).round(2)
print("Tỷ lệ giá trị thiếu theo cột (%):"); display(na[na > 0].to_frame("% thiếu").T if (na > 0).any() else
      "Không có giá trị thiếu")
print("Số dòng trùng lặp hoàn toàn:", df.duplicated().sum())

if {"Product Price", "Quantity"} <= set(df.columns):
    diff = (pd.to_numeric(df["Product Price"], errors="coerce") * pd.to_numeric(df["Quantity"], errors="coerce")
            - df["Total Purchase Amount"]).abs()
    print(f"Tỷ lệ dòng mà Total != Price x Quantity: {(diff > 1e-6).mean():.2%} (nếu cao, cần chọn cột tiền đáng tin)")

if {"Customer ID", "Churn"} <= set(df.columns):
    n_lab = df.groupby("Customer ID")["Churn"].nunique()
    print(f"Khách có nhãn Churn không nhất quán giữa các giao dịch: {(n_lab > 1).mean():.2%}")

display(df["Total Purchase Amount"].describe().to_frame("Total Purchase Amount").T)
```

Output

Tỷ lệ giá trị thiếu theo cột (%):

```

      Returns
% thiếu  18.9500
Số dòng trùng lặp hoàn toàn: 0
Tỷ lệ dòng mà Total != Price x Quantity: 99.98% (nếu cao, cần chọn cột tiền đáng tin)
Khách có nhãn Churn không nhất quán giữa các giao dịch: 0.00%

      count      mean      std      min      25%  \
Total Purchase Amount  250,000.0000  2,725.3852  1,442.5761  100.0000  1,476.0000

      50%      75%      max
Total Purchase Amount  2,725.0000  3,975.0000  5,350.0000
```


Giải thích code

Thống kê giá trị thiếu, dòng trùng, tính nhất quán của cột tiền và nhãn Churn.

na tính tỷ lệ % giá trị thiếu theo từng cột. duplicated().sum() đếm dòng trùng hoàn toàn. Nếu có cả Product Price và Quantity thì so sánh $\text{Price} \times \text{Quantity}$ với Total Purchase Amount để xem cột tổng tiền có được tính từ hai cột kia không. groupby("Customer ID")["Churn"].nunique() kiểm tra một khách có bị gán nhiều nhãn Churn khác nhau không. Cuối cùng describe() thống kê mô tả cột doanh thu.

Giải thích output

Chỉ cột Returns có giá trị thiếu (18,95%). Không có dòng trùng. Đáng chú ý nhất là 99,98% dòng có $\text{Total} \neq \text{Price} \times \text{Quantity}$: cột tổng tiền không suy ra từ đơn giá và số lượng, tức dữ liệu là dữ liệu tổng hợp (sinh ngẫu nhiên). Notebook vì vậy chọn Total Purchase Amount làm doanh thu, đúng như gợi ý trong dòng in. Nhãn Churn nhất quán tuyệt đối theo từng khách (0,00% không nhất quán).

Thống kê Total Purchase Amount: trung bình 2.725, độ lệch chuẩn 1.443, nhỏ nhất 100, lớn nhất 5.350; ba tứ phân vị 1.476 / 2.725 / 3.975 cách đều nhau, gợi ý phân phối gần đều (uniform) trong khoảng 100 đến 5.350 chứ không lệch phải như doanh thu thực tế.

c) Phân tích khám phá (EDA) trên dữ liệu giao dịch

Xem cơ cấu doanh thu, phương thức thanh toán, churn, tuổi và tỷ lệ trả hàng để hiểu bức tranh chung trước khi dựng chuỗi thời gian.

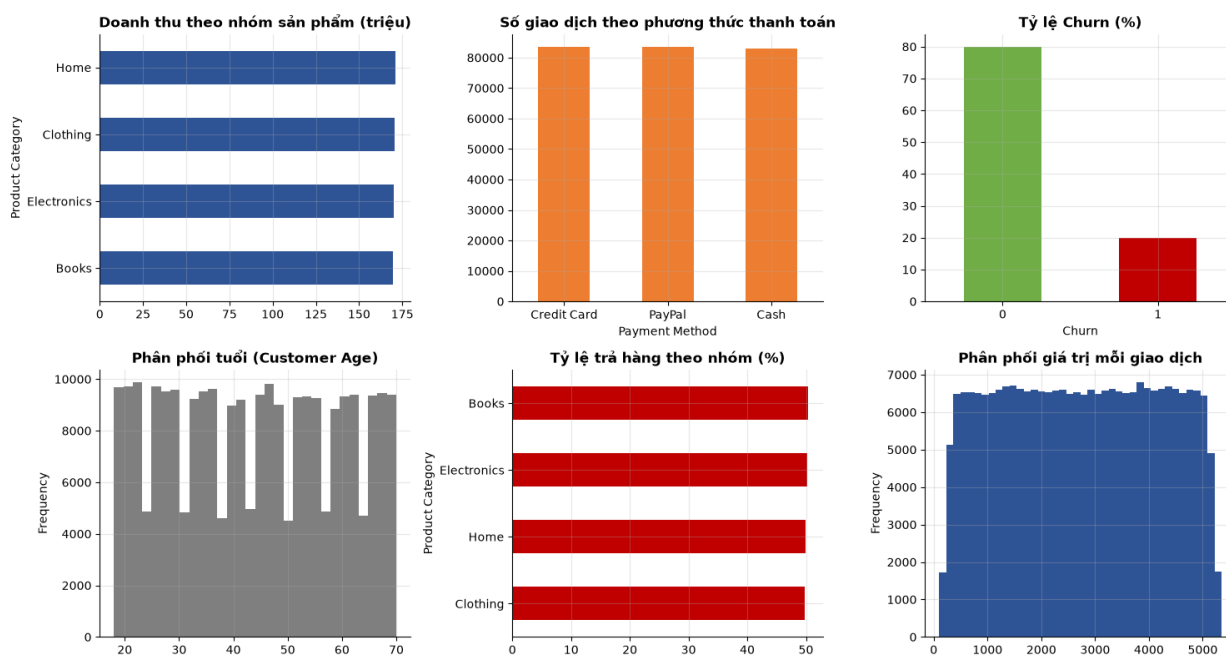
Cell 10: EDA trên dữ liệu giao dịch (6 biểu đồ)

Code

```
# --- Trực quan hóa dữ liệu giao dịch ---
age_col = "Customer Age" if "Customer Age" in df.columns else ("Age" if "Age" in df.columns else None)
panels = []
if "Product Category" in df.columns: panels.append("cat_rev")
if "Payment Method" in df.columns: panels.append("pay")
if "Churn" in df.columns: panels.append("churn")
if age_col: panels.append("age")
if {"Product Category", "Returns"} <= set(df.columns): panels.append("ret")
panels.append("amount")

ncols = 3; nrows = math.ceil(len(panels) / ncols)
fig, axes = plt.subplots(nrows, ncols, figsize=(15, 4 * nrows)); axes = np.atleast_1d(axes).ravel()
for ax, p in zip(axes, panels):
    if p == "cat_rev":
        s = df.groupby("Product Category")["Total Purchase Amount"].sum().sort_values(
            (s / 1e6).plot.bar(ax=ax, color="#2F5496"); ax.set_title("Doanh thu theo nhóm sản phẩm (triệu)")
        )
    elif p == "pay":
        df["Payment Method"].value_counts().plot.bar(ax=ax, color="#ED7D31"); ax.set_title("Số giao dịch theo phương thức thanh toán")
        ax.tick_params(axis="x", rotation=0)
    elif p == "churn":
        df["Churn"].value_counts(normalize=True).sort_index().mul(100).plot.bar(ax=ax, color=["#70AD47", "#C00000"]);
        ax.set_title("Tỷ lệ Churn (%)"); ax.tick_params(axis="x", rotation=0)
    elif p == "age":
        df[age_col].plot.hist(bins=30, ax=ax, color="#7F7F7F"); ax.set_title(f"Phân phối tuổi ({age_col})")
    elif p == "ret":
        (df.groupby("Product Category")["Returns"].mean() * 100).sort_values().plot.bar(ax=ax, color="#C00000")
        ax.set_title("Tỷ lệ trả hàng theo nhóm (%)")
    elif p == "amount":
        df["Total Purchase Amount"].plot.hist(bins=40, ax=ax, color="#2F5496"); ax.set_title("Phân phối giá trị mỗi giao dịch")
for ax in axes[len(panels):]: ax.axis("off")
plt.tight_layout(); plt.show()
```

Output



Giải thích code

Vẽ lưới 2 hàng $\times$ 3 cột biểu đồ mô tả tổng quan giao dịch.

Code xác định trước danh sách panel tùy theo cột có mặt (doanh thu theo nhóm, phương thức thanh toán, churn, tuổi, tỷ lệ trả hàng, phân phối giá trị đơn). Mỗi panel dùng groupby hoặc value_counts rồi vẽ cột ngang/dọc/histogram; các ô thừa được tắt bằng axis("off").

Giải thích output

Cả sáu biểu đồ đều 'quá đều': doanh thu bốn nhóm Home, Clothing, Electronics, Books gần bằng nhau (khoảng 165 đến 170 triệu mỗi nhóm); số giao dịch của Credit Card, PayPal, Cash gần như bằng nhau (khoảng 83.000); tỷ lệ trả hàng mọi nhóm xấp xỉ 50%; tuổi trải đều từ 18 đến 70; giá trị mỗi giao dịch phân phối phẳng từ 100 đến khoảng 5.350. Riêng Churn lệch: khoảng 80% không rời bỏ và 20% rời bỏ.

Nhận xét: không có nhóm sản phẩm hay phương thức nào nổi trội, đúng với đặc điểm dữ liệu sinh ngẫu nhiên. Các khe hở đều đặn trong histogram tuổi là do cách chia bin, không phải dữ liệu thiếu.

d) Xây dựng chuỗi thời gian theo ngày và kiểm tra cấu trúc

RNN chỉ có ích khi chuỗi có phụ thuộc theo thời gian. Ta kiểm tra xu hướng, mùa vụ theo thứ, tự tương quan (ACF).

Cell 12: Tổng hợp giao dịch thành chuỗi thời gian theo ngày

Code

```
# ---- Tổng hợp giao dịch thành chuỗi thời gian theo NGÀY ----
orders = df.set_index("Purchase Date").sort_index()
daily = orders.resample("D").agg(
    revenue=("Total Purchase Amount", "sum"),
    orders=("Total Purchase Amount", "size"),
    avg_order=("Total Purchase Amount", "mean"),
)
daily["return_rate"] = orders["Returns"].resample("D").mean() if "Returns" in orders.columns else 0.0
daily[["avg_order", "return_rate"]] = daily[["avg_order", "return_rate"]].ffill().bfill() # ngày không có đơn
n_empty = int((daily["orders"] == 0).sum())

def calendar_feats(idx):
    """Đặc trưng lịch (biết trước cả trong tương lai): mã hóa vòng bằng sin/cos."""
    dow = np.asarray(idx.dayofweek); mon = np.asarray(idx.month) - 1
    return pd.DataFrame({
        "dow_sin": np.sin(2 * np.pi * dow / 7), "dow_cos": np.cos(2 * np.pi * dow / 7),
        "month_sin": np.sin(2 * np.pi * mon / 12), "month_cos": np.cos(2 * np.pi * mon / 12),
    }, index=idx)

daily = daily.join(calendar_feats(daily.index))
print(f"Chuỗi theo ngày: {len(daily)} ngày | Ngày không có giao dịch: {n_empty}")
display(daily.head())
display(daily[["revenue", "orders", "avg_order", "return_rate"]].describe().T)
```

Output

Chuỗi theo ngày: 1352 ngày | Ngày không có giao dịch: 0

	revenue	orders	avg_order	return_rate	dow_sin	dow_cos	\
Purchase Date							
2020-01-01	464688	179	2,596.0223	0.5143	0.9749	-0.2225	
2020-01-02	546673	196	2,789.1480	0.5427	0.4339	-0.9010	
2020-01-03	498883	176	2,834.5625	0.5105	-0.4339	-0.9010	
2020-01-04	504343	186	2,711.5215	0.5031	-0.9749	-0.2225	
2020-01-05	500861	190	2,636.1105	0.5064	-0.7818	0.6235	
					month_sin	month_cos	
Purchase Date							
2020-01-01					0.0000	1.0000	
2020-01-02					0.0000	1.0000	
2020-01-03					0.0000	1.0000	
2020-01-04					0.0000	1.0000	
2020-01-05					0.0000	1.0000	
	count	mean	std	min	25%	\	
revenue	1,352.0000	503,954.3632	42,592.3069	357,710.0000	474,821.0000		
orders	1,352.0000	184.9112	14.0778	136.0000	175.0000		
avg_order	1,352.0000	2,725.5420	103.7020	2,307.8065	2,653.5183		
return_rate	1,352.0000	0.5008	0.0401	0.3608	0.4730		
	50%	75%	max				
revenue	504,141.5000	530,477.5000	649,755.0000				
orders	185.0000	194.0000	235.0000				
avg_order	2,729.0090	2,798.9239	3,042.5223				
return_rate	0.5000	0.5295	0.6479				

Giải thích code

Biến bảng giao dịch thành chuỗi thời gian đa biến theo ngày, thêm đặc trưng lịch. `set_index("Purchase Date").sort_index()` đưa cột ngày làm chỉ mục và sắp xếp. `resample("D").agg(...)` gộp theo ngày: `revenue` là tổng tiền, `orders` là số giao dịch (size), `avg_order` là giá trị đơn trung bình. `return_rate` là trung bình cột `Returns`

mỗi ngày. Những ngày không có đơn (nếu có) được điền ffill/bfill; n_empty đếm số ngày trống.

calendar_feats mã hóa thứ trong tuần (0 đến 6) và tháng (0 đến 11) bằng cặp sin/cos, ví dụ $\sin(2\pi \cdot \text{thứ}/7)$ và $\cos(2\pi \cdot \text{thứ}/7)$. Cách mã hóa vòng này giúp mô hình hiểu chủ nhật và thứ hai là 'liền kề', và ngày lịch luôn biết trước nên dùng được cả khi dự báo tương lai. Cuối cell in số ngày, hiển thị 5 dòng đầu và thống kê mô tả của bốn đại lượng chính.

Giải thích output

Chuỗi dài 1.352 ngày liên tục và không có ngày nào thiếu giao dịch (0 ngày trống), nên không phải nội suy.

Bảng đầu: ngày 01/01/2020 doanh thu 464.688 với 179 đơn, giá trị đơn trung bình 2.596, tỷ lệ trả hàng 0,514; các cột dow/month là giá trị sin/cos (tháng 1 nên month_sin = 0 và month_cos = 1).

Thống kê: doanh thu trung bình 503.954/ngày, độ lệch chuẩn 42.592 (chỉ khoảng 8,5% so với trung bình), nhỏ nhất 357.710, lớn nhất 649.755; số đơn trung bình 184,9 (từ 136 đến 235); tỷ lệ trả hàng trung bình 0,5008. Biến động tương đối nhỏ cho thấy chuỗi dao động quanh một mức gần cố định.

Cell 13: Trực quan hóa cấu trúc chuỗi thời gian (3 hình)

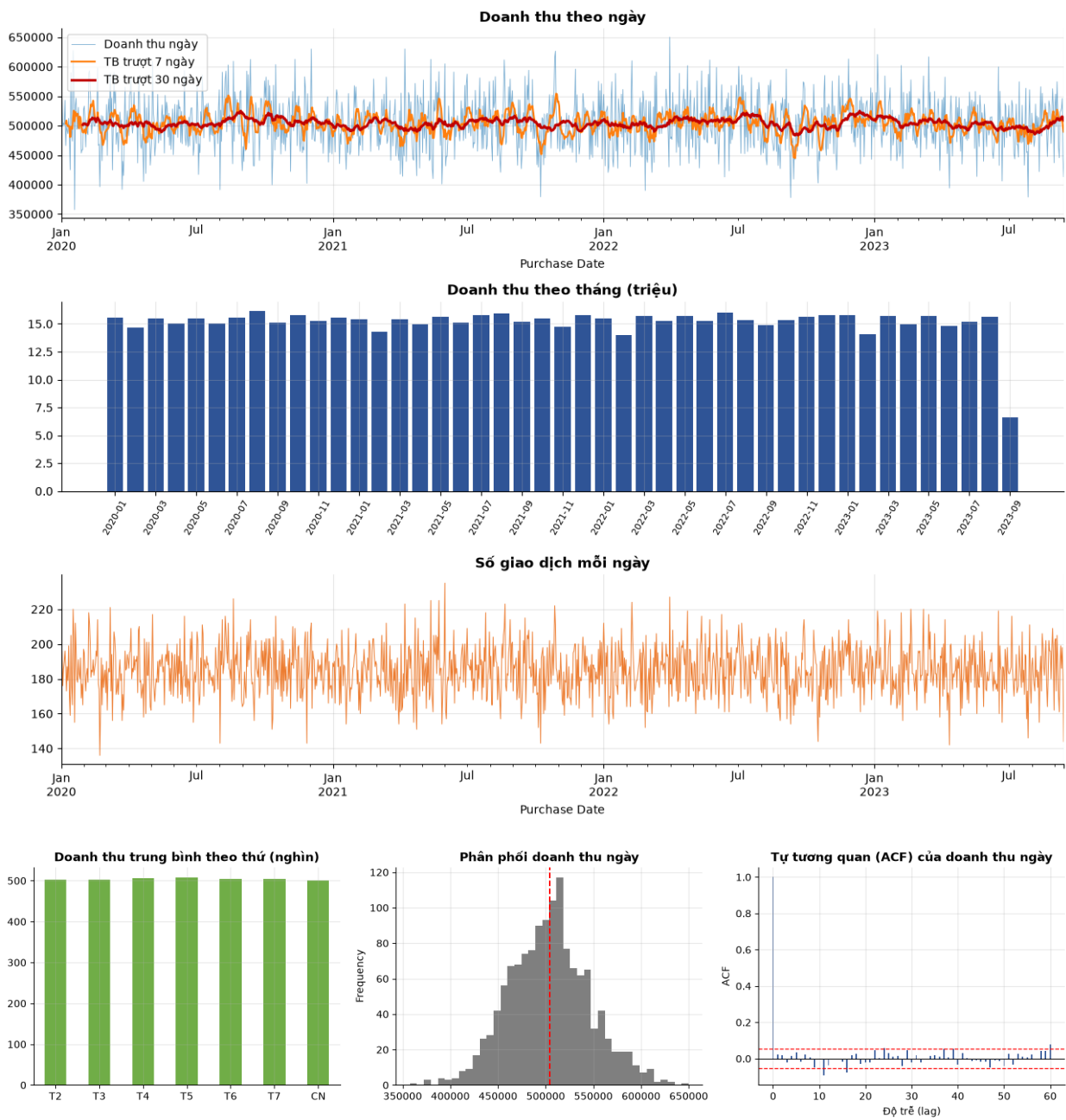
Code

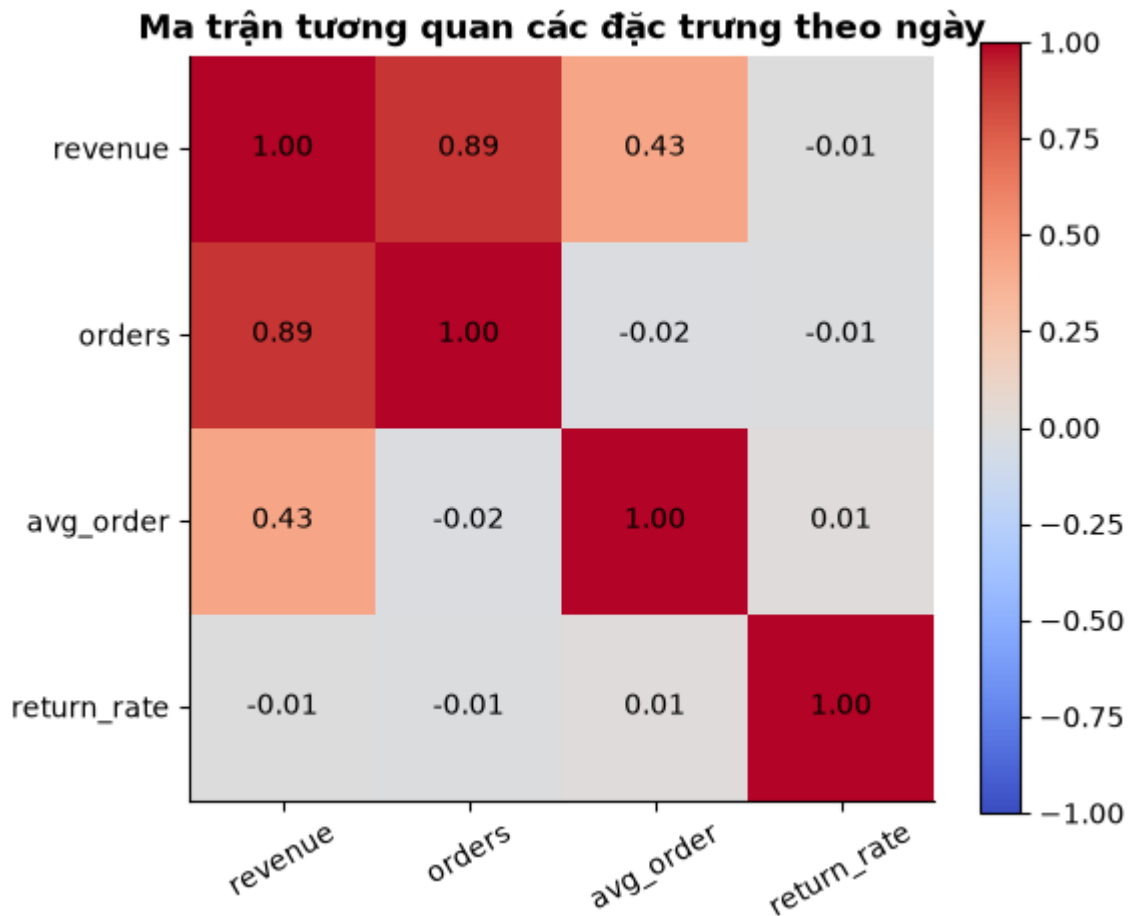
```
fig, axes = plt.subplots(3, 1, figsize=(13, 10), sharex=False)
daily["revenue"].plot(ax=axes[0], lw=0.6, alpha=0.6, label="Doanh thu ngày")
daily["revenue"].rolling(7).mean().plot(ax=axes[0], lw=1.6, label="TB trượt 7 ngày")
daily["revenue"].rolling(30).mean().plot(ax=axes[0], lw=2.2, c="#C00000", label="TB trượt 30 ngày")
axes[0].set_title("Doanh thu theo ngày"); axes[0].legend(loc="upper left")
monthly = daily["revenue"].resample("MS").sum() / 1e6
axes[1].bar(range(len(monthly)), monthly.values, color="#2F5496")
step = max(1, len(monthly) // 20)
axes[1].set_xticks(range(0, len(monthly), step))
axes[1].set_xticklabels([d.strftime("%Y-%m") for d in monthly.index[::step]], rotation=60, fontsize=8)
axes[1].set_title("Doanh thu theo tháng (triệu)")
daily["orders"].plot(ax=axes[2], lw=0.7, c="#ED7D31"); axes[2].set_title("Số giao dịch mỗi ngày")
plt.tight_layout(); plt.show()

fig, axes = plt.subplots(1, 3, figsize=(15, 4))
wd = daily.groupby(daily.index.dayofweek)["revenue"].mean()
wd.index = ["T2", "T3", "T4", "T5", "T6", "T7", "CN"]
(wd / 1e3).plot.bar(ax=axes[0], color="#70AD47"); axes[0].set_title("Doanh thu trung bình theo thứ (nghìn)")
axes[0].tick_params(axis="x", rotation=0)
daily["revenue"].plot.hist(bins=40, ax=axes[1], color="#7F7F7F"); axes[1].set_title("Phân phối doanh thu ngày")
axes[1].axvline(daily["revenue"].mean(), c="r", ls="--")
a, conf = plot_acf(axes[2], daily["revenue"], 60, "Tự tương quan (ACF) của doanh thu ngày")
plt.tight_layout(); plt.show()

fig, ax = plt.subplots(figsize=(6, 5))
cm = daily[["revenue", "orders", "avg_order", "return_rate"]].corr()
im = ax.imshow(cm, cmap="coolwarm", vmin=-1, vmax=1); ax.grid(False)
ax.set_xticks(range(4)); ax.set_xticklabels(cm.columns, rotation=30); ax.set_yticks(range(4));
ax.set_yticklabels(cm.columns)
for i in range(4):
    for j in range(4): ax.text(j, i, f"{cm.iloc[i, j]:.2f}", ha="center", va="center")
ax.set_title("Ma trận tương quan các đặc trưng theo ngày"); plt.colorbar(im); plt.show()
```

Output





Giải thích code

Vẽ ba hình để kiểm tra xu hướng, mùa vụ, phân phối, tự tương quan và tương quan giữa các đặc trưng.

Hình 1 (3 hàng): doanh thu theo ngày cùng trung bình trượt 7 và 30 ngày; doanh thu theo tháng (triệu) dạng cột; số giao dịch mỗi ngày. Hình 2 (1 hàng $\times$ 3): doanh thu trung bình theo thứ trong tuần, histogram doanh thu ngày (kèm đường trung bình đồ), và ACF của doanh thu tới độ trễ 60. Hình 3: ma trận tương quan giữa revenue, orders, avg_order, return_rate vẽ bằng imshow kèm số.

Giải thích output

Hình 1: doanh thu ngày dao động quanh 500 nghìn, đường trung bình trượt 30 ngày gần như nằm ngang, không có xu hướng tăng/giảm hay chu kỳ mùa vụ nào. Doanh thu các tháng đều xấp xỉ 15 triệu; cột cuối (tháng 9/2023) thấp hơn nhiều (khoảng 6,5 triệu) chỉ vì dữ liệu dừng ở ngày 13/09. Số giao dịch ngày dao động 140 đến 235.

Hình 2: doanh thu trung bình theo thứ trong tuần gần như bằng nhau (khoảng 500 nghìn mỗi thứ) nên không có mùa vụ theo tuần. Histogram có dạng chuông quanh

504 nghìn (đúng như định lý giới hạn trung tâm cho tổng của nhiều giao dịch độc lập). ACF gần như toàn bộ nằm trong dải $\pm 0,053$, tức chuỗi không nhớ quá khứ.

Hình 3: revenue tương quan mạnh với orders (0,89) và vừa với avg_order (0,43), vì doanh thu = số đơn $\times$ giá trị đơn; các cặp còn lại xấp xỉ 0, cho thấy các đặc trưng này gần như độc lập.

Cell 14: Nhận định tự động về cấu trúc chuỗi

Code

```
# ---- Nhận định tự động về cấu trúc chuỗi (quyết định RNN có "đắt tiền" hay không) ----
x = daily["revenue"].values
n = len(x)
sig_lags = [k for k in range(1, 31) if abs(a[k]) > conf]
slope = np.polyfit(np.arange(n), x, 1)[0] * 365
wd_cv = wd.std() / wd.mean()
print(f"- Tự tương quan lag-1 = {a[1]:.3f}, lag-7 = {a[7]:.3f} (ngưỡng ý nghĩa  $\pm$ {conf:.3f})")
print(f"- Các lag (1-30) có ý nghĩa thống kê: {sig_lags if sig_lags else 'không có'}")
print(f"- Xu hướng tuyến tính: {slope:.0f} đơn vị doanh thu/năm ({slope / x.mean():+.1%} so với mức trung bình)")
print(f"- Chênh lệch giữa các thứ trong tuần (hệ số biến thiên): {wd_cv:.1%}")
if not sig_lags and wd_cv < 0.05 and abs(slope / x.mean()) < 0.05:
    print("=> Chuỗi gần với nhiễu trắng: RNN khó vượt được mô hình cơ sở. Đây là một phát hiện hợp lệ, "
          "không phải lỗi mô hình (dữ liệu này là dữ liệu tổng hợp).")
else:
    print("=> Chuỗi có cấu trúc thời gian (tự tương quan/xu hướng/mùa vụ tuần): RNN có cơ sở để học.")
```

Output

```
- Tự tương quan lag-1 = 0.024, lag-7 = 0.023 (ngưỡng ý nghĩa  $\pm 0.053$ )
- Các lag (1-30) có ý nghĩa thống kê: [11, 16, 24]
- Xu hướng tuyến tính: -513 đơn vị doanh thu/năm (-0.1% so với mức trung bình)
- Chênh lệch giữa các thứ trong tuần (hệ số biến thiên): 0.5%
=> Chuỗi có cấu trúc thời gian (tự tương quan/xu hướng/mùa vụ tuần): RNN có cơ sở để học.
```

Giải thích code

Tính vài chỉ số định lượng rồi in kết luận: chuỗi có đáng để dùng RNN hay không.

Lấy các độ trễ 1 đến 30 mà $|ACF|$ vượt ngưỡng (sig_lags); tính độ dốc xu hướng tuyến tính bằng np.polyfit nhân 365 để quy về mỗi năm; tính hệ số biến thiên giữa các thứ trong tuần (wd_cv = độ lệch chuẩn / trung bình). Nếu không có lag có ý nghĩa và chênh lệch theo thứ dưới 5% và xu hướng dưới 5% thì in cảnh báo 'gần nhiễu trắng', ngược lại in 'có cấu trúc thời gian'.

Giải thích output

Tự tương quan lag-1 là 0,024 và lag-7 là 0,023, đều rất nhỏ so với ngưỡng 0,053. Ba lag được coi là có ý nghĩa là 11, 16, 24. Xu hướng -513 đơn vị/năm (-0,1% so với trung bình), gần bằng 0. Chênh lệch giữa các thứ chỉ 0,5%.

Kết luận in ra là 'chuỗi có cấu trúc thời gian'. Cần đọc thật trọng: điều kiện trong code là 'có ít nhất một lag có ý nghĩa', nhưng với 30 lag kiểm định ở mức 5% thì trung bình có khoảng 1,5 lag vượt ngưỡng chỉ do ngẫu nhiên; các lag 11, 16, 24 lại rời rạc và không có ý nghĩa kinh tế. Xét đồng thời lag-1, lag-7, xu hướng và

mùa vụ tuần đều bằng 0, chuỗi thực chất gần với nhiễu trắng. Điều này giải thích các kết quả ở phần sau ($R^2 \approx 0$, dự báo phẳng).

e) Chia dữ liệu theo thời gian, chuẩn hóa và tạo cửa sổ trượt

- Chia **theo trình tự thời gian** (70% / 15% / 15%), không xáo trộn để tránh rò rỉ thông tin tương lai.
- StandardScaler chỉ được **fit trên tập huấn luyện**.
- Mỗi mẫu là tensor (WINDOW, 8); nhãn là 4 đại lượng của ngày kế tiếp (doanh thu, số đơn, giá trị đơn TB, tỷ lệ trả hàng). Dự đoán đa đầu ra cho phép dự báo đệ quy nhiều ngày ở phần 9. Chỉ số đánh giá tập trung vào **doanh thu**.

Cell 16: Chia dữ liệu theo thời gian, chuẩn hóa, tạo cửa sổ trượt

Code

```
FEATURES = ["revenue", "orders", "avg_order", "return_rate", "dow_sin", "dow_cos", "month_sin", "month_cos"]
OUT_DIM = 4 # mô hình dự đoán 4 đại lượng đầu tiên của ngày kế tiếp
N_FEATURES = len(FEATURES)

arr = daily[FEATURES].values.astype("float32")
n = len(arr)
i1, i2 = int(n * TRAIN_RATIO), int(n * (TRAIN_RATIO + VAL_RATIO)) # mốc chia THEO THỜI GIAN

scaler = StandardScaler().fit(arr[:i1]) # chỉ ước lượng trên tập huấn luyện -> không rò rỉ
arr_s = scaler.transform(arr).astype("float32")
inv_rev = lambda z: z * scaler.scale_[0] + scaler.mean_[0] # đưa doanh thu về đơn vị gốc

X_all, y_all, pos = [], [], []
for i in range(WINDOW, n):
    X_all.append(arr_s[i - WINDOW:i]) # WINDOW ngày trước đó
    y_all.append(arr_s[i, :OUT_DIM]) # đặc trưng của ngày i
    pos.append(i)
X_all, y_all, pos = np.array(X_all, "float32"), np.array(y_all, "float32"), np.array(pos)

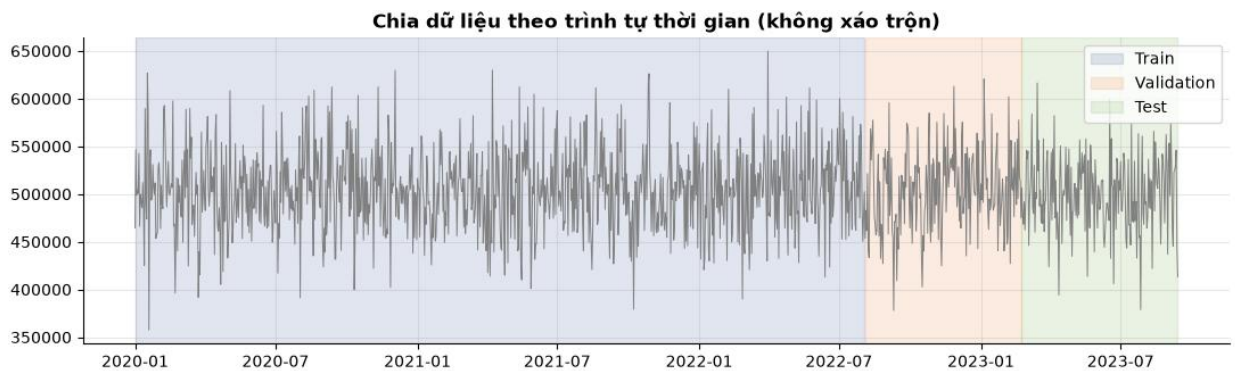
m_tr, m_va, m_te = pos < i1, (pos >= i1) & (pos < i2), pos >= i2
X_train, y_train = X_all[m_tr], y_all[m_tr]
X_val, y_val = X_all[m_va], y_all[m_va]
X_test, y_test = X_all[m_te], y_all[m_te]
raw_rev = daily["revenue"].values
dates = daily.index

print(f"Train: {X_train.shape} | Val: {X_val.shape} | Test: {X_test.shape}")
print(f"Train: {dates[WINDOW].date()} -> {dates[i1 - 1].date()}")
print(f"Val : {dates[i1].date()} -> {dates[i2 - 1].date()}")
print(f"Test : {dates[i2].date()} -> {dates[-1].date()}")

fig, ax = plt.subplots(figsize=(13, 3.5))
ax.plot(dates, raw_rev, lw=0.7, c="gray")
for lo, hi, c, lb in [(0, i1, "#2F5496", "Train"), (i1, i2, "#ED7D31", "Validation"), (i2, n, "#70AD47", "Test")]:
    ax.axvspan(dates[lo], dates[hi - 1], color=c, alpha=0.15, label=lb)
ax.set_title("Chia dữ liệu theo trình tự thời gian (không xáo trộn)"); ax.legend(); plt.show()
```

Output

```
Train: (916, 30, 8) | Val: (203, 30, 8) | Test: (203, 30, 8)
Train: 2020-01-31 -> 2022-08-03
Val : 2022-08-04 -> 2023-02-22
Test : 2023-02-23 -> 2023-09-13
```

Giải thích code

Tạo tập huấn luyện/kiểm định/kiểm thử dạng tensor (mẫu, 30, 8) mà không rò rỉ dữ liệu tương lai.

FEATURES gồm 8 đặc trưng đầu vào; OUT_DIM = 4 nghĩa là mô hình dự đoán bốn đại lượng đầu (revenue, orders, avg_order, return_rate) của ngày kế tiếp để có thể dự báo đệ quy ở phần 9. Mốc chia $i1 = 70\%$ và $i2 = 85\%$ tổng số ngày. StandardScaler chỉ fit trên `arr[:i1]` (phần huấn luyện) rồi biến đổi toàn bộ mảng, tránh rò rỉ thông tin. `inv_rev` đưa doanh thu chuẩn hóa về đơn vị gốc.

Vòng `for i in range(WINDOW, n)` tạo từng mẫu: X là 30 dòng liền trước ngày i , y là 4 đại lượng của ngày i , `pos` lưu vị trí ngày. Ba mặt nạ `m_tr`, `m_va`, `m_te` tách mẫu theo vị trí ngày. Cuối cell in kích thước, khoảng ngày của từng tập và vẽ đồ thị chia vùng bằng `axvspan`.

Giải thích output

Train (916, 30, 8), Val (203, 30, 8), Test (203, 30, 8): tức 916, 203 và 203 mẫu, mỗi mẫu 30 ngày $\times$ 8 đặc trưng. Train từ 31/01/2020 đến 03/08/2022, Val từ 04/08/2022 đến 22/02/2023, Test từ 23/02/2023 đến 13/09/2023 (30 ngày đầu chỉ dùng làm lịch sử của mẫu đầu tiên).

Đồ thị tô ba vùng màu xanh (train), cam (validation), xanh lá (test) chồng lên doanh thu xám, minh họa việc chia theo trình tự thời gian và không xáo trộn.

f) Mô hình cơ sở (baseline)

Mọi mô hình RNN phải được so với hai baseline đơn giản: **Naive** (doanh thu hôm qua) và **trung bình 7 ngày**.

Cell 18: Mô hình cơ sở (baseline) và các hàm đo lỗi

Code

```
def rmse(y, p): return float(np.sqrt(np.mean((np.asarray(y) - np.asarray(p)) ** 2)))
def mae(y, p): return float(np.mean(np.abs(np.asarray(y) - np.asarray(p))))
def mape(y, p):
    y, p = np.asarray(y), np.asarray(p); m = y > 0
    return float(np.mean(np.abs((y[m] - p[m]) / y[m])) * 100)
def r2(y, p):
    y, p = np.asarray(y), np.asarray(p)
    return float(1 - np.sum((y - p) ** 2) / np.sum((y - y.mean()) ** 2))

split_pos = {"val": pos[m_va], "test": pos[m_te]}
rev_true = {k: raw_rev[v] for k, v in split_pos.items()}
preds = {"val": {}, "test": {}}
for k, v in split_pos.items():
    preds[k]["Naive (hôm qua)"] = raw_rev[v - 1]
    preds[k]["TB 7 ngày"] = np.array([raw_rev[p - 7:p].mean() for p in v])
print("Baseline tập test -> ", {k: round(rmse(rev_true['test'], v), 1) for k, v in preds['test'].items()})
```

Output

```
Baseline tập test -> {'Naive (hôm qua)': 55658.1, 'TB 7 ngày': 41844.4}
```

Giải thích code

Định nghĩa bốn thước đo và dựng hai mô hình cơ sở để làm mốc so sánh.

rmse (căn bình phương sai số trung bình), mae (sai số tuyệt đối trung bình), mape (% sai số tương đối, bỏ qua giá trị ≤ 0) và r2 (hệ số xác định so với việc luôn đoán trung bình của tập đang xét). split_pos lấy vị trí ngày của tập val/test; rev_true là doanh thu thật. Baseline Naive dự báo doanh thu ngày t bằng ngày t-1; baseline TB 7 ngày dự báo bằng trung bình 7 ngày trước đó. Dòng in cuối tính RMSE của hai baseline trên tập test.

Giải thích output

RMSE test: Naive 55.658 và trung bình 7 ngày 41.844. Naive tệ hơn nhiều vì chuỗi gần như nhiễu trắng: sai số của việc lấy giá trị hôm qua là hiệu của hai nhiễu độc lập nên phương sai gấp đôi (độ lệch chuẩn khoảng $42,6 \text{ nghìn} \times \sqrt{2} \approx 60 \text{ nghìn}$, gần với 55,7 nghìn quan sát được). Trung bình 7 ngày khá hơn vì trung bình hóa bớt nhiễu. Đây là mốc mà các mạng RNN phải vượt.

g) Xây dựng và huấn luyện RNN / LSTM / GRU (Keras (TensorFlow))

Ba kiến trúc dùng cùng siêu tham số (2 lớp $\times$ UNITS đơn vị, dropout, Adam, MSE, dùng sớm, giảm learning rate khi chững) để so sánh công bằng.

Cell 20: Xây dựng mô hình Keras (RNN/LSTM/GRU) và xem cấu trúc

Code

```
def build_model(kind, input_shape, out_dim):
    """2 lớp RNN/LSTM/GRU xếp chồng + Dropout + đầu ra Dense."""
    Cell = {"RNN": layers.SimpleRNN, "LSTM": layers.LSTM, "GRU": layers.GRU}[kind]
    model = keras.Sequential([
        layers.Input(shape=input_shape),
        Cell(UNITS, return_sequences=True),
        layers.Dropout(DROPOUT),
        Cell(UNITS),
        layers.Dropout(DROPOUT),
        layers.Dense(32, activation="relu"),
        layers.Dense(out_dim),
    ], name=kind)
    model.compile(optimizer=keras.optimizers.Adam(LR), loss="mse")
    return model

def train_model(kind, X_tr, y_tr, X_va, y_va):
    keras.utils.set_random_seed(SEED)
    model = build_model(kind, X_tr.shape[1:], y_tr.shape[1])
    callbacks = [
        keras.callbacks.EarlyStopping(monitor="val_loss", patience=PATIENCE, restore_best_weights=True),
        keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=max(3, PATIENCE // 3),
                                           min_lr=1e-5),
    ]
    h = model.fit(X_tr, y_tr, validation_data=(X_va, y_va), epochs=EPOCHS,
                  batch_size=BATCH_SIZE, callbacks=callbacks, verbose=1)
    n_ep = len(h.history["loss"])
    print(f"{kind:5s} | dừng sau {n_ep:3d} epoch | val_loss tốt nhất = {min(h.history['val_loss']):.4f} "
          f"| số tham số = {model.count_params():,}")
    return model, {"loss": h.history["loss"], "val_loss": h.history["val_loss"]}

def predict_scaled(model, X):
    """Dự đoán trên không gian đã chuẩn hóa, trả về mảng (n_mẫu, out_dim)."""
    return model.predict(X, batch_size=256, verbose=1).reshape(len(X), -1)

# Xem cấu trúc một mô hình mẫu
build_model("LSTM", (WINDOW, N_FEATURES), OUT_DIM).summary()
```

Output

Model: "LSTM"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 30, 64)	18,688
dropout (Dropout)	(None, 30, 64)	0
lstm_1 (LSTM)	(None, 64)	33,024
dropout_1 (Dropout)	(None, 64)	0
dense (Dense)	(None, 32)	2,080
dense_1 (Dense)	(None, 4)	132

Total params: 53,924 (210.64 KB)

Trainable params: 53,924 (210.64 KB)

Non-trainable params: 0 (0.00 B)

Giải thích code

Định nghĩa hàm dựng mô hình, hàm huấn luyện, hàm dự đoán, rồi in summary của một mô hình LSTM mẫu.

`build_model(kind, input_shape, out_dim)`: chọn lớp SimpleRNN, LSTM hoặc GRU theo tên; mạng Sequential gồm Input, lớp RNN thứ nhất (`return_sequences=True` để trả về chuỗi trạng thái), Dropout, lớp RNN thứ hai

(chỉ trả về bước cuối), Dropout, Dense(32, relu) và Dense(out_dim) là đầu ra tuyến tính. Biên dịch với Adam(1e-3) và hàm mất mát MSE.

train_model: cố định seed, dựng mô hình, dùng hai callback: EarlyStopping (theo val_loss, kiên nhẫn 12 epoch, khôi phục trọng số tốt nhất) và ReduceLROnPlateau (giảm một nửa learning rate khi val_loss chững 4 epoch). Gọi model.fit rồi in số epoch đã chạy, val_loss tốt nhất và số tham số. predict_scaled gọi model.predict và làm phẳng thành (n_mẫu, out_dim).

Giải thích output

Summary của LSTM: lớp LSTM 1 có 18.688 tham số, LSTM 2 có 33.024, Dense(32) có 2.080, Dense(4) có 132, tổng 53.924 tham số (210,64 KB), tất cả đều huấn luyện được. Có thể kiểm tra: lớp LSTM có 4 cổng, mỗi cổng $(64 \times (8 + 64) + 64) = 4.672$ nên tổng $4 \times 4.672 = 18.688$. Dropout không có tham số. Đầu ra hình (None, 4) khớp OUT_DIM.

Cell 21: Huấn luyện ba mô hình RNN, LSTM, GRU

Code

```
KINDS = ["RNN", "LSTM", "GRU"]
models, histories = {}, {}
for kind in KINDS:
    models[kind], histories[kind] = train_model(kind, X_train, y_train, X_val, y_val)
```

Output

```
Epoch 1/100
29/29 _____ 4s 26ms/step - loss: 1.2295 - val_loss: 1.1163 - learning_rate: 0.0010
Epoch 2/100
29/29 _____ 0s 12ms/step - loss: 1.0759 - val_loss: 1.1091 - learning_rate: 0.0010
Epoch 3/100
29/29 _____ 1s 11ms/step - loss: 1.0294 - val_loss: 1.1026 - learning_rate: 0.0010
Epoch 4/100
29/29 _____ 0s 13ms/step - loss: 1.0049 - val_loss: 1.0971 - learning_rate: 0.0010
Epoch 5/100
29/29 _____ 1s 16ms/step - loss: 1.0061 - val_loss: 1.0966 - learning_rate: 0.0010
Epoch 6/100
29/29 _____ 0s 16ms/step - loss: 0.9700 - val_loss: 1.1019 - learning_rate: 0.0010
Epoch 7/100
29/29 _____ 0s 16ms/step - loss: 0.9769 - val_loss: 1.0976 - learning_rate: 0.0010
Epoch 8/100
29/29 _____ 0s 12ms/step - loss: 0.9540 - val_loss: 1.0867 - learning_rate: 0.0010
Epoch 9/100
29/29 _____ 1s 13ms/step - loss: 0.9357 - val_loss: 1.0951 - learning_rate: 0.0010
Epoch 10/100
29/29 _____ 0s 15ms/step - loss: 0.9253 - val_loss: 1.0961 - learning_rate: 0.0010
Epoch 11/100
29/29 _____ 0s 12ms/step - loss: 0.9096 - val_loss: 1.1011 - learning_rate: 0.0010
... (đã rút gọn 59 dòng ở giữa) ...
Epoch 8/100
29/29 _____ 1s 35ms/step - loss: 0.9680 - val_loss: 1.0293 - learning_rate: 5.0000e-04
Epoch 9/100
29/29 _____ 1s 29ms/step - loss: 0.9610 - val_loss: 1.0282 - learning_rate: 5.0000e-04
Epoch 10/100
29/29 _____ 1s 30ms/step - loss: 0.9632 - val_loss: 1.0303 - learning_rate: 5.0000e-04
Epoch 11/100
29/29 _____ 1s 25ms/step - loss: 0.9529 - val_loss: 1.0337 - learning_rate: 5.0000e-04
Epoch 12/100
29/29 _____ 1s 28ms/step - loss: 0.9563 - val_loss: 1.0328 - learning_rate: 2.5000e-04
Epoch 13/100
29/29 _____ 1s 26ms/step - loss: 0.9560 - val_loss: 1.0341 - learning_rate: 2.5000e-04
Epoch 14/100
29/29 _____ 1s 27ms/step - loss: 0.9447 - val_loss: 1.0348 - learning_rate: 2.5000e-04
GRU | dừng sau 14 epoch | val_loss tốt nhất = 1.0155 | số tham số = 41,380
```

Giải thích code

Lần lượt huấn luyện ba kiến trúc với cùng dữ liệu và siêu tham số.

Vòng lặp trên KINDS gọi `train_model` cho từng loại và lưu mô hình vào từ điển `models`, lịch sử loss vào `histories`.

Giải thích output

Log Keras in từng epoch dạng 'loss – val_loss – learning_rate'. Ví dụ RNN epoch 1: loss 1,2295, val_loss 1,1163; các epoch sau train loss giảm dần còn val_loss gần như không giảm.

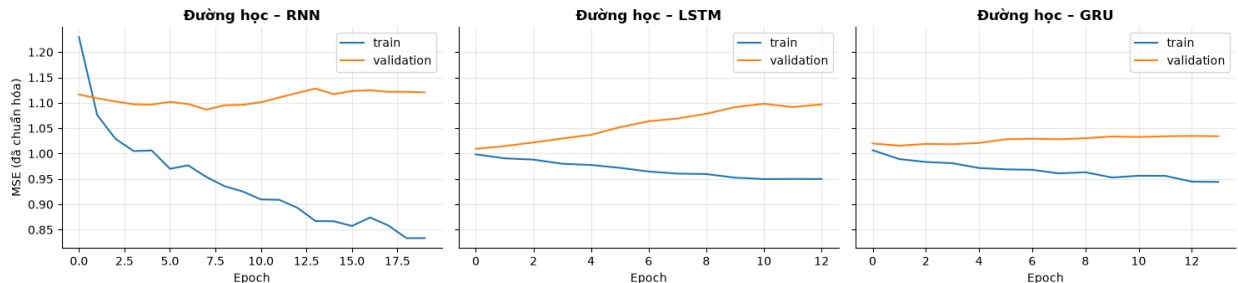
Dòng tổng kết: RNN dừng sau 20 epoch, val_loss tốt nhất 1,0867, 15.140 tham số; LSTM dừng sau 13 epoch, val_loss tốt nhất 1,0093, 53.924 tham số; GRU dừng sau 14 epoch, val_loss tốt nhất 1,0155, 41.380 tham số. Vì dữ liệu đã chuẩn hóa (phương sai = 1), val_loss ≈ 1 nghĩa là mô hình chỉ ngang việc đoán giá trị trung bình; các giá trị này cho thấy mạng chưa học được thêm gì. Nhặt ký dài nên ảnh output đã được rút gọn ở giữa.

Cell 22: Đường học (learning curves)

Code

```
fig, axes = plt.subplots(1, 3, figsize=(15, 3.6), sharey=True)
for ax, kind in zip(axes, KINDS):
    ax.plot(histories[kind]["loss"], label="train"); ax.plot(histories[kind]["val_loss"], label="validation")
    ax.set_title(f"Đường học - {kind}"); ax.set_xlabel("Epoch"); ax.legend()
axes[0].set_ylabel("MSE (đã chuẩn hóa)")
plt.tight_layout(); plt.show()
```

Output



Giải thích code

Vẽ loss huấn luyện và loss kiểm định theo epoch cho từng mô hình, dùng chung trục y.

Mỗi panel vẽ `histories[kind]["loss"]` và `["val_loss"]`; `sharey=True` để so sánh trực tiếp giữa ba mô hình.

Giải thích output

Cả ba mô hình đều có train loss giảm đều nhưng validation loss đi ngang hoặc tăng nhẹ ngay từ những epoch đầu; RNN là rõ nhất (train giảm từ khoảng 1,2 xuống dưới 0,85 trong khi val tăng từ 1,09 lên khoảng 1,12). Khoảng cách ngày càng mở rộng là dấu hiệu quá khớp: mạng ghi nhớ nhiều của tập huấn luyện thay vì học quy luật, khớp với việc dữ liệu không có cấu trúc để học. Early stopping đã chặn ở epoch tốt nhất và khôi phục trọng số ở đó.

h) Đánh giá và trực quan hóa dự đoán

*Bảng chỉ số, biểu đồ dự đoán so với thực tế và phân tích phần dư. Mô hình "tốt nhất" được chọn theo **tập kiểm định** để tập kiểm thử vẫn là phép đo độc lập.*

Cell 24: Đánh giá trên val/test, bảng chỉ số, chọn mô hình tốt nhất

Code

```
for kind in KINDS:
    for k, X in [("val", X_val), ("test", X_test)]:
        preds[k][kind] = inv_rev(predict_scaled(models[kind], X)[: , 0])

rows = []
for name in preds["test"]:
    rows.append({
        "Mô hình": name,
        "RMSE val": rmse(rev_true["val"], preds["val"][name]),
        "RMSE test": rmse(rev_true["test"], preds["test"][name]),
        "MAE test": mae(rev_true["test"], preds["test"][name]),
        "MAPE test (%)": mape(rev_true["test"], preds["test"][name]),
        "R2 test": r2(rev_true["test"], preds["test"][name]),
    })
results = pd.DataFrame(rows).set_index("Mô hình")
display(results.sort_values("RMSE test"))

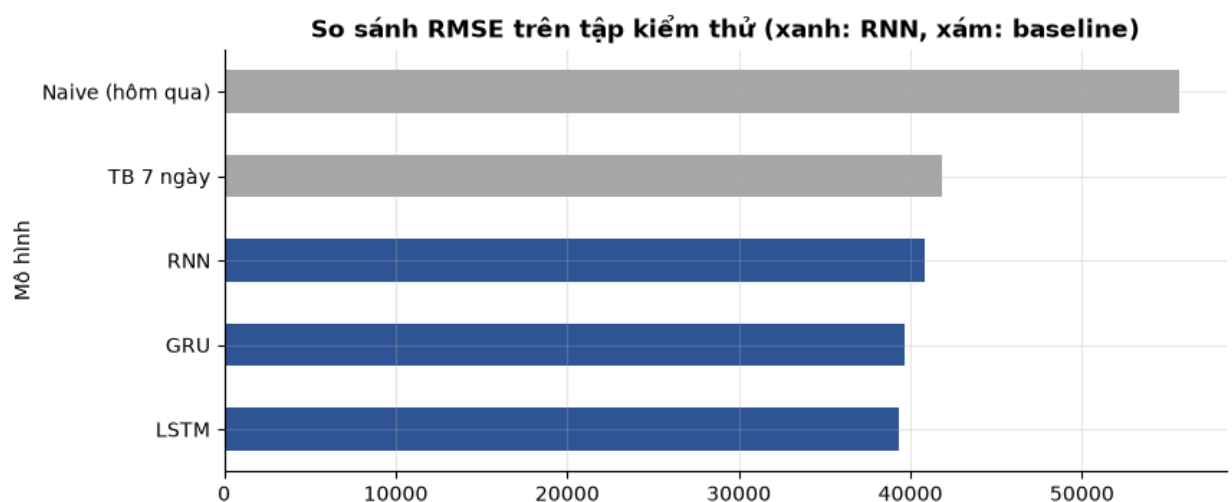
best_kind = min(KINDS, key=lambda k: results.loc[k, "RMSE val"]) # chọn mô hình theo tập KIỂM ĐỊNH
print(f"Mô hình tốt nhất trên tập kiểm định: {best_kind}")

fig, ax = plt.subplots(figsize=(9, 3.8))
results["RMSE test"].sort_values().plot.barh(ax=ax, color=["#2F5496" if i in KINDS else "#A6A6A6" for i in
results["RMSE test"].sort_values().index])
ax.set_title("So sánh RMSE trên tập kiểm thử (xanh: RNN, xám: baseline)"); plt.show()
```

Output

```
1/1 _____ 0s 320ms/step
1/1 _____ 0s 69ms/step
1/1 _____ 0s 435ms/step
1/1 _____ 0s 85ms/step
1/1 _____ 0s 446ms/step
1/1 _____ 0s 65ms/step
```

	RMSE val	RMSE test	MAE test	MAPE test (%)	R ² test
Mô hình					
LSTM	42,721.3917	39,317.8920	31,227.2235	6.3999	-0.0209
GRU	43,104.1923	39,683.9119	31,326.1298	6.4383	-0.0400
RNN	45,442.9649	40,874.6293	32,568.8247	6.6950	-0.1034
TB 7 ngày	45,189.3939	41,844.4026	32,771.7424	6.6565	-0.1563
Naive (hôm qua)	55,250.8168	55,658.1393	44,753.7635	9.0211	-1.0458
Mô hình tốt nhất trên tập kiểm định: LSTM					



Giải thích code

Dự đoán trên val và test, đưa doanh thu về đơn vị gốc, tổng hợp bảng chỉ số và chọn mô hình theo tập kiểm định.

`inv_rev(predict_scaled(...)[:, 0])` lấy cột đầu (doanh thu) của dự đoán và đảo chuẩn hóa. Bảng results gồm RMSE val, RMSE test, MAE, MAPE và R^2 trên test cho cả 3 mô hình lẫn 2 baseline. `best_kind` được chọn theo RMSE của tập kiểm định (không nhìn tập test) để tập test vẫn là phép đo khách quan. Biểu đồ cột ngang so sánh RMSE test, xanh là RNN và xám là baseline.

Giải thích output

Các dòng '1/1 ... step' là tiến trình `model.predict` của Keras (mỗi lần gọi một batch).

Bảng kết quả (sắp theo RMSE test): LSTM 39.318 (MAE 31.227, MAPE 6,40%, $R^2 -0,0209$), GRU 39.684 ($R^2 -0,0400$), RNN 40.875 ($R^2 -0,1034$), TB 7 ngày 41.844 ($R^2 -0,1563$), Naive 55.658 ($R^2 -1,0458$). Mô hình tốt nhất trên tập kiểm định là LSTM (RMSE val 42.721).

Cả ba RNN thấp hơn baseline khoảng 2 đến 6% về RMSE, nhưng R^2 đều âm hoặc xấp xỉ 0: chúng không giải thích được biến thiên nào ngoài mức trung bình. Chênh lệch giữa ba mô hình (39.318 so với 40.875) cũng nhỏ so với RMSE, nên không thể khẳng định LSTM thực sự vượt RNN/GRU.

Cell 25: Đồ thị dự đoán và phân tích sai số

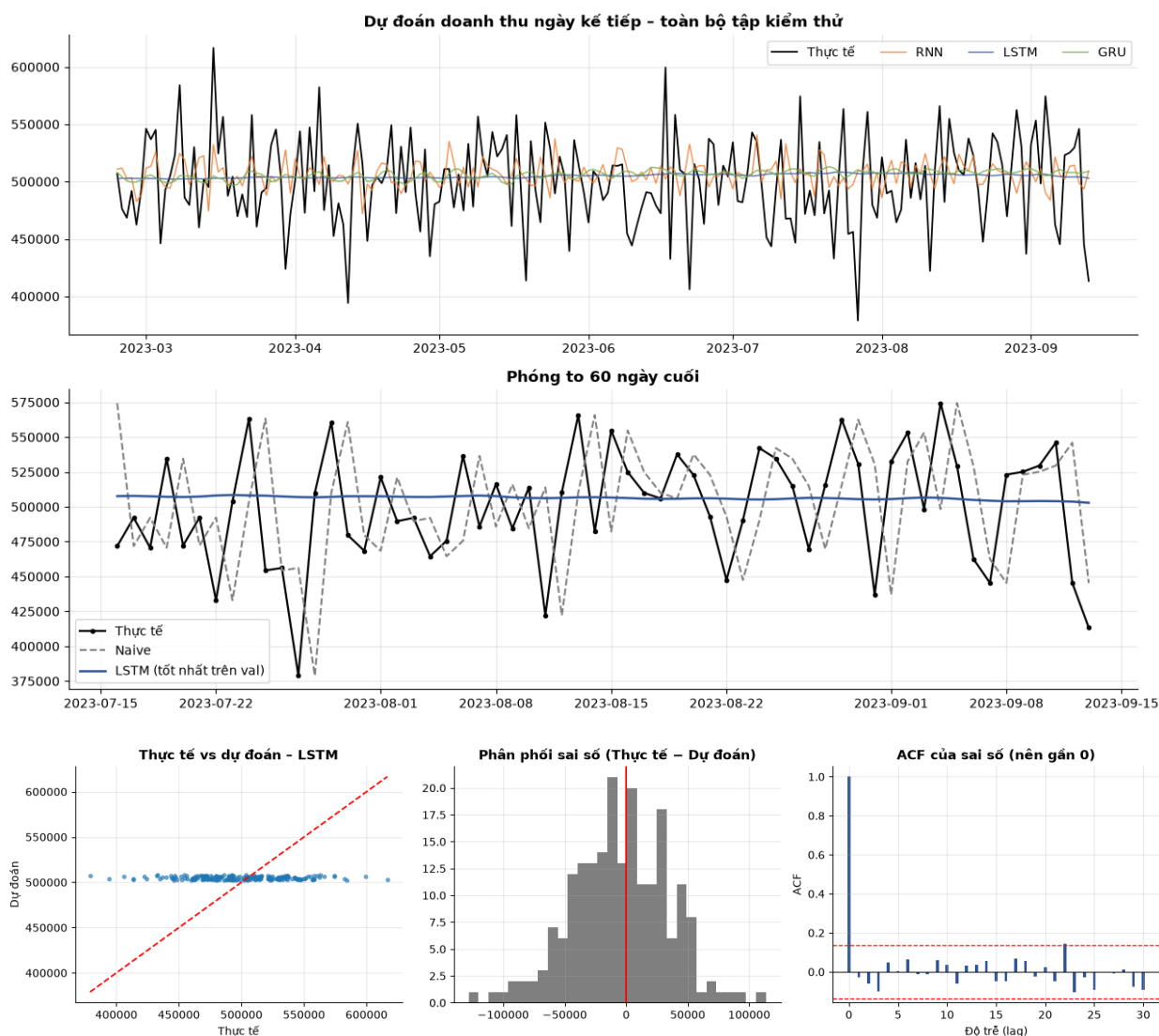
Code

```
d_test = dates[split_pos["test"]]
colors = {"RNN": "#ED7D31", "LSTM": "#2F5496", "GRU": "#70AD47"}
fig, axes = plt.subplots(2, 1, figsize=(13, 8))
axes[0].plot(d_test, rev_true["test"], c="k", lw=1.3, label="Thực tế")
for kind in KINDS: axes[0].plot(d_test, preds["test"][kind], c=colors[kind], lw=1, alpha=0.85, label=kind)
axes[0].set_title("Dự đoán doanh thu ngày kế tiếp - toàn bộ tập kiểm thử"); axes[0].legend(ncol=4)

zoom = slice(-60, None)
axes[1].plot(d_test[zoom], rev_true["test"][zoom], c="k", lw=1.6, marker="o", ms=3, label="Thực tế")
axes[1].plot(d_test[zoom], preds["test"]["Naive (hôm qua)"][zoom], c="gray", ls="--", label="Naive")
axes[1].plot(d_test[zoom], preds["test"][best_kind][zoom], c=colors[best_kind], lw=1.8, label=f"{best_kind} (tốt nhất trên val)")
axes[1].set_title("Phóng to 60 ngày cuối"); axes[1].legend(); plt.tight_layout(); plt.show()

# --- Phân tích sai số của mô hình tốt nhất ---
res = rev_true["test"] - preds["test"][best_kind]
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
axes[0].scatter(rev_true["test"], preds["test"][best_kind], s=10, alpha=0.6)
lím = [min(rev_true["test"].min(), preds["test"][best_kind].min()), max(rev_true["test"].max(), preds["test"][best_kind].max())]
axes[0].plot(lím, lím, "r--"); axes[0].set_xlabel("Thực tế"); axes[0].set_ylabel("Dự đoán");
axes[0].set_title(f"Thực tế vs dự đoán - {best_kind}")
axes[1].hist(res, bins=30, color="#7F7F7F"); axes[1].axvline(0, c="r"); axes[1].set_title("Phân phối sai số (Thực tế - Dự đoán)")
plot_acf(axes[2], res, 30, "ACF của sai số (nên gần 0)")
plt.tight_layout(); plt.show()
print(f"Sai số trung bình (bias) = {res.mean():.1f} | độ lệch chuẩn sai số = {res.std():.1f}")
```


Output



Sai số trung bình (bias) = -4,839.8 | độ lệch chuẩn sai số = 39,018.9

Giải thích code

Vẽ dự đoán so với thực tế và phân tích phần dư của mô hình tốt nhất.

Hình 1: hai panel, trên là toàn bộ tập test với đường thực tế (đen) và dự đoán của 3 mô hình; dưới là 60 ngày cuối, so sánh thực tế, Naive và mô hình tốt nhất. Hình 2: (a) phân tán thực tế – dự đoán kèm đường $y = x$, (b) histogram sai số (thực tế – dự đoán), (c) ACF của sai số. Dòng in cuối là sai số trung bình (bias) và độ lệch chuẩn sai số.

Giải thích output

Trong hình 1, đường thực tế (đen) dao động mạnh trong khoảng 380 nghìn đến 620 nghìn, còn dự đoán của LSTM và GRU gần như là đường ngang quanh 505 nghìn; RNN dao động nhẹ hơn một chút. Ở hình phóng to, đường Naive (nét đứt)

chỉ là bản sao của đường thực tế bị trễ một ngày nên luôn 'đuối theo' sau, trong khi đường LSTM phẳng.

Hình 2: đám điểm phân tán nằm thành dải ngang quanh 505 nghìn chứ không bám đường $y = x$, nghĩa là dự đoán gần như không phụ thuộc vào giá trị thực. Histogram sai số gần đối xứng quanh 0. ACF của sai số hầu như nằm trong dải tin cậy (lag 0 luôn bằng 1, một điểm nhỏ quanh lag 22 chạm ngưỡng), tức sai số không còn cấu trúc tuyến tính.

Bias = $-4.839,8$ (thực tế trừ dự đoán): trung bình dự đoán cao hơn thực tế khoảng 4,8 nghìn. Độ lệch chuẩn sai số 39.019 gần bằng độ lệch chuẩn của chính doanh thu, cho thấy mô hình hầu như không giảm được sai số so với dự đoán hằng số.

i) Phân tích đặc trưng nào quan trọng

Xáo trộn từng đặc trưng trong tập kiểm thử; đặc trưng nào làm RMSE tăng nhiều nhất là đặc trưng mô hình dựa vào nhiều nhất.

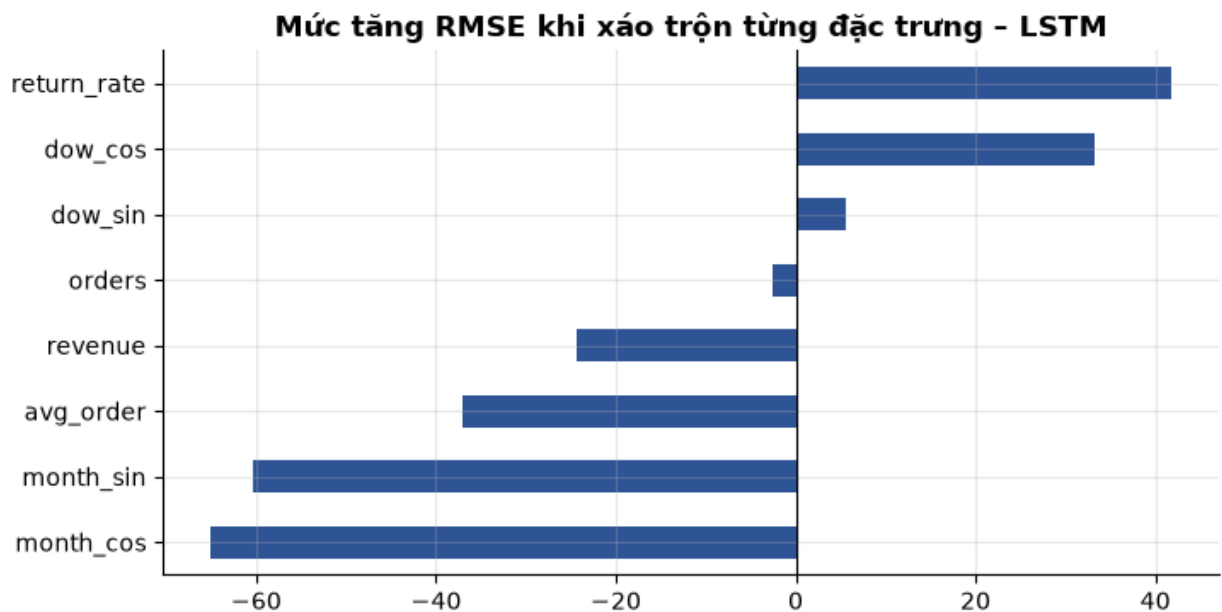
Cell 27: Độ quan trọng đặc trưng bằng hoán vị

Code

```
# ---- Độ quan trọng đặc trưng bằng hoán vị (permutation importance) trên tập kiểm thử ----
base_rmse = rmse(rev_true["test"], preds["test"][best_kind])
rng = np.random.default_rng(SEED)
imp = {}
for j, name in enumerate(FEATURES):
    deltas = []
    for _ in range(5):
        Xp = X_test.copy()
        Xp[:, :, j] = Xp[rng.permutation(len(Xp))][:, :, j] # xáo trộn đặc trưng j giữa các mẫu
        deltas.append(rmse(rev_true["test"], inv_rev(predict_scaled(models[best_kind], Xp[:, 0]))) - base_rmse)
    imp[name] = np.mean(deltas)
imp = pd.Series(imp).sort_values()
fig, ax = plt.subplots(figsize=(8, 4))
imp.plot.barh(ax=ax, color="#2F5496"); ax.axvline(0, c="k", lw=0.8)
ax.set_title(f"Mức tăng RMSE khi xáo trộn từng đặc trưng - {best_kind}"); plt.show()
print("Đặc trưng ảnh hưởng nhất:", imp.idxmax(), "| mức tăng RMSE ≈ %.1f" % imp.max())
```

Output

1/1	_____	0s	70ms/step
1/1	_____	0s	60ms/step
1/1	_____	0s	71ms/step
1/1	_____	0s	70ms/step
1/1	_____	0s	66ms/step
1/1	_____	0s	62ms/step
1/1	_____	0s	64ms/step
1/1	_____	0s	65ms/step
1/1	_____	0s	81ms/step
1/1	_____	0s	63ms/step
1/1	_____	0s	63ms/step
1/1	_____	0s	76ms/step
1/1	_____	0s	71ms/step
1/1	_____	0s	72ms/step
1/1	_____	0s	71ms/step
1/1	_____	0s	69ms/step
1/1	_____	0s	62ms/step
1/1	_____	0s	65ms/step
1/1	_____	0s	78ms/step
1/1	_____	0s	76ms/step
1/1	_____	0s	77ms/step
1/1	_____	0s	72ms/step
1/1	_____	0s	68ms/step
1/1	_____	0s	65ms/step
1/1	_____	0s	65ms/step
1/1	_____	0s	86ms/step
1/1	_____	0s	68ms/step
1/1	_____	0s	66ms/step
1/1	_____	0s	63ms/step
1/1	_____	0s	65ms/step
1/1	_____	0s	63ms/step
1/1	_____	0s	68ms/step
1/1	_____	0s	77ms/step
1/1	_____	0s	69ms/step
1/1	_____	0s	66ms/step
1/1	_____	0s	60ms/step
1/1	_____	0s	64ms/step
1/1	_____	0s	68ms/step
1/1	_____	0s	62ms/step
1/1	_____	0s	60ms/step



Đặc trưng ảnh hưởng nhất: return_rate | mức tăng RMSE ≈ 41.8

Giải thích code

Ước lượng đặc trưng nào ảnh hưởng đến dự đoán bằng cách xáo trộn từng đặc trưng trên tập test.

Với mỗi trong 8 đặc trưng, lặp 5 lần: sao chép `X_test`, hoán vị ngẫu nhiên giá trị của đặc trưng đó giữa các mẫu (phá quan hệ với nhãn), dự đoán lại và tính mức tăng RMSE so với gốc (`base_rmse`). Lấy trung bình 5 lần rồi vẽ cột ngang; giá trị dương lớn nghĩa là mô hình phụ thuộc nhiều vào đặc trưng đó.

Giải thích output

Vì mỗi lần gọi `predict` in một dòng tiến trình nên có rất nhiều dòng `'1/1 ... step'` (đã rút gọn trong ảnh).

Biểu đồ: `return_rate` tăng RMSE nhiều nhất ($\approx +41,8$), tiếp theo `dow_cos` (khoảng $+33$), `dow_sin` (khoảng $+5$); các đặc trưng còn lại có mức tăng âm (khoảng -65 cho `month_cos`, -60 cho `month_sin`, -37 `avg_order`, -24 `revenue`). Mức tăng lớn nhất chỉ $\approx 41,8$ trên RMSE 39.318, tức $0,1\%$, vô nghĩa về thực tế. Giá trị âm nghĩa là xáo trộn đặc trưng đó lại làm RMSE giảm nhẹ, chứng tỏ đặc trưng chỉ mang nhiều và mô hình gần như trả về hằng số. Kết luận '`return_rate` quan trọng nhất' chỉ mang tính hình thức.

j) Dự báo tương lai

Dự báo đệ quy HORIZON ngày sau ngày cuối cùng của dữ liệu bằng mô hình tốt nhất.

Cell 29: Dự báo đệ quy 14 ngày tới

Code

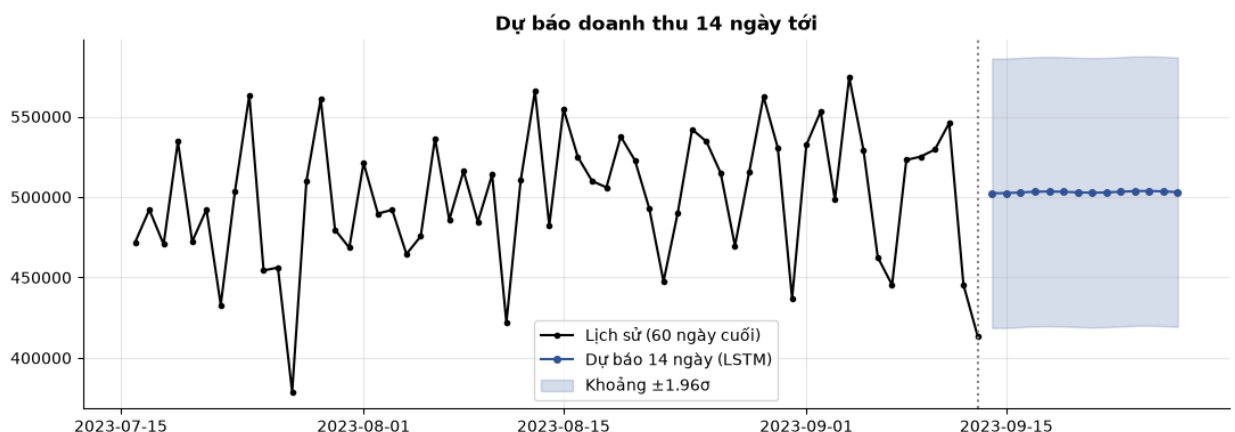
```
def forecast_future(model, horizon):
    """Dự báo đệ quy: đưa dự đoán của ngày t làm đầu vào cho ngày t+1. Đặc trưng lịch được biết trước."""
    window = arr_s[-WINDOW:].copy()
    rows = []
    for h in range(1, horizon + 1):
        d = dates[-1] + pd.Timedelta(days=h)
        p = predict_scaled(model, window[None, ...])[0] # (OUT_DIM,) đã chuẩn hóa
        cal = calendar_feats(pd.DatetimeIndex([d])).values[0]
        cal_s = (cal - scaler.mean_[OUT_DIM:]) / scaler.scale_[OUT_DIM:]
        window = np.vstack([window[1:], np.concatenate([p, cal_s]).astype("float32")])
        rows.append((d, float(inv_rev(p[0]))))
    return pd.DataFrame(rows, columns=["Ngày", "Doanh thu dự báo"]).set_index("Ngày")

fc = forecast_future(models[best_kind], HORIZON)
sigma = (rev_true["val"] - preds["val"][best_kind]).std() # độ lệch chuẩn sai số trên tập val
fc["Cận dưới (95%)"] = fc["Doanh thu dự báo"] - 1.96 * sigma
fc["Cận trên (95%)"] = fc["Doanh thu dự báo"] + 1.96 * sigma

hist_tail = daily["revenue"].iloc[-60:]
fig, ax = plt.subplots(figsize=(13, 4.2))
ax.plot(hist_tail.index, hist_tail.values, c="k", marker="o", ms=3, label="Lịch sử (60 ngày cuối)")
ax.plot(fc.index, fc["Doanh thu dự báo"], c=colors[best_kind], marker="o", ms=4, label=f"Dự báo {HORIZON} ngày ({best_kind})")
ax.fill_between(fc.index, fc["Cận dưới (95%)"], fc["Cận trên (95%)"], color=colors[best_kind], alpha=0.2,
               label="Khoảng ±1.96σ")
ax.axvline(dates[-1], c="gray", ls=":"); ax.set_title(f"Dự báo doanh thu {HORIZON} ngày tới"); ax.legend();
plt.show()
display(fc.round(0))
print("Lưu ý: dự báo đệ quy nhiều bước tích lũy sai số; khoảng tin cậy chỉ mang tính tham khảo.")
```

Output

```
1/1 ————— 0s 363ms/step
1/1 ————— 0s 39ms/step
1/1 ————— 0s 43ms/step
1/1 ————— 0s 44ms/step
1/1 ————— 0s 44ms/step
1/1 ————— 0s 40ms/step
1/1 ————— 0s 41ms/step
1/1 ————— 0s 41ms/step
1/1 ————— 0s 47ms/step
1/1 ————— 0s 41ms/step
1/1 ————— 0s 40ms/step
1/1 ————— 0s 41ms/step
1/1 ————— 0s 51ms/step
1/1 ————— 0s 43ms/step
```



	Doanh thu dự báo	Cận dưới (95%)	Cận trên (95%)
Ngày			
2023-09-14	502,347.0000	418,637.0000	586,058.0000
2023-09-15	502,414.0000	418,704.0000	586,124.0000
2023-09-16	502,878.0000	419,168.0000	586,588.0000
2023-09-17	503,308.0000	419,598.0000	587,019.0000
2023-09-18	503,458.0000	419,747.0000	587,168.0000
2023-09-19	503,283.0000	419,573.0000	586,994.0000
2023-09-20	502,903.0000	419,193.0000	586,613.0000
2023-09-21	502,677.0000	418,967.0000	586,387.0000
2023-09-22	502,839.0000	419,129.0000	586,550.0000
2023-09-23	503,299.0000	419,588.0000	587,009.0000
2023-09-24	503,705.0000	419,994.0000	587,415.0000
2023-09-25	503,785.0000	420,075.0000	587,496.0000
2023-09-26	503,503.0000	419,793.0000	587,213.0000
2023-09-27	503,023.0000	419,313.0000	586,733.0000

Lưu ý: dự báo đệ quy nhiều bước tích lũy sai số; khoảng tin cậy chỉ mang tính tham khảo.

Giải thích code

Dự báo 14 ngày sau ngày cuối của dữ liệu và dựng khoảng biến động.

forecast_future lấy 30 dòng cuối đã chuẩn hóa làm cửa sổ; mỗi bước dự đoán 4 đại lượng của ngày kế tiếp, tính đặc trưng lịch của ngày đó (biết trước), chuẩn hóa bằng thống kê của scaler, rồi đẩy vào cuối cửa sổ và bỏ dòng đầu (dự đoán được dùng làm đầu vào cho bước sau, gọi là đệ quy). Kết quả đổi về doanh thu gốc bằng inv_rev.

sigma là độ lệch chuẩn sai số trên tập val; khoảng 95% = dự báo $\pm 1,96\sigma$. Đồ thị vẽ 60 ngày lịch sử cuối, đường dự báo và dải mờ; display(fc.round(0)) in bảng số.

Giải thích output

14 dòng '1/1 ... step' ứng với 14 lần predict. Dự báo gần như phẳng: từ 502.347 (14/09/2023) đến khoảng 503.785 (25/09/2023), tức quanh mức trung bình lịch sử (~504 nghìn). Khoảng 95% rộng $\pm 83,7$ nghìn (từ khoảng 418,6 nghìn đến 586,1 nghìn), rất rộng so với độ dịch chuyển của đường dự báo vài trăm đơn vị, cho thấy mô hình thực chất dự báo 'mức trung bình' và độ bất định do nhiễu ngẫu nhiên chiếm trọn. Dòng cuối nhắc dự báo nhiều bước tích lũy sai số.

k) Kết luận

Cell 31: Kết luận tự động

Code

```
# ---- Kết luận tự động dựa trên kết quả thực tế ----
best_rmse = results.loc[best_kind, "RMSE test"]
naive_rmse = results.loc["Naive (hôm qua)", "RMSE test"]
ma_rmse = results.loc["TB 7 ngày", "RMSE test"]
gain = (1 - best_rmse / min(naive_rmse, ma_rmse)) * 100
print(f"RMSE test - {best_kind}: {best_rmse:,.0f} | Naive: {naive_rmse:,.0f} | TB 7 ngày: {ma_rmse:,.0f}")
if gain > 3:
    print(f"=> {best_kind} tốt hơn baseline tốt nhất khoảng {gain:.1f}%: chuỗi có cấu trúc mà RNN khai thác được.")
elif gain > -3:
    print(f"=> {best_kind} chỉ tương đương baseline (chênh {gain:+.1f}%): RNN không mang lại lợi ích rõ rệt.")
else:
    print(f"=> {best_kind} kém baseline {abs(gain):.1f}%: nên ưu tiên mô hình đơn giản (trung bình trượt) cho chuỗi này.")
print("R² test của mô hình tốt nhất:", round(results.loc[best_kind, "R² test"], 3),
      "(gần 0 hoặc âm nghĩa là chưa giải thích được biến thiên ngoài mức trung bình)")
```

Output

```
RMSE test - LSTM: 39,318 | Naive: 55,658 | TB 7 ngày: 41,844
=> LSTM tốt hơn baseline tốt nhất khoảng 6.0%: chuỗi có cấu trúc mà RNN khai thác được.
R² test của mô hình tốt nhất: -0.021 (gần 0 hoặc âm nghĩa là chưa giải thích được biến thiên ngoài mức trung bình)
```

Giải thích code

So sánh RMSE của mô hình tốt nhất với baseline tốt nhất và in nhận định.

$gain = (1 - RMSE_{\text{mô hình}} / \min(RMSE_{\text{naive}}, RMSE_{\text{TB7}})) \times 100$. Nếu $gain > 3\%$ thì kết luận 'tốt hơn baseline', từ -3% đến 3% là 'tương đương', còn lại là 'kém hơn'. Ngoài ra in R^2 test.

Giải thích output

LSTM có RMSE 39.318 so với Naive 55.658 và TB 7 ngày 41.844, tốt hơn baseline tốt nhất 6,0% nên code in 'có cấu trúc mà RNN khai thác được'. R^2 test = $-0,021$ (gần 0, hơi âm).

Cần diễn giải cẩn trọng: mức 6% đến từ việc dự đoán phẳng quanh trung bình (mà trung bình dài hạn chính xác hơn trung bình 7 ngày và hơn Naive), chứ không phải mô hình dự đoán được biến động từng ngày. R^2 âm nhẹ xác nhận điều đó. Kết luận trung thực: RNN chỉ ngang với dự đoán hằng số, chuỗi không có tín hiệu để học.

1.2. Phiên bản PyTorch

Dữ liệu: **E-commerce Customer Data For Behavior Analysis** (Kaggle) – bảng giao dịch, mỗi dòng là một giao dịch.

Ý tưởng: dữ liệu gốc là dạng bảng nên trước hết **tổng hợp theo ngày** thành chuỗi thời gian đa biến (doanh thu, số đơn, giá trị đơn trung bình, tỷ lệ trả hàng, đặc trưng lịch). Sau đó dùng cửa sổ trượt WINDOW ngày để dự đoán các đại lượng của **ngày kế tiếp**, và so sánh ba kiến trúc RNN, LSTM, GRU với hai mô hình cơ sở (Naive, trung bình 7 ngày).

Quy trình: 1) Nạp và kiểm tra dữ liệu → 2) Phân tích khám phá (EDA) → 3) Tạo chuỗi thời gian, kiểm tra cấu trúc → 4) Chia theo thời gian, chuẩn hóa, tạo cửa sổ → 5) Baseline → 6) Huấn luyện 3 mô hình → 7) Đánh giá và trực quan hóa → 8) Phân tích đặc trưng → 9) Dự báo tương lai → 10) Kết luận.

a) Thư viện và cấu hình

Cell 2: Nhập thư viện và thiết lập hiển thị

Code

```
import os, glob, math, random, warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display
from sklearn.preprocessing import StandardScaler

warnings.filterwarnings("ignore")
plt.rcParams.update({
    "figure.dpi": 100, "figure.figsize": (11, 4),
    "axes.grid": True, "grid.alpha": 0.3,
    "axes.spines.top": False, "axes.spines.right": False,
    "axes.titlesize": 12, "axes.titleweight": "bold",
})
pd.set_option("display.float_format", lambda v: f"{v:., .4f}")
pd.set_option("display.max_columns", 30)
```

Cell này không tạo output hiển thị.

Giải thích code

Code giống hệt cell 2 của bản Keras (mục 1.1). Nhập các thư viện nền tảng và đặt cấu hình chung cho toàn bộ notebook.

Cell 3: Khai báo tham số cấu hình

Code

```
# ===== CẤU HÌNH (có thể chỉnh) =====
DATA_PATH = "../data/ecommerce_customer_data_large.csv" # <-- đường dẫn tới tập CSV đã tải về
WINDOW    = 30      # số ngày lịch sử đưa vào RNN
HORIZON    = 14      # số ngày dự báo tương lai
TRAIN_RATIO = 0.70    # tỷ lệ tập huấn luyện (theo thời gian)
VAL_RATIO  = 0.15     # tỷ lệ tập kiểm định; phần còn lại là tập kiểm thử

UNITS      = 64      # số đơn vị ẩn mỗi lớp
DROPOUT    = 0.2
LR         = 1e-3
BATCH_SIZE = 32
EPOCHS     = int(os.environ.get("NB_EPOCHS", 100)) # tối đa; có dùng sớm nên thường ít hơn
PATIENCE   = 12
SEED       = 42
```

Cell này không tạo output hiển thị.

Giải thích code

Code giống hết cell 3 của bản Keras (mục 1.1). Gõ toàn bộ siêu tham số vào một chỗ để dễ chỉnh sửa.

Cell 4: Nạp PyTorch, cố định seed, chọn thiết bị

Code

```
import copy
import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader

random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("PyTorch:", torch.__version__, "| Thiết bị:", device)
```

Output

```
PyTorch: 2.14.0+cu126 | Thiết bị: cuda
```

Giải thích code

Nạp PyTorch thay cho TensorFlow: torch, torch.nn, TensorDataset và DataLoader (tạo batch), copy (lưu trọng số tốt nhất).

torch.manual_seed(SEED) cố định seed; device chọn cuda nếu có GPU, ngược lại dùng cpu.

Giải thích output

PyTorch 2.14.0+cu126 và thiết bị cuda: bản PyTorch chạy trên GPU (CUDA 12.6), khác với bản Keras chạy CPU. Do khác framework, khác thiết bị và cách khởi tạo trọng số, số liệu huấn luyện của hai bản sẽ không trùng nhau dù dùng chung seed và dữ liệu.

Cell 5: Hàm tiện ích: tìm tập dữ liệu và tự tương quan (ACF)

Code

```
def find_data_file(path, pattern):
    """Trả về đường dẫn tệp dữ liệu. Nếu DATA_PATH không tồn tại thì tự tìm theo mẫu tên trong thư mục hiện tại."""
    if os.path.exists(path):
        return path
    found = glob.glob(pattern) + glob.glob(os.path.join("...", pattern), recursive=True)
    found = sorted(set(found), key=lambda p: ("large" not in p.lower(), len(p), p))
    if not found:
        raise FileNotFoundError(
            f"Không tìm thấy '{path}'. Hãy sửa biến DATA_PATH ở ô cấu hình thành đường dẫn tới tệp CSV đã tải về.")
    print(f"Không tìm thấy '{path}', tự động dùng: {found[0]}")
    return found[0]

def acf(x, nlags):
    """Hàm tự tương quan (ACF) tính trực tiếp bằng numpy."""
    x = np.asarray(x, dtype=float)
    x = x - x.mean()
    d = np.dot(x, x)
    return np.array([np.dot(x[:len(x) - k], x[k:]) / d for k in range(nlags + 1)])

def plot_acf(ax, x, nlags, title):
    a = acf(x, nlags)
    conf = 1.96 / np.sqrt(len(x))
    ax.bar(range(nlags + 1), a, width=0.3, color="#2F5496")
    ax.axhline(conf, ls="--", c="r", lw=1); ax.axhline(-conf, ls="--", c="r", lw=1)
    ax.axhline(0, c="k", lw=0.8)
    ax.set_title(title); ax.set_xlabel("Độ trễ (lag)"); ax.set_ylabel("ACF")
    return a, conf
```

Cell này không tạo output hiển thị.

Giải thích code

Code giống hệt cell 5 của bản Keras (mục 1.1). Định nghĩa ba hàm dùng lại nhiều lần: `find_data_file`, `acf` và `plot_acf`.

b) Nạp dữ liệu và kiểm tra chất lượng

Cell 7: Nạp dữ liệu và kiểm tra ban đầu

Code

```
DATA_PATH = find_data_file(DATA_PATH, "ecommerce*.csv")
df = pd.read_csv(DATA_PATH)
df.columns = df.columns.str.strip()

required = ["Purchase Date", "Total Purchase Amount"]
missing_cols = [c for c in required if c not in df.columns]
assert not missing_cols, f"Thiếu cột bắt buộc: {missing_cols}. Các cột hiện có: {list(df.columns)}"

df["Purchase Date"] = pd.to_datetime(df["Purchase Date"], errors="coerce")
df["Total Purchase Amount"] = pd.to_numeric(df["Total Purchase Amount"], errors="coerce")
if "Returns" in df.columns:
    df["Returns"] = pd.to_numeric(df["Returns"], errors="coerce")
n_bad = df["Purchase Date"].isna().sum() + df["Total Purchase Amount"].isna().sum()
df = df.dropna(subset=["Purchase Date", "Total Purchase Amount"]).reset_index(drop=True)

print(f"Số dòng (giao dịch): {len(df):,} | Số cột: {df.shape[1]}")
print(f"Khoảng thời gian: {df['Purchase Date'].min().date()} -> {df['Purchase Date'].max().date()}")
if "Customer ID" in df.columns:
    print(f"Số khách hàng duy nhất: {df['Customer ID'].nunique():,}")
print(f"Dòng bị loại vì thiếu ngày/số tiền: {n_bad}")
display(df.head())
display(df.dtypes.to_frame("Kiểu dữ liệu").T)
```

Output

Số dòng (giao dịch): 250,000 | Số cột: 13
Khoảng thời gian: 2020-01-01 -> 2023-09-13
Số khách hàng duy nhất: 49,661
Dòng bị loại vì thiếu ngày/số tiền: 0

	Customer ID	Purchase Date	Product Category	Product Price	Quantity	\
0	44605	2023-05-03 21:30:02	Home	177	1	
1	44605	2021-05-16 13:57:44	Electronics	174	3	
2	44605	2020-07-13 06:16:57	Books	413	1	
3	44605	2023-01-17 13:14:36	Electronics	396	3	
4	44605	2021-05-01 11:29:27	Books	259	4	

	Total Purchase Amount	Payment Method	Customer Age	Returns	Customer Name	\
0	2427	PayPal	31	1.0000	John Rivera	
1	2448	PayPal	31	1.0000	John Rivera	
2	2345	Credit Card	31	1.0000	John Rivera	
3	937	Cash	31	0.0000	John Rivera	
4	2598	PayPal	31	1.0000	John Rivera	

	Age	Gender	Churn
0	31	Female	0
1	31	Female	0
2	31	Female	0
3	31	Female	0
4	31	Female	0

	Customer ID	Purchase Date	Product Category	Product Price	\
Kiểu dữ liệu	int64	datetime64[us]	str	int64	

	Quantity	Total Purchase Amount	Payment Method	Customer Age	\
Kiểu dữ liệu	int64	int64	str	int64	

	Returns	Customer Name	Age	Gender	Churn
Kiểu dữ liệu	float64	str	int64	str	int64

Giải thích code

Code giống hệt cell 7 của bản Keras (mục 1.1). Đọc CSV, kiểm tra cột bắt buộc, ép kiểu và loại các dòng lỗi.

Giải thích output

Output giống hệt bản Keras (mục 1.1, cell 7). Bảng dữ liệu có 250.000 giao dịch và 13 cột, trải từ 01/01/2020 đến 13/09/2023, thuộc 49.661 khách hàng. Không có dòng nào bị loại (0), tức cột ngày và số tiền sạch hoàn toàn. Chi tiết đầy đủ xem giải thích ở mục 1.1.

Cell 8: Kiểm tra chất lượng dữ liệu

Code

```
# --- Chất lượng dữ liệu ---
na = df.isna().mean().mul(100).round(2)
print("Tỷ lệ giá trị thiếu theo cột (%):"); display(na[na > 0].to_frame("% thiếu").T if (na > 0).any() else
    "Không có giá trị thiếu")
print("Số dòng trùng lặp hoàn toàn:", df.duplicated().sum())

if {"Product Price", "Quantity"} <= set(df.columns):
    diff = (pd.to_numeric(df["Product Price"], errors="coerce") * pd.to_numeric(df["Quantity"], errors="coerce")
            - df["Total Purchase Amount"]).abs()
    print(f"Tỷ lệ dòng mà Total != Price x Quantity: {(diff > 1e-6).mean():.2%} (nếu cao, cần chọn cột tiền
        đáng tin)")

if {"Customer ID", "Churn"} <= set(df.columns):
    n_lab = df.groupby("Customer ID")["Churn"].nunique()
    print(f"Khách có nhãn Churn không nhất quán giữa các giao dịch: {(n_lab > 1).mean():.2%}")

display(df["Total Purchase Amount"].describe().to_frame("Total Purchase Amount").T)
```

Output

Tỷ lệ giá trị thiếu theo cột (%):

```

      Returns
% thiếu  18.9500
Số dòng trùng lặp hoàn toàn: 0
Tỷ lệ dòng mà Total != Price x Quantity: 99.98% (nếu cao, cần chọn cột tiền đáng tin)
Khách có nhãn Churn không nhất quán giữa các giao dịch: 0.00%

      count      mean      std      min      25%  \
Total Purchase Amount  250,000.0000  2,725.3852  1,442.5761  100.0000  1,476.0000

      50%      75%      max
Total Purchase Amount  2,725.0000  3,975.0000  5,350.0000
```

Giải thích code

Code giống hệt cell 8 của bản Keras (mục 1.1). Thống kê giá trị thiếu, dòng trùng, tính nhất quán của cột tiền và nhãn Churn.

Giải thích output

Output giống hệt bản Keras (mục 1.1, cell 8). Chỉ cột Returns có giá trị thiếu (18,95%). Không có dòng trùng. Đáng chú ý nhất là 99,98% dòng có $Total \neq Price \times Quantity$: cột tổng tiền không suy ra từ đơn giá và số lượng, tức dữ liệu là dữ liệu tổng hợp (sinh ngẫu nhiên). Notebook vì vậy chọn Total Purchase Amount làm doanh thu, đúng như gợi ý trong dòng in. Nhãn Churn nhất quán tuyệt đối theo từng khách (0,00% không nhất quán). Chi tiết đầy đủ xem giải thích ở mục 1.1.

c) Phân tích khám phá (EDA) trên dữ liệu giao dịch

Xem cơ cấu doanh thu, phương thức thanh toán, churn, tuổi và tỷ lệ trả hàng để hiểu bức tranh chung trước khi dựng chuỗi thời gian.

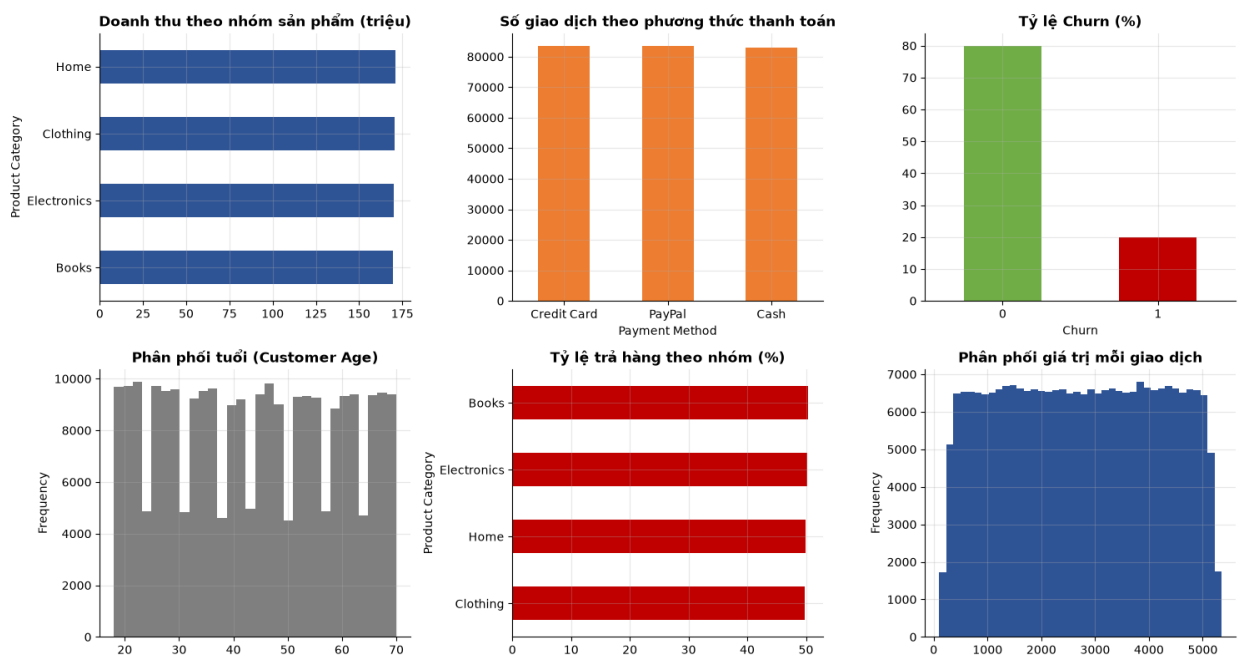
Cell 10: EDA trên dữ liệu giao dịch (6 biểu đồ)

Code

```
# --- Trực quan hóa dữ liệu giao dịch ---
age_col = "Customer Age" if "Customer Age" in df.columns else ("Age" if "Age" in df.columns else None)
panels = []
if "Product Category" in df.columns: panels.append("cat_rev")
if "Payment Method" in df.columns: panels.append("pay")
if "Churn" in df.columns: panels.append("churn")
if age_col: panels.append("age")
if {"Product Category", "Returns"} <= set(df.columns): panels.append("ret")
panels.append("amount")

ncols = 3; nrows = math.ceil(len(panels) / ncols)
fig, axes = plt.subplots(nrows, ncols, figsize=(15, 4 * nrows)); axes = np.atleast_1d(axes).ravel()
for ax, p in zip(axes, panels):
    if p == "cat_rev":
        s = df.groupby("Product Category")["Total Purchase Amount"].sum().sort_values()
        (s / 1e6).plot.barh(ax=ax, color="#2F5496"); ax.set_title("Doanh thu theo nhóm sản phẩm (triệu)")
    elif p == "pay":
        df["Payment Method"].value_counts().plot.bar(ax=ax, color="#ED7D31"); ax.set_title("Số giao dịch theo phương thức thanh toán")
        ax.tick_params(axis="x", rotation=0)
    elif p == "churn":
        df["Churn"].value_counts(normalize=True).sort_index().mul(100).plot.bar(ax=ax, color=["#70AD47", "#C00000"]); ax.set_title("Tỷ lệ Churn (%)"); ax.tick_params(axis="x", rotation=0)
    elif p == "age":
        df[age_col].plot.hist(bins=30, ax=ax, color="#7F7F7F"); ax.set_title(f"Phân phối tuổi ({age_col})")
    elif p == "ret":
        (df.groupby("Product Category")["Returns"].mean() * 100).sort_values().plot.barh(ax=ax, color="#C00000"); ax.set_title("Tỷ lệ trả hàng theo nhóm (%)")
    elif p == "amount":
        df["Total Purchase Amount"].plot.hist(bins=40, ax=ax, color="#2F5496"); ax.set_title("Phân phối giá trị mỗi giao dịch")
for ax in axes[len(panels):]: ax.axis("off")
plt.tight_layout(); plt.show()
```

Output



Giải thích code

Code giống hệt cell 10 của bản Keras (mục 1.1). Vẽ lưới 2 hàng $\times$ 3 cột biểu đồ mô tả tổng quan giao dịch.

Giải thích output

Output giống hệt bản Keras (mục 1.1, cell 10). Cả sáu biểu đồ đều 'quá đều': doanh thu bốn nhóm Home, Clothing, Electronics, Books gần bằng nhau (khoảng 165 đến 170 triệu mỗi nhóm); số giao dịch của Credit Card, PayPal, Cash gần như bằng nhau (khoảng 83.000); tỷ lệ trả hàng mọi nhóm xấp xỉ 50%; tuổi trải đều từ 18 đến 70; giá trị mỗi giao dịch phân phối phẳng từ 100 đến khoảng 5.350. Riêng

Churn lịch: khoảng 80% không rời bỏ và 20% rời bỏ. Chi tiết đầy đủ xem giải thích ở mục 1.1.

d) Xây dựng chuỗi thời gian theo ngày và kiểm tra cấu trúc

RNN chỉ có ích khi chuỗi có phụ thuộc theo thời gian. Ta kiểm tra xu hướng, mùa vụ theo thứ, tự tương quan (ACF).

Cell 12: Tổng hợp giao dịch thành chuỗi thời gian theo ngày

Code

```
# ---- Tổng hợp giao dịch thành chuỗi thời gian theo NGÀY ----
orders = df.set_index("Purchase Date").sort_index()
daily = orders.resample("D").agg(
    revenue=("Total Purchase Amount", "sum"),
    orders=("Total Purchase Amount", "size"),
    avg_order=("Total Purchase Amount", "mean"),
)
daily["return_rate"] = orders["Returns"].resample("D").mean() if "Returns" in orders.columns else 0.0
daily[["avg_order", "return_rate"]] = daily[["avg_order", "return_rate"]].ffill().bfill() # ngày không có đơn
n_empty = int((daily["orders"] == 0).sum())

def calendar_feats(idx):
    """Đặc trưng lịch (biết trước cả trong tương lai): mã hóa vòng bằng sin/cos."""
    dow = np.asarray(idx.dayofweek); mon = np.asarray(idx.month) - 1
    return pd.DataFrame({
        "dow_sin": np.sin(2 * np.pi * dow / 7), "dow_cos": np.cos(2 * np.pi * dow / 7),
        "month_sin": np.sin(2 * np.pi * mon / 12), "month_cos": np.cos(2 * np.pi * mon / 12),
    }, index=idx)

daily = daily.join(calendar_feats(daily.index))
print(f"Chuỗi theo ngày: {len(daily)} ngày | Ngày không có giao dịch: {n_empty}")
display(daily.head())
display(daily[["revenue", "orders", "avg_order", "return_rate"]].describe().T)
```

Output

Chuỗi theo ngày: 1352 ngày | Ngày không có giao dịch: 0

	revenue	orders	avg_order	return_rate	dow_sin	dow_cos	\
Purchase Date							
2020-01-01	464688	179	2,596.0223	0.5143	0.9749	-0.2225	
2020-01-02	546673	196	2,789.1480	0.5427	0.4339	-0.9010	
2020-01-03	498883	176	2,834.5625	0.5105	-0.4339	-0.9010	
2020-01-04	504343	186	2,711.5215	0.5031	-0.9749	-0.2225	
2020-01-05	500861	190	2,636.1105	0.5064	-0.7818	0.6235	
		month_sin	month_cos				
Purchase Date							
2020-01-01		0.0000	1.0000				
2020-01-02		0.0000	1.0000				
2020-01-03		0.0000	1.0000				
2020-01-04		0.0000	1.0000				
2020-01-05		0.0000	1.0000				
	count	mean	std	min	25%	\	
revenue	1,352.0000	503,954.3632	42,592.3069	357,710.0000	474,821.0000		
orders	1,352.0000	184.9112	14.0778	136.0000	175.0000		
avg_order	1,352.0000	2,725.5420	103.7020	2,307.8065	2,653.5183		
return_rate	1,352.0000	0.5008	0.0401	0.3608	0.4730		
	50%	75%	max				
revenue	504,141.5000	530,477.5000	649,755.0000				
orders	185.0000	194.0000	235.0000				
avg_order	2,729.0090	2,798.9239	3,042.5223				
return_rate	0.5000	0.5295	0.6479				

Giải thích code

Code giống hệt cell 12 của bản Keras (mục 1.1). Biến bảng giao dịch thành chuỗi thời gian đa biến theo ngày, thêm đặc trưng lịch.

Giải thích output

Output giống hệt bản Keras (mục 1.1, cell 12). Chuỗi dài 1.352 ngày liên tục và không có ngày nào thiếu giao dịch (0 ngày trống), nên không phải nội suy. Chi tiết đầy đủ xem giải thích ở mục 1.1.

Cell 13: Trực quan hóa cấu trúc chuỗi thời gian (3 hình)

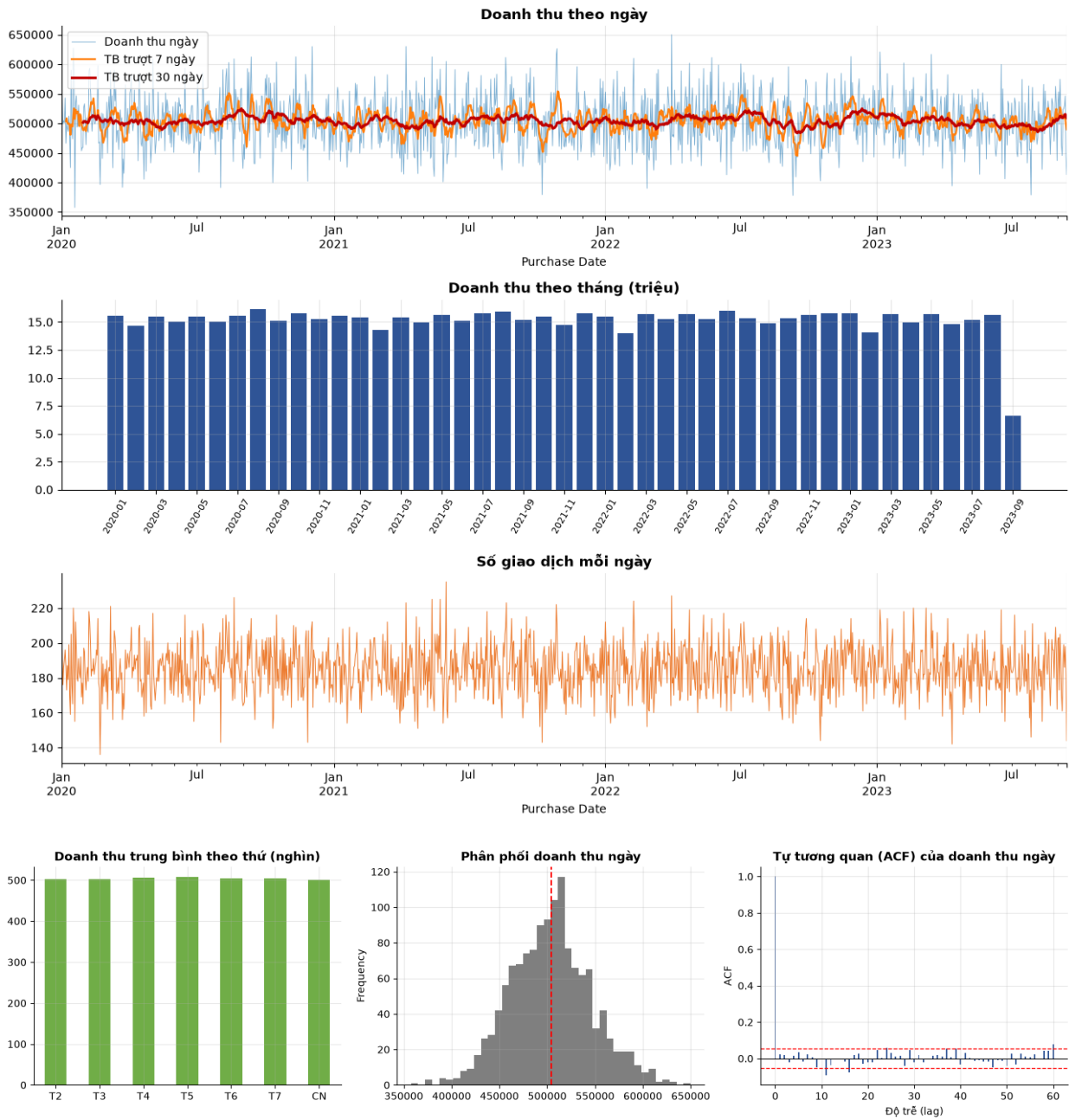
Code

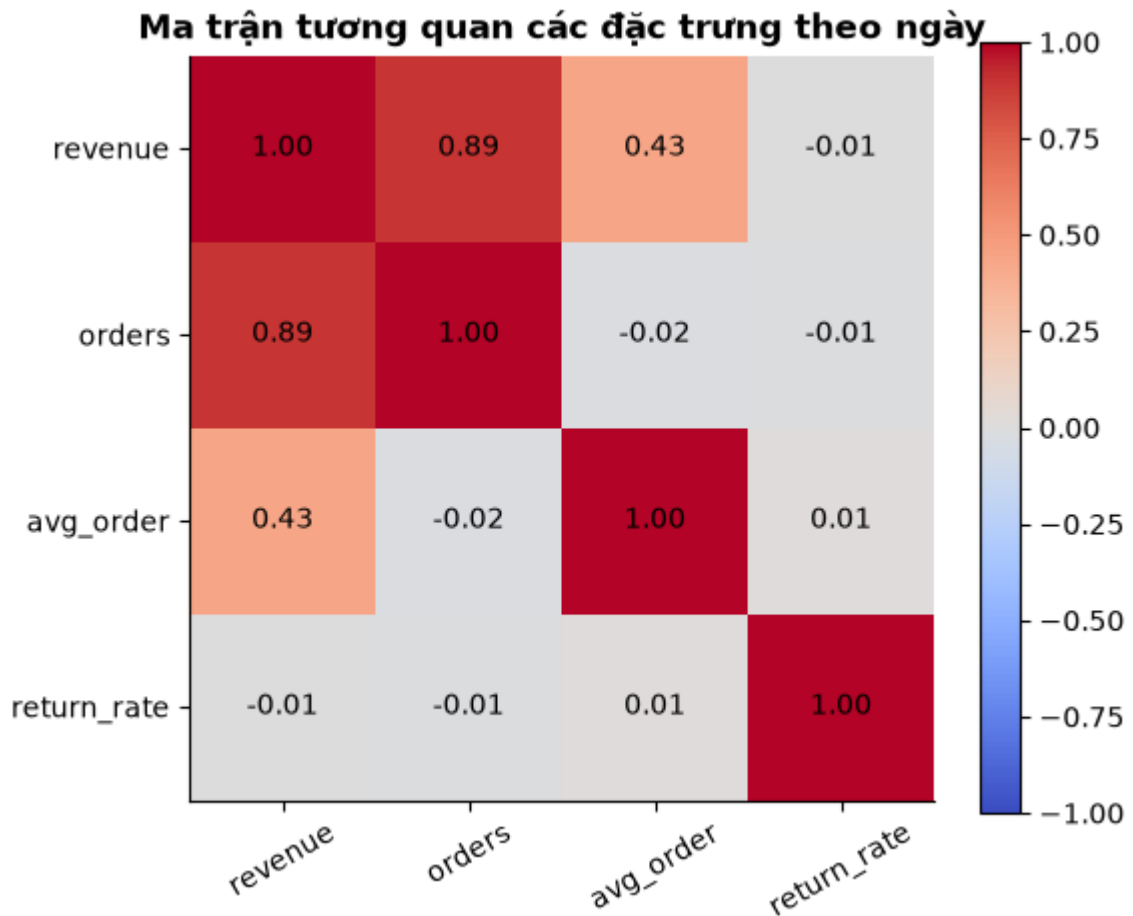
```
fig, axes = plt.subplots(3, 1, figsize=(13, 10), sharex=False)
daily["revenue"].plot(ax=axes[0], lw=0.6, alpha=0.6, label="Doanh thu ngày")
daily["revenue"].rolling(7).mean().plot(ax=axes[0], lw=1.6, label="TB trượt 7 ngày")
daily["revenue"].rolling(30).mean().plot(ax=axes[0], lw=2.2, c="#C00000", label="TB trượt 30 ngày")
axes[0].set_title("Doanh thu theo ngày"); axes[0].legend(loc="upper left")
monthly = daily["revenue"].resample("MS").sum() / 1e6
axes[1].bar(range(len(monthly)), monthly.values, color="#2F5496")
step = max(1, len(monthly) // 20)
axes[1].set_xticks(range(0, len(monthly), step))
axes[1].set_xticklabels([d.strftime("%Y-%m") for d in monthly.index[::step]], rotation=60, fontsize=8)
axes[1].set_title("Doanh thu theo tháng (triệu)")
daily["orders"].plot(ax=axes[2], lw=0.7, c="#ED7D31"); axes[2].set_title("Số giao dịch mỗi ngày")
plt.tight_layout(); plt.show()

fig, axes = plt.subplots(1, 3, figsize=(15, 4))
wd = daily.groupby(daily.index.dayofweek)["revenue"].mean()
wd.index = ["T2", "T3", "T4", "T5", "T6", "T7", "CN"]
(wd / 1e3).plot.bar(ax=axes[0], color="#70AD47"); axes[0].set_title("Doanh thu trung bình theo thứ (nghìn)")
axes[0].tick_params(axis="x", rotation=0)
daily["revenue"].plot.hist(bins=40, ax=axes[1], color="#7F7F7F"); axes[1].set_title("Phân phối doanh thu ngày")
axes[1].axvline(daily["revenue"].mean(), c="r", ls="--")
a, conf = plot_acf(axes[2], daily["revenue"], 60, "Tự tương quan (ACF) của doanh thu ngày")
plt.tight_layout(); plt.show()

fig, ax = plt.subplots(figsize=(6, 5))
cm = daily[["revenue", "orders", "avg_order", "return_rate"]].corr()
im = ax.imshow(cm, cmap="coolwarm", vmin=-1, vmax=1); ax.grid(False)
ax.set_xticks(range(4)); ax.set_xticklabels(cm.columns, rotation=30); ax.set_yticks(range(4));
ax.set_yticklabels(cm.columns)
for i in range(4):
    for j in range(4): ax.text(j, i, f"{cm.iloc[i, j]:.2f}", ha="center", va="center")
ax.set_title("Ma trận tương quan các đặc trưng theo ngày"); plt.colorbar(im); plt.show()
```

Output





Giải thích code

Code giống hệt cell 13 của bản Keras (mục 1.1). Vẽ ba hình để kiểm tra xu hướng, mùa vụ, phân phối, tự tương quan và tương quan giữa các đặc trưng.

Giải thích output

Output giống hệt bản Keras (mục 1.1, cell 13). Hình 1: doanh thu ngày dao động quanh 500 nghìn, đường trung bình trượt 30 ngày gần như nằm ngang, không có xu hướng tăng/giảm hay chu kỳ mùa vụ nào. Doanh thu các tháng đều xấp xỉ 15 triệu; cột cuối (tháng 9/2023) thấp hơn nhiều (khoảng 6,5 triệu) chỉ vì dữ liệu dừng ở ngày 13/09. Số giao dịch ngày dao động 140 đến 235. Chi tiết đầy đủ xem giải thích ở mục 1.1.

Cell 14: Nhận định tự động về cấu trúc chuỗi

Code

```
# ---- Nhận định tự động về cấu trúc chuỗi (quyết định RNN có "đắt tiền" hay không) ----
x = daily["revenue"].values
n = len(x)
sig_lags = [k for k in range(1, 31) if abs(a[k]) > conf]
slope = np.polyfit(np.arange(n), x, 1)[0] * 365
wd_cv = wd.std() / wd.mean()
print(f"- Tự tương quan lag-1 = {a[1]:.3f}, lag-7 = {a[7]:.3f} (ngưỡng ý nghĩa ±{conf:.3f})")
print(f"- Các lag (1-30) có ý nghĩa thống kê: {sig_lags if sig_lags else 'không có'}")
print(f"- Xu hướng tuyến tính: {slope:,.0f} đơn vị doanh thu/năm ({slope / x.mean():+.1%} so với mức trung bình)")
print(f"- Chênh lệch giữa các thứ trong tuần (hệ số biến thiên): {wd_cv:.1%}")
if not sig_lags and wd_cv < 0.05 and abs(slope / x.mean()) < 0.05:
    print("=> Chuỗi gần với nhiễu trắng: RNN khó vượt được mô hình cơ sở. Đây là một phát hiện hợp lệ, "
          "không phải lỗi mô hình (dữ liệu này là dữ liệu tổng hợp).")
else:
    print("=> Chuỗi có cấu trúc thời gian (tự tương quan/xu hướng/mùa vụ tuần): RNN có cơ sở để học.")
```

Output

```
- Tự tương quan lag-1 = 0.024, lag-7 = 0.023 (ngưỡng ý nghĩa ±0.053)
- Các lag (1-30) có ý nghĩa thống kê: [11, 16, 24]
- Xu hướng tuyến tính: -513 đơn vị doanh thu/năm (-0.1% so với mức trung bình)
- Chênh lệch giữa các thứ trong tuần (hệ số biến thiên): 0.5%
=> Chuỗi có cấu trúc thời gian (tự tương quan/xu hướng/mùa vụ tuần): RNN có cơ sở để học.
```

Giải thích code

Code giống hệt cell 14 của bản Keras (mục 1.1). Tính vài chỉ số định lượng rồi in kết luận: chuỗi có đáng để dùng RNN hay không.

Giải thích output

Output giống hệt bản Keras (mục 1.1, cell 14). Tự tương quan lag-1 là 0,024 và lag-7 là 0,023, đều rất nhỏ so với ngưỡng 0,053. Ba lag được coi là có ý nghĩa là 11, 16, 24. Xu hướng -513 đơn vị/năm (-0,1% so với trung bình), gần bằng 0. Chênh lệch giữa các thứ chỉ 0,5%. Chi tiết đầy đủ xem giải thích ở mục 1.1.

e) Chia dữ liệu theo thời gian, chuẩn hóa và tạo cửa sổ trượt

- Chia **theo trình tự thời gian** (70% / 15% / 15%), không xáo trộn để tránh rò rỉ thông tin tương lai.
- StandardScaler chỉ được **fit trên tập huấn luyện**.
- Mỗi mẫu là tensor (WINDOW, 8); nhãn là 4 đại lượng của ngày kế tiếp (doanh thu, số đơn, giá trị đơn TB, tỷ lệ trả hàng). Dự đoán đa đầu ra cho phép dự báo đệ quy nhiều ngày ở phần 9. Chỉ số đánh giá tập trung vào **doanh thu**.

Cell 16: Chia dữ liệu theo thời gian, chuẩn hóa, tạo cửa sổ trượt

Code

```
FEATURES = ["revenue", "orders", "avg_order", "return_rate", "dow_sin", "dow_cos", "month_sin", "month_cos"]
OUT_DIM = 4 # mô hình dự đoán 4 đại lượng đầu tiên của ngày kế tiếp
N_FEATURES = len(FEATURES)

arr = daily[FEATURES].values.astype("float32")
n = len(arr)
i1, i2 = int(n * TRAIN_RATIO), int(n * (TRAIN_RATIO + VAL_RATIO)) # mốc chia THEO THỜI GIAN

scaler = StandardScaler().fit(arr[:i1]) # chỉ ước lượng trên tập huấn luyện -> không rò rỉ
arr_s = scaler.transform(arr).astype("float32")
inv_rev = lambda z: z * scaler.scale_[0] + scaler.mean_[0] # đưa doanh thu về đơn vị gốc

X_all, y_all, pos = [], [], []
for i in range(WINDOW, n):
    X_all.append(arr_s[i - WINDOW:i]) # WINDOW ngày trước đó
    y_all.append(arr_s[i, :OUT_DIM]) # đặc trưng của ngày i
    pos.append(i)
X_all, y_all, pos = np.array(X_all, "float32"), np.array(y_all, "float32"), np.array(pos)

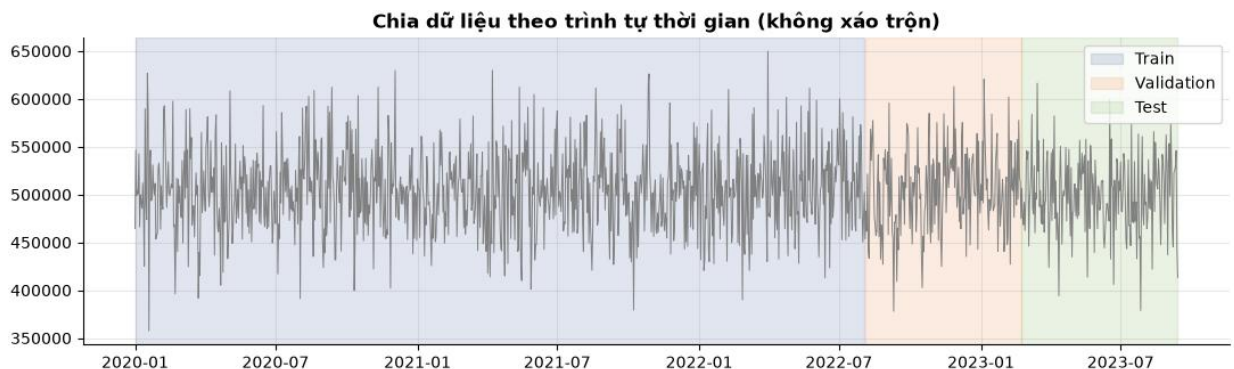
m_tr, m_va, m_te = pos < i1, (pos >= i1) & (pos < i2), pos >= i2
X_train, y_train = X_all[m_tr], y_all[m_tr]
X_val, y_val = X_all[m_va], y_all[m_va]
X_test, y_test = X_all[m_te], y_all[m_te]
raw_rev = daily["revenue"].values
dates = daily.index

print(f"Train: {X_train.shape} | Val: {X_val.shape} | Test: {X_test.shape}")
print(f"Train: {dates[WINDOW].date()} -> {dates[i1 - 1].date()}")
print(f"Val : {dates[i1].date()} -> {dates[i2 - 1].date()}")
print(f"Test : {dates[i2].date()} -> {dates[-1].date()}")

fig, ax = plt.subplots(figsize=(13, 3.5))
ax.plot(dates, raw_rev, lw=0.7, c="gray")
for lo, hi, c, lb in [(0, i1, "#2F5496", "Train"), (i1, i2, "#ED7D31", "Validation"), (i2, n, "#70AD47", "Test")]:
    ax.axvspan(dates[lo], dates[hi - 1], color=c, alpha=0.15, label=lb)
ax.set_title("Chia dữ liệu theo trình tự thời gian (không xáo trộn)"); ax.legend(); plt.show()
```

Output

```
Train: (916, 30, 8) | Val: (203, 30, 8) | Test: (203, 30, 8)
Train: 2020-01-31 -> 2022-08-03
Val : 2022-08-04 -> 2023-02-22
Test : 2023-02-23 -> 2023-09-13
```



Giải thích code

Code giống hệt cell 16 của bản Keras (mục 1.1). Tạo tập huấn luyện/kiểm định/kiểm thử dạng tensor (mẫu, 30, 8) mà không rò rỉ dữ liệu tương lai.

Giải thích output

Output giống hệt bản Keras (mục 1.1, cell 16). Train (916, 30, 8), Val (203, 30, 8), Test (203, 30, 8): tức 916, 203 và 203 mẫu, mỗi mẫu 30 ngày $\times$ 8 đặc trưng. Train từ 31/01/2020 đến 03/08/2022, Val từ 04/08/2022 đến 22/02/2023, Test từ

23/02/2023 đến 13/09/2023 (30 ngày đầu chỉ dùng làm lịch sử của mẫu đầu tiên).
Chi tiết đầy đủ xem giải thích ở mục 1.1.

f) Mô hình cơ sở (baseline)

Mọi mô hình RNN phải được so với hai baseline đơn giản: **Naive** (doanh thu hôm qua) và **trung bình 7 ngày**.

Cell 18: Mô hình cơ sở (baseline) và các hàm đo lỗi

Code

```
def rmse(y, p): return float(np.sqrt(np.mean((np.asarray(y) - np.asarray(p)) ** 2)))
def mae(y, p): return float(np.mean(np.abs(np.asarray(y) - np.asarray(p))))
def mape(y, p):
    y, p = np.asarray(y), np.asarray(p); m = y > 0
    return float(np.mean(np.abs((y[m] - p[m]) / y[m])) * 100)
def r2(y, p):
    y, p = np.asarray(y), np.asarray(p)
    return float(1 - np.sum((y - p) ** 2) / np.sum((y - y.mean()) ** 2))

split_pos = {"val": pos[m_va], "test": pos[m_te]}
rev_true = {k: raw_rev[v] for k, v in split_pos.items()}
preds = {"val": {}, "test": {}}
for k, v in split_pos.items():
    preds[k]["Naive (hôm qua)"] = raw_rev[v - 1]
    preds[k]["TB 7 ngày"] = np.array([raw_rev[p - 7:p].mean() for p in v])
print("Baseline tập test -> ", {k: round(rmse(rev_true['test'], v), 1) for k, v in preds['test'].items()})
```

Output

```
Baseline tập test -> {'Naive (hôm qua)': 55658.1, 'TB 7 ngày': 41844.4}
```

Giải thích code

Code giống hệt cell 18 của bản Keras (mục 1.1). Định nghĩa bốn thước đo và dựng hai mô hình cơ sở để làm mốc so sánh.

Giải thích output

Output giống hệt bản Keras (mục 1.1, cell 18). RMSE test: Naive 55.658 và trung bình 7 ngày 41.844. Naive tệ hơn nhiều vì chuỗi gần như nhiễu trắng: sai số của việc lấy giá trị hôm qua là hiệu của hai nhiễu độc lập nên phương sai gấp đôi (độ lệch chuẩn khoảng $42,6 \text{ nghìn} \times \sqrt{2} \approx 60 \text{ nghìn}$, gần với 55,7 nghìn quan sát được). Trung bình 7 ngày khá hơn vì trung bình hóa bớt nhiễu. Đây là mốc mà các mạng RNN phải vượt.

g) Xây dựng và huấn luyện RNN / LSTM / GRU (PyTorch)

Ba kiến trúc dùng cùng siêu tham số (2 lớp $\times$ UNITS đơn vị, dropout, Adam, MSE, dùng sớm, giảm learning rate khi chững) để so sánh công bằng.

Cell 20: Mô hình PyTorch và vòng huấn luyện tự viết

Code

```
class RNNRegressor(nn.Module):
    """2 lớp RNN/LSTM/GRU xếp chồng, lấy trạng thái ẩn của bước cuối -> Dropout -> MLP nhỏ."""
    def __init__(self, kind, n_features, out_dim, hidden=UNITS, n_layers=2, dropout=DROPOUT):
        super().__init__()
        cell = {"RNN": nn.RNN, "LSTM": nn.LSTM, "GRU": nn.GRU}[kind]
        self.rnn = cell(n_features, hidden, num_layers=n_layers, batch_first=True, dropout=dropout)
        self.drop = nn.Dropout(dropout)
        self.head = nn.Sequential(nn.Linear(hidden, 32), nn.ReLU(), nn.Linear(32, out_dim))

    def forward(self, x):
        out, _ = self.rnn(x)
        return self.head(self.drop(out[:, -1, :]))

def train_model(kind, X_tr, y_tr, X_va, y_va):
    torch.manual_seed(SEED)
    model = RNNRegressor(kind, X_tr.shape[2], y_tr.shape[1]).to(device)
    opt = torch.optim.Adam(model.parameters(), lr=LR)
    sched = torch.optim.lr_scheduler.ReduceLROnPlateau(
        opt, factor=0.5, patience=max(3, PATIENCE // 3), min_lr=1e-5
    )
    loss_fn = nn.MSELoss()
    loader = DataLoader(
        TensorDataset(torch.tensor(X_tr), torch.tensor(y_tr)),
        batch_size=BATCH_SIZE,
        shuffle=True
    )
    Xv, yv = torch.tensor(X_va).to(device), torch.tensor(y_va).to(device)

    hist = {"loss": [], "val_loss": []}
    best, best_state, wait = float("inf"), None, 0

    for epoch in range(EPOCHS):
        model.train()
        total = 0.0

        for xb, yb in loader:
            xb, yb = xb.to(device), yb.to(device)
            opt.zero_grad()
            loss = loss_fn(model(xb), yb)
            loss.backward()
            nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            opt.step()
            total += loss.item() * len(xb)

        tr_loss = total / len(loader.dataset)

        model.eval()
        with torch.no_grad():
            va_loss = loss_fn(model(Xv), yv).item()

        sched.step(va_loss)
        hist["loss"].append(tr_loss)
        hist["val_loss"].append(va_loss)

        print(
            f"Epoch {epoch + 1:3d}/{EPOCHS} | "
            f"loss: {tr_loss:.6f} | "
            f"val_loss: {va_loss:.6f}"
        )

        if va_loss < best - 1e-6:
            best, best_state, wait = va_loss, copy.deepcopy(model.state_dict()), 0
        else:
            wait += 1
            if wait >= PATIENCE:
                break

    model.load_state_dict(best_state)
    n_params = sum(p.numel() for p in model.parameters())
    print(
        f"{kind:5s} | dừng sau {len(hist['loss']):3d} epoch | "
        f"val_loss tốt nhất = {best:.4f} | số tham số = {n_params:}"
    )
    return model, hist

def predict_scaled(model, X):
    model.eval()
    outs = []
    with torch.no_grad():
        for i in range(0, len(X), 256):
            xb = torch.tensor(X[i:i + 256]).to(device)
            outs.append(model(xb).cpu().numpy())
    return np.concatenate(outs, axis=0)

print(RNNRegressor("LSTM", N_FEATURES, OUT_DIM))
```

Output

```
RNNRegressor(  
  (rnn): LSTM(8, 64, num_layers=2, batch_first=True, dropout=0.2)  
  (drop): Dropout(p=0.2, inplace=False)  
  (head): Sequential(  
    (0): Linear(in_features=64, out_features=32, bias=True)  
    (1): ReLU()  
    (2): Linear(in_features=32, out_features=4, bias=True)  
  )  
)
```

Giải thích code

PyTorch không có sẵn `model.fit`, nên notebook tự viết lớp mô hình và toàn bộ vòng lặp huấn luyện.

Lớp `RNNRegressor` kế thừa `nn.Module`: `self.rnn` là `nn.RNN/LSTM/GRU` với `num_layers=2`, `batch_first=True` và `dropout=0,2` giữa hai lớp; `self.drop` là `Dropout`; `self.head` là MLP nhỏ `Linear(64→32) – ReLU – Linear(32→out_dim)`. Hàm `forward` lấy trạng thái ẩn của bước thời gian cuối `out[:, -1, :]`, qua `dropout` và `head`. Đây tương đương hai lớp RNN xếp chồng trong Keras.

`train_model`: dùng `Adam`, `ReduceLROnPlateau` (giảm một nửa `lr`, kiên nhẫn 4 epoch), `MSELoss` và `DataLoader(shuffle=True)`. Mỗi epoch gồm vòng huấn luyện (`zero_grad`, `forward`, tính `loss`, `backward`, `clip_grad_norm` với ngưỡng 1,0 để chống bùng nổ gradient, bước tối ưu) và bước kiểm định với `model.eval()` cùng `torch.no_grad()`. Early stopping thủ công: lưu `deepcopy(state_dict)` khi `val_loss` cải thiện hơn $1e-6$, dừng nếu quá `PATIENCE = 12` epoch, cuối cùng nạp lại trọng số tốt nhất.

`predict_scaled` chia dữ liệu thành batch 256, dự đoán trong chế độ eval rồi ghép lại thành mảng numpy. Dòng cuối in cấu trúc mô hình LSTM.

Giải thích output

`print(model)` cho thấy `LSTM(8, 64, num_layers=2, batch_first=True, dropout=0.2)`, `Dropout(p=0.2)` và đầu `Sequential` gồm `Linear(64→32)`, `ReLU`, `Linear(32→4)`: đúng cấu trúc đã thiết kế. Khác Keras, PyTorch không in bảng tham số ở đây (số tham số được in sau huấn luyện).

Cell 21: Huấn luyện ba mô hình (PyTorch)

Code

```
KINDS = ["RNN", "LSTM", "GRU"]
models, histories = {}, {}
for kind in KINDS:
    models[kind], histories[kind] = train_model(kind, X_train, y_train, X_val, y_val)
```

Output

```
Epoch 1/100 | loss: 0.999940 | val_loss: 1.003882
Epoch 2/100 | loss: 0.989313 | val_loss: 1.005319
Epoch 3/100 | loss: 0.983259 | val_loss: 1.004814
Epoch 4/100 | loss: 0.979407 | val_loss: 1.012220
Epoch 5/100 | loss: 0.976544 | val_loss: 1.006348
Epoch 6/100 | loss: 0.969421 | val_loss: 1.010852
Epoch 7/100 | loss: 0.962626 | val_loss: 1.009197
Epoch 8/100 | loss: 0.957133 | val_loss: 1.015332
Epoch 9/100 | loss: 0.951148 | val_loss: 1.017007
Epoch 10/100 | loss: 0.949923 | val_loss: 1.019038
Epoch 11/100 | loss: 0.948523 | val_loss: 1.018071
Epoch 12/100 | loss: 0.938436 | val_loss: 1.017966
Epoch 13/100 | loss: 0.933786 | val_loss: 1.021356
RNN | dừng sau 13 epoch | val_loss tốt nhất = 1.0039 | số tham số = 15,268
Epoch 1/100 | loss: 1.000915 | val_loss: 1.003471
Epoch 2/100 | loss: 0.993105 | val_loss: 1.003469
Epoch 3/100 | loss: 0.992167 | val_loss: 1.006994
Epoch 4/100 | loss: 0.990980 | val_loss: 1.008868
Epoch 5/100 | loss: 0.991585 | val_loss: 1.005779
Epoch 6/100 | loss: 0.992085 | val_loss: 1.008524
Epoch 7/100 | loss: 0.988054 | val_loss: 1.007586
Epoch 8/100 | loss: 0.986620 | val_loss: 1.010748
... (đã rút gọn 5 dòng ở giữa) ...
Epoch 14/100 | loss: 0.977303 | val_loss: 1.016840
LSTM | dừng sau 14 epoch | val_loss tốt nhất = 1.0035 | số tham số = 54,436
Epoch 1/100 | loss: 0.997642 | val_loss: 0.998390
Epoch 2/100 | loss: 0.992922 | val_loss: 1.000556
Epoch 3/100 | loss: 0.990635 | val_loss: 1.002066
Epoch 4/100 | loss: 0.989364 | val_loss: 1.001503
Epoch 5/100 | loss: 0.986348 | val_loss: 1.003807
Epoch 6/100 | loss: 0.983462 | val_loss: 1.006614
Epoch 7/100 | loss: 0.978154 | val_loss: 1.006155
Epoch 8/100 | loss: 0.974919 | val_loss: 1.008048
Epoch 9/100 | loss: 0.971271 | val_loss: 1.009755
Epoch 10/100 | loss: 0.967519 | val_loss: 1.008637
Epoch 11/100 | loss: 0.972440 | val_loss: 1.009083
Epoch 12/100 | loss: 0.964128 | val_loss: 1.009651
Epoch 13/100 | loss: 0.960273 | val_loss: 1.012376
GRU | dừng sau 13 epoch | val_loss tốt nhất = 0.9984 | số tham số = 41,380
```

Giải thích code

Code giống hệt bản Keras: vòng lặp gọi `train_model` cho RNN, LSTM, GRU.

Giải thích output

Log dạng 'Epoch k/100 | loss | val_loss'. RNN dừng sau 13 epoch (val_loss tốt nhất 1,0039, 15.268 tham số); LSTM dừng sau 14 epoch (1,0035, 54.436 tham số); GRU dừng sau 13 epoch (0,9984, 41.380 tham số). Ở cả ba mô hình, val_loss

$\approx 1,00$ ngay từ epoch 1 và không cải thiện thêm, trong khi train loss giảm dần (RNN từ 1,0000 xuống 0,9338): kiểu quá khớp giống bản Keras.

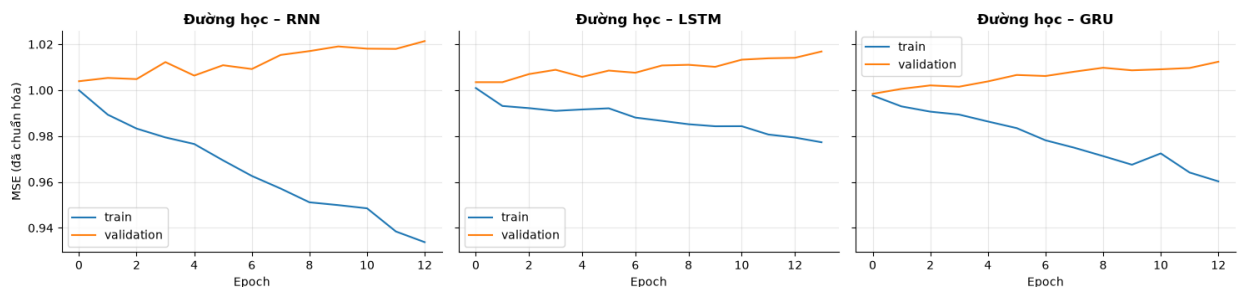
Số tham số lớn hơn Keras một chút (LSTM 54.436 so với 53.924, RNN 15.268 so với 15.140) vì PyTorch dùng hai vector bias (b_{ih} và b_{hh}) cho mỗi lớp, còn Keras dùng một. GRU của Keras (`reset_after=True`) cũng có hai bias nên hai bản đều là 41.380.

Cell 22: Đường học (PyTorch)

Code

```
fig, axes = plt.subplots(1, 3, figsize=(15, 3.6), sharey=True)
for ax, kind in zip(axes, KINDS):
    ax.plot(histories[kind]["loss"], label="train"); ax.plot(histories[kind]["val_loss"], label="validation")
    ax.set_title(f"Đường học - {kind}"); ax.set_xlabel("Epoch"); ax.legend()
axes[0].set_ylabel("MSE (đã chuẩn hóa)")
plt.tight_layout(); plt.show()
```

Output



Giải thích code

Code giống hệt bản Keras.

Giải thích output

Cả ba panel có dạng tương tự bản Keras: train loss giảm chậm (RNN từ 1,00 xuống khoảng 0,935) còn validation loss nằm ngang hoặc tăng nhẹ (1,00 lên khoảng 1,02). Điểm khác là các đường của PyTorch nằm sát mức 1,0 ngay từ epoch đầu, còn ở bản Keras loss ban đầu cao hơn (khoảng 1,1 đến 1,2); hai framework khởi tạo trọng số khác nhau.

h) Đánh giá và trực quan hóa dự đoán

*Bảng chỉ số, biểu đồ dự đoán so với thực tế và phân tích phần dư. Mô hình "tốt nhất" được chọn theo **tập kiểm định** để tập kiểm thử vẫn là phép đo độc lập.*

Cell 24: Đánh giá val/test (PyTorch)

Code

```
for kind in KINDS:
    for k, X in [("val", X_val), ("test", X_test)]:
        preds[k][kind] = inv_rev(predict_scaled(models[kind], X)[: , 0])

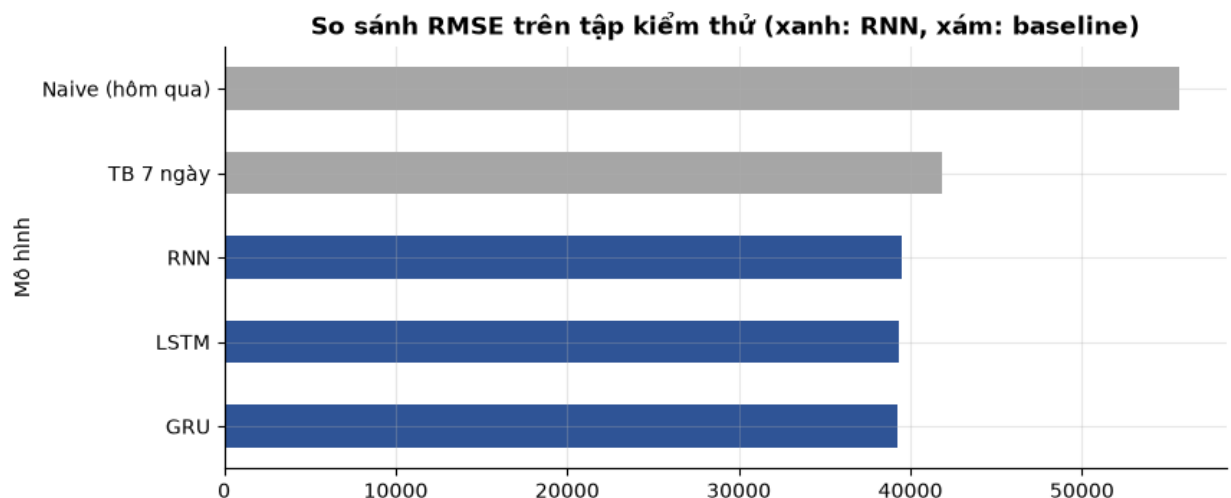
rows = []
for name in preds["test"]:
    rows.append({
        "Mô hình": name,
        "RMSE val": rmse(rev_true["val"], preds["val"][name]),
        "RMSE test": rmse(rev_true["test"], preds["test"][name]),
        "MAE test": mae(rev_true["test"], preds["test"][name]),
        "MAPE test (%)": mape(rev_true["test"], preds["test"][name]),
        "R2 test": r2(rev_true["test"], preds["test"][name]),
    })
results = pd.DataFrame(rows).set_index("Mô hình")
display(results.sort_values("RMSE test"))

best_kind = min(KINDS, key=lambda k: results.loc[k, "RMSE val"]) # chọn mô hình theo tập KIỂM ĐỊNH
print(f"Mô hình tốt nhất trên tập kiểm định: {best_kind}")

fig, ax = plt.subplots(figsize=(9, 3.8))
results["RMSE test"].sort_values().plot.barh(ax=ax, color=["#2F5496" if i in KINDS else "#A6A6A6" for i in
    results["RMSE test"].sort_values().index])
ax.set_title("So sánh RMSE trên tập kiểm thử (xanh: RNN, xám: baseline)"); plt.show()
```

Output

	RMSE val	RMSE test	MAE test	MAPE test (%)	R ² test
Mô hình					
GRU	42,633.9712	39,229.7529	31,187.9552	6.3956	-0.0164
LSTM	42,834.3770	39,358.3362	31,200.9343	6.4006	-0.0230
RNN	42,669.2375	39,497.2189	31,225.3494	6.4094	-0.0303
TB 7 ngày	45,189.3939	41,844.4026	32,771.7424	6.6565	-0.1563
Naive (hôm qua)	55,250.8168	55,658.1393	44,753.7635	9.0211	-1.0458
Mô hình tốt nhất trên tập kiểm định: GRU					



Giải thích code

Code giống hệt bản Keras (dự đoán, bảng chỉ số, chọn mô hình theo val, biểu đồ RMSE).

Giải thích output

Bảng (sắp theo RMSE test): GRU 39.230 (R^2 -0,0164), LSTM 39.358 (R^2 -0,0230), RNN 39.497 (R^2 -0,0303), TB 7 ngày 41.844, Naive 55.658. Hai baseline trùng số với bản Keras vì không phụ thuộc mô hình. Mô hình tốt nhất trên tập kiểm định là GRU (RMSE val 42.634), khác với bản Keras (LSTM).

Ba mạng PyTorch sát nhau hơn nhiều (39,2 đến 39,5 nghìn) so với Keras (39,3 đến 40,9 nghìn). Việc mô hình 'tốt nhất' đổi từ LSTM sang GRU giữa hai framework cho thấy chênh lệch giữa các kiến trúc nằm trong phạm vi nhiễu khởi tạo.

Cell 25: Đồ thị dự đoán và phân tích sai số (PyTorch)

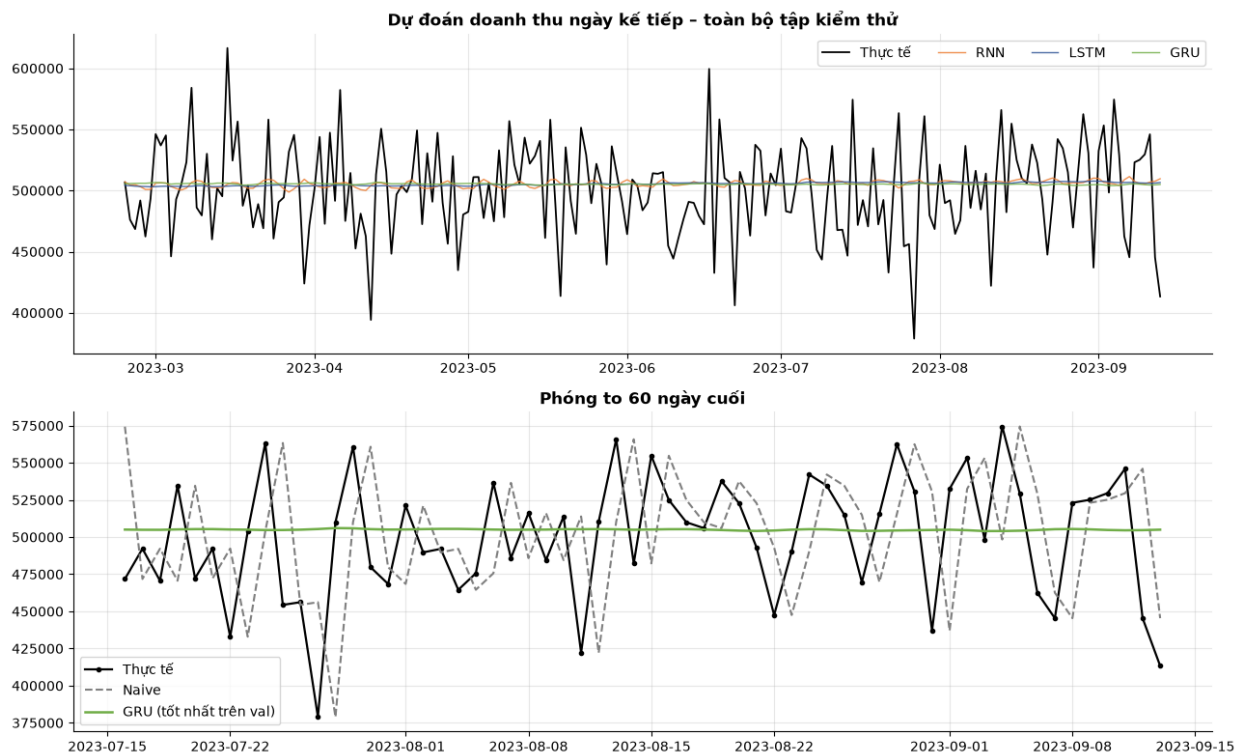
Code

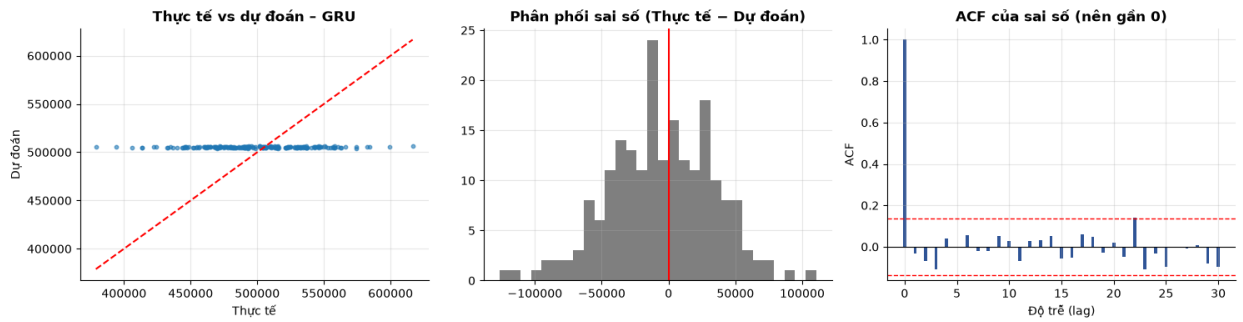
```
d_test = dates[split_pos["test"]]
colors = {"RNN": "#ED7D31", "LSTM": "#2F5496", "GRU": "#70AD47"}
fig, axes = plt.subplots(2, 1, figsize=(13, 8))
axes[0].plot(d_test, rev_true["test"], c="k", lw=1.3, label="Thực tế")
for kind in KINDS: axes[0].plot(d_test, preds["test"][kind], c=colors[kind], lw=1, alpha=0.85, label=kind)
axes[0].set_title("Dự đoán doanh thu ngày kế tiếp - toàn bộ tập kiểm thử"); axes[0].legend(ncol=4)

zoom = slice(-60, None)
axes[1].plot(d_test[zoom], rev_true["test"][zoom], c="k", lw=1.6, marker="o", ms=3, label="Thực tế")
axes[1].plot(d_test[zoom], preds["test"]["Naive (hôm qua)"][zoom], c="gray", ls="--", label="Naive")
axes[1].plot(d_test[zoom], preds["test"][best_kind][zoom], c=colors[best_kind], lw=1.8, label=f"{best_kind} (tốt nhất trên val)")
axes[1].set_title("Phóng to 60 ngày cuối"); axes[1].legend(); plt.tight_layout(); plt.show()

# --- Phân tích sai số của mô hình tốt nhất ---
res = rev_true["test"] - preds["test"][best_kind]
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
axes[0].scatter(rev_true["test"], preds["test"][best_kind], s=10, alpha=0.6)
lim = [min(rev_true["test"].min(), preds["test"][best_kind].min()), max(rev_true["test"].max(), preds["test"][best_kind].max())]
axes[0].plot(lim, lim, "r--"); axes[0].set_xlabel("Thực tế"); axes[0].set_ylabel("Dự đoán");
axes[0].set_title(f"Thực tế vs dự đoán - {best_kind}")
axes[1].hist(res, bins=30, color="#7F777F"); axes[1].axvline(0, c="r"); axes[1].set_title("Phân phối sai số (Thực tế - Dự đoán)")
plot_acf(axes[2], res, 30, "ACF của sai số (nên gần 0)")
plt.tight_layout(); plt.show()
print(f"Sai số trung bình (bias) = {res.mean():.1f} | độ lệch chuẩn sai số = {res.std():.1f}")
```

Output





Sai số trung bình (bias) = -5,195.2 | độ lệch chuẩn sai số = 38,884.2

Giải thích code

Code giống hệt bản Keras, áp dụng cho mô hình tốt nhất là GRU.

Giải thích output

Ba hình có bố cục như bản Keras. Bias = -5.195,2 và độ lệch chuẩn sai số 38.884,2 (Keras: -4.839,8 và 39.018,9). Sai số của GRU nhỏ hơn không đáng kể, vẫn xấp xỉ độ lệch chuẩn của chính doanh thu, tức dự đoán vẫn gần như hằng số.

i) Phân tích đặc trưng nào quan trọng

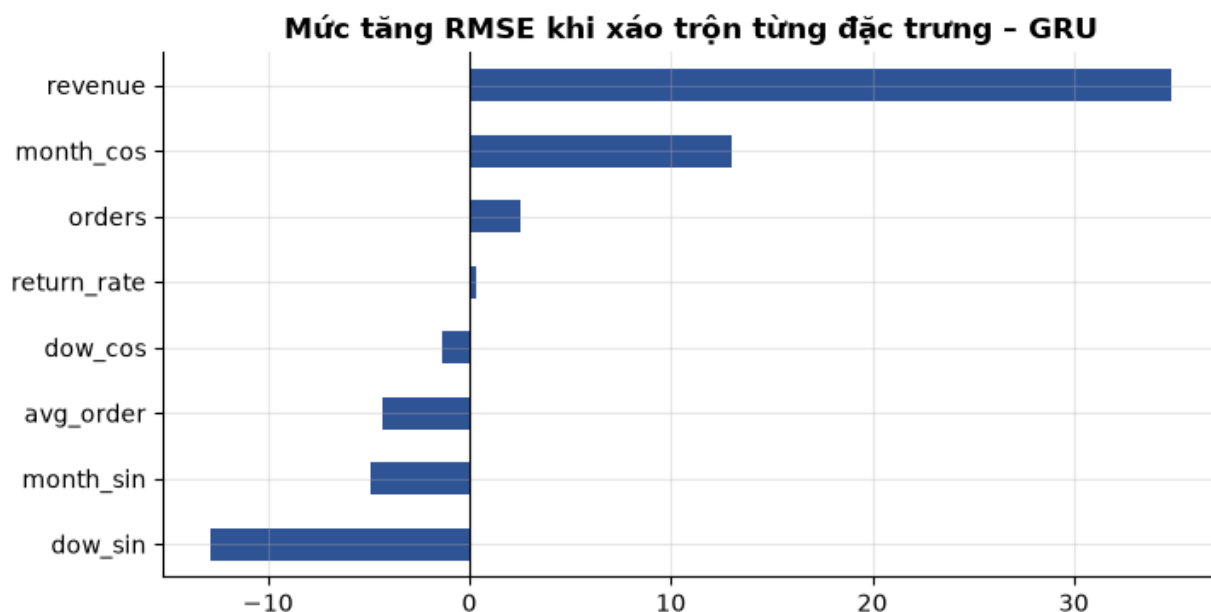
Xáo trộn từng đặc trưng trong tập kiểm thử; đặc trưng nào làm RMSE tăng nhiều nhất là đặc trưng mô hình dựa vào nhiều nhất.

Cell 27: Độ quan trọng đặc trưng (PyTorch)

Code

```
# ---- Độ quan trọng đặc trưng bằng hoán vị (permutation importance) trên tập kiểm thử ----
base_rmse = rmse(rev_true["test"], preds["test"][best_kind])
rng = np.random.default_rng(SEED)
imp = {}
for j, name in enumerate(FEATURES):
    deltas = []
    for _ in range(5):
        Xp = X_test.copy()
        Xp[:, :, j] = Xp[rng.permutation(len(Xp))][:, :, j] # xáo trộn đặc trưng j giữa các mẫu
        deltas.append(rmse(rev_true["test"], inv_rev(predict_scaled(models[best_kind], Xp[:, 0]))) - base_rmse)
    imp[name] = np.mean(deltas)
imp = pd.Series(imp).sort_values()
fig, ax = plt.subplots(figsize=(8, 4))
imp.plot.barh(ax=ax, color="#2F5496"); ax.axvline(0, c="k", lw=0.8)
ax.set_title(f"Mức tăng RMSE khi xáo trộn từng đặc trưng - {best_kind}"); plt.show()
print("Đặc trưng ảnh hưởng nhất:", imp.idxmax(), "| mức tăng RMSE ≈ %.1f" % imp.max())
```

Output



Đặc trưng ảnh hưởng nhất: revenue | mức tăng RMSE ≈ 34.8

Giải thích code

Code giống hệt bản Keras, chạy trên mô hình GRU.

Giải thích output

Đặc trưng ảnh hưởng nhất là revenue với mức tăng RMSE $\approx 34,8$ (Keras là $\text{return_rate} \approx 41,8$). Cả hai mức tăng đều chỉ khoảng 0,1% RMSE. Đặc trưng 'quan trọng nhất' đổi khác giữa hai lần huấn luyện là bằng chứng nữa cho thấy đó là nhiễu, không phải tín hiệu. Bản PyTorch không in tiến trình predict nên output gọn hơn.

j) Dự báo tương lai

Dự báo đệ quy HORIZON ngày sau ngày cuối cùng của dữ liệu bằng mô hình tốt nhất.

Cell 29: Dự báo đệ quy 14 ngày (PyTorch)

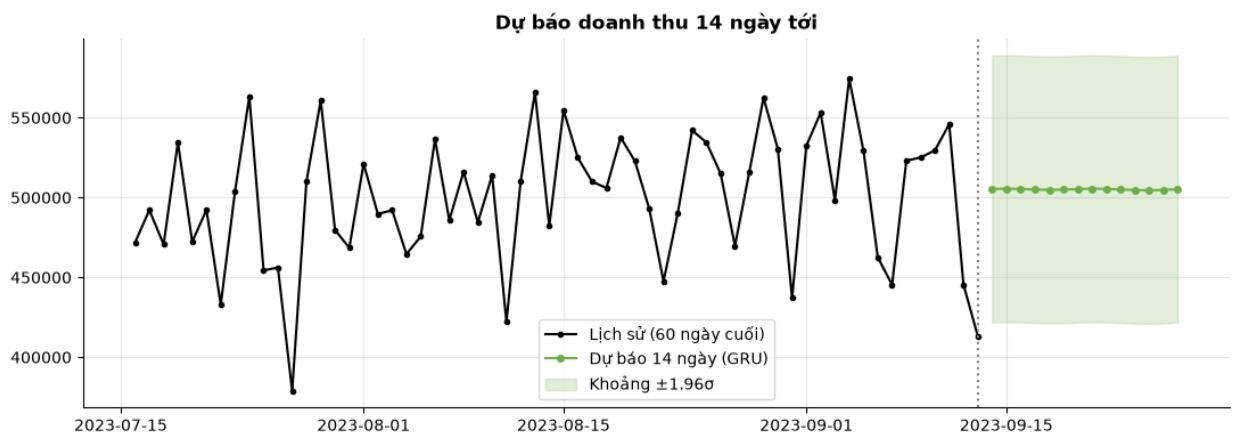
Code

```
def forecast_future(model, horizon):
    """Dự báo đệ quy: đưa dự đoán của ngày t làm đầu vào cho ngày t+1. Đặc trưng lịch được biết trước."""
    window = arr_s[-WINDOW:].copy()
    rows = []
    for h in range(1, horizon + 1):
        d = dates[-1] + pd.Timedelta(days=h)
        p = predict_scaled(model, window[None, ...])[0] # (OUT_DIM,) đã chuẩn hóa
        cal = calendar_feats(pd.DatetimeIndex([d])).values[0]
        cal_s = (cal - scaler.mean_[OUT_DIM:]) / scaler.scale_[OUT_DIM:]
        window = np.vstack([window[1:], np.concatenate([p, cal_s]).astype("float32")])
        rows.append((d, float(inv_rev(p[0]))))
    return pd.DataFrame(rows, columns=["Ngày", "Doanh thu dự báo"]).set_index("Ngày")

fc = forecast_future(models[best_kind], HORIZON)
sigma = (rev_true["val"] - preds["val"][best_kind]).std() # độ lệch chuẩn sai số trên tập val
fc["Cận dưới (95%)"] = fc["Doanh thu dự báo"] - 1.96 * sigma
fc["Cận trên (95%)"] = fc["Doanh thu dự báo"] + 1.96 * sigma

hist_tail = daily["revenue"].iloc[-60:]
fig, ax = plt.subplots(figsize=(13, 4.2))
ax.plot(hist_tail.index, hist_tail.values, c="k", marker="o", ms=3, label="Lịch sử (60 ngày cuối)")
ax.plot(fc.index, fc["Doanh thu dự báo"], c=colors[best_kind], marker="o", ms=4, label=f"Dự báo {HORIZON} ngày ({best_kind})")
ax.fill_between(fc.index, fc["Cận dưới (95%)"], fc["Cận trên (95%)"], color=colors[best_kind], alpha=0.2, label="Khoảng ±1.96σ")
ax.axvline(dates[-1], c="gray", ls=":"); ax.set_title(f"Dự báo doanh thu {HORIZON} ngày tới"); ax.legend();
plt.show()
display(fc.round(0))
print("Lưu ý: dự báo đệ quy nhiều bước tích lũy sai số; khoảng tin cậy chỉ mang tính tham khảo.")
```

Output



	Doanh thu dự báo	Cận dưới (95%)	Cận trên (95%)
Ngày			
2023-09-14	505,363.0000	421,813.0000	588,913.0000
2023-09-15	505,533.0000	421,983.0000	589,083.0000
2023-09-16	505,380.0000	421,830.0000	588,930.0000
2023-09-17	505,030.0000	421,479.0000	588,580.0000
2023-09-18	504,820.0000	421,270.0000	588,371.0000
2023-09-19	504,924.0000	421,373.0000	588,474.0000
2023-09-20	505,349.0000	421,799.0000	588,899.0000
2023-09-21	505,571.0000	422,021.0000	589,121.0000
2023-09-22	505,411.0000	421,861.0000	588,962.0000
2023-09-23	505,054.0000	421,503.0000	588,604.0000
2023-09-24	504,658.0000	421,108.0000	588,208.0000
2023-09-25	504,521.0000	420,971.0000	588,071.0000
2023-09-26	504,755.0000	421,204.0000	588,305.0000
2023-09-27	505,235.0000	421,685.0000	588,785.0000

Lưu ý: dự báo đệ quy nhiều bước tích lũy sai số; khoảng tin cậy chỉ mang tính tham khảo.

Giải thích code

Code giống hệt bản Keras, dùng GRU.

Giải thích output

Dự báo phẳng quanh 505 nghìn (từ 504.521 đến 505.571), khoảng 95% từ khoảng 421,8 nghìn đến 588,9 nghìn ($\pm 83,5$ nghìn). Bản Keras dự báo quanh 503 nghìn. Chênh lệch chỉ $\sim 0,5\%$ và nhỏ hơn nhiều so với độ rộng khoảng tin cậy.

k) Kết luận

Cell 31: Kết luận tự động (PyTorch)

Code

```
# ---- Kết luận tự động dựa trên kết quả thực tế ----
best_rmse = results.loc[best_kind, "RMSE test"]
naive_rmse = results.loc["Naive (hôm qua)", "RMSE test"]
ma_rmse = results.loc["TB 7 ngày", "RMSE test"]
gain = (1 - best_rmse / min(naive_rmse, ma_rmse)) * 100
print(f"RMSE test - {best_kind}: {best_rmse:,.0f} | Naive: {naive_rmse:,.0f} | TB 7 ngày: {ma_rmse:,.0f}")
if gain > 3:
    print(f"=> {best_kind} tốt hơn baseline tốt nhất khoảng {gain:.1f}%: chuỗi có cấu trúc mà RNN khai thác được.")
elif gain > -3:
    print(f"=> {best_kind} chỉ tương đương baseline (chênh {gain:+.1f}%): RNN không mang lại lợi ích rõ rệt.")
else:
    print(f"=> {best_kind} kém baseline {abs(gain):.1f}%: nên ưu tiên mô hình đơn giản (trung bình trượt) cho chuỗi này.")
print("R² test của mô hình tốt nhất:", round(results.loc[best_kind, "R² test"], 3),
      "(gần 0 hoặc âm nghĩa là chưa giải thích được biến thiên ngoài mức trung bình)")
```

Output

```
RMSE test - GRU: 39,230 | Naive: 55,658 | TB 7 ngày: 41,844
=> GRU tốt hơn baseline tốt nhất khoảng 6.2%: chuỗi có cấu trúc mà RNN khai thác được.
R² test của mô hình tốt nhất: -0.016 (gần 0 hoặc âm nghĩa là chưa giải thích được biến thiên ngoài mức trung bình)
```

Giải thích code

Code giống hệt bản Keras.

Giải thích output

GRU có RMSE 39.230 so với Naive 55.658 và TB 7 ngày 41.844, tốt hơn baseline 6,2%; R^2 test = $-0,016$. Diễn giải giống bản Keras: cải thiện đến từ dự đoán quanh trung bình chứ không phải học được quy luật; kết luận đúng là chuỗi doanh thu tổng hợp này không dự báo được ngoài mức trung bình.

2. BÀI TOÁN 2 – DỰ BÁO GIÁ CỔ PHIẾU IBM (DJIA)

2.1. Phiên bản Keras (TensorFlow)

Dữ liệu: *DJIA 30 Stock Time Series* (Kaggle) – tệp `all_stocks_2006-01-01_to_2018-01-01.csv` gồm OHLCV hằng ngày của 30 cổ phiếu, 2006–2017.

Bài toán: với một mã cổ phiếu, dùng WINDOW phiên gần nhất để dự đoán **lợi suất log của phiên kế tiếp**, rồi quy đổi thành **giá đóng cửa dự đoán**. So sánh RNN, LSTM, GRU với hai baseline (giá không đổi, drift).

Vì sao dự báo lợi suất mà không phải giá thô? Giá là chuỗi không dừng: tập kiểm thử (2016–2017) thường có giá cao hơn hẳn phạm vi của tập huấn luyện nên mô hình huấn luyện trên giá thô sẽ ngoại suy kém. Lợi suất là chuỗi gần dừng, và là đại lượng phản ánh đúng khả năng dự báo.

Quy trình: 1) Nạp và kiểm tra → 2) EDA → 3) Đặc trưng và cửa sổ trượt → 4) Baseline → 5) Huấn luyện 3 mô hình → 6) Đánh giá, trực quan hóa → 7) Phân tích đặc trưng → 8) Dự báo tương lai → 9) Kết luận.

a) Thư viện và cấu hình

Cell 2: Nhập thư viện và thiết lập hiển thị

Code

```
import os, glob, math, random, warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display
from sklearn.preprocessing import StandardScaler

warnings.filterwarnings("ignore")
plt.rcParams.update({
    "figure.dpi": 100, "figure.figsize": (11, 4),
    "axes.grid": True, "grid.alpha": 0.3,
    "axes.spines.top": False, "axes.spines.right": False,
    "axes.titlesize": 12, "axes.titleweight": "bold",
})
pd.set_option("display.float_format", lambda v: f"{v:., .4f}")
pd.set_option("display.max_columns", 30)
```

Cell này không tạo output hiển thị.

Giải thích code

Giống cell 2 của Bài 1: nhập `os`, `glob`, `math`, `random`, `warnings`, `numpy`, `pandas`, `matplotlib`, `display` và `StandardScaler`; đặt style biểu đồ (lưới mờ, bỏ viền trên/phải, tiêu đề đậm) và định dạng số của `pandas` (dấu phân cách nghìn, 4 chữ số thập phân).

Cell 3: Khai báo tham số cấu hình

Code

```
# ===== CẤU HÌNH (có thể chỉnh) =====
DATA_PATH = "../data/all_stocks_2006-01-01_to_2018-01-01.csv" # <-- đường dẫn tới tệp CSV đã tải về
TICKER = "IBM" # mã cổ phiếu cần phân tích (AAPL, MSFT, GE, JPM, ...)
WINDOW = 60 # số phiên giao dịch lịch sử đưa vào RNN
HORIZON = 5 # số phiên dự báo tương lai
TRAIN_RATIO = 0.70
VAL_RATIO = 0.15

UNITS = 64
DROPOUT = 0.2
LR = 1e-3
BATCH_SIZE = 32
EPOCHS = int(os.environ.get("NB_EPOCHS", 100))
PATIENCE = 12
SEED = 42
```

Cell này không tạo output hiển thị.

Giải thích code

Gom tham số của bài toán cổ phiếu.

DATA_PATH trỏ tới tệp all_stocks_2006-01-01_to_2018-01-01.csv; TICKER = "IBM" là mã cổ phiếu được phân tích; WINDOW = 60 phiên lịch sử là đầu vào mỗi mẫu (lớn hơn Bài 1 vì cần nhìn xa hơn để tính các đặc trưng trung bình động); HORIZON = 5 phiên dự báo tương lai. Tỷ lệ chia 70/15/15, UNITS = 64, DROPOUT = 0,2, LR = 1e-3, BATCH_SIZE = 32, EPOCHS tối đa 100, PATIENCE = 12, SEED = 42 giống Bài 1.

Cell 4: Nạp TensorFlow/Keras và kiểm tra thiết bị

Code

```
os.environ.setdefault("TF_CPP_MIN_LOG_LEVEL", "2")
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

random.seed(SEED); np.random.seed(SEED); tf.random.set_seed(SEED)
print("TensorFlow:", tf.__version__, "| Keras:", keras.__version__)
print("Thiết bị:", [d.name for d in tf.config.list_physical_devices()]])
```

Output

```
TensorFlow: 2.21.0 | Keras: 3.15.1
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUD
A/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
Thiết bị: ['/physical_device:CPU:0']
```

Giải thích code

Giống cell 4 của Bài 1: nhập tensorflow, keras, layers, cố định seed và in phiên bản và thiết bị.

Giải thích output

TensorFlow 2.21.0, Keras 3.15.1; cảnh báo không hỗ trợ GPU trên Windows native nên chỉ có CPU:0 (giống Bài 1).

Cell 5: Hàm tiện ích: tìm tệp, ACF, vẽ ACF

Code

```
def find_data_file(path, pattern):
    """Trả về đường dẫn tệp dữ liệu. Nếu DATA_PATH không tồn tại thì tự tìm theo mẫu tên trong thư mục hiện tại."""
    if os.path.exists(path):
        return path
    found = glob.glob(pattern) + glob.glob(os.path.join("...", pattern), recursive=True)
    found = sorted(set(found), key=lambda p: ("large" not in p.lower(), len(p), p))
    if not found:
        raise FileNotFoundError(
            f"Không tìm thấy '{path}'. Hãy sửa biến DATA_PATH ở ô cấu hình thành đường dẫn tới tệp CSV đã tải về.")
    print(f"Không thấy '{path}', tự động dùng: {found[0]}")
    return found[0]

def acf(x, nlags):
    """Hàm tự tương quan (ACF) tính trực tiếp bằng numpy."""
    x = np.asarray(x, dtype=float)
    x = x - x.mean()
    d = np.dot(x, x)
    return np.array([np.dot(x[:len(x) - k], x[k:]) / d for k in range(nlags + 1)])

def plot_acf(ax, x, nlags, title):
    a = acf(x, nlags)
    conf = 1.96 / np.sqrt(len(x))
    ax.bar(range(nlags + 1), a, width=0.3, color="#2F5496")
    ax.axhline(conf, ls="--", c="r", lw=1); ax.axhline(-conf, ls="--", c="r", lw=1)
    ax.axhline(0, c="k", lw=0.8)
    ax.set_title(title); ax.set_xlabel("Độ trễ (lag)"); ax.set_ylabel("ACF")
    return a, conf
```

Cell này không tạo output hiển thị.

Giải thích code

Giống cell 5 của Bài 1: `find_data_file` tìm tệp CSV theo mẫu tên khi đường dẫn sai; `acf` tính tự tương quan bằng numpy; `plot_acf` vẽ ACF kèm ngưỡng tin cậy $\pm 1,96/\sqrt{n}$ và trả về (mảng ACF, ngưỡng).

b) Nạp dữ liệu và kiểm tra chất lượng

Cell 7: Nạp dữ liệu và kiểm tra chất lượng

Code

```
DATA_PATH = find_data_file(DATA_PATH, "all_stocks*.csv")
raw = pd.read_csv(DATA_PATH)
raw.columns = raw.columns.str.strip()
need = ["Date", "Open", "High", "Low", "Close", "Volume", "Name"]
assert all(c in raw.columns for c in need), f"Thiếu cột. Cần {need}, hiện có {list(raw.columns)}"
raw["Date"] = pd.to_datetime(raw["Date"], errors="coerce")
raw = raw.dropna(subset=["Date"]).sort_values(["Name", "Date"]).reset_index(drop=True)

print(f"Tổng số dòng: {len(raw):,} | Số mã cổ phiếu: {raw['Name'].nunique():,}")
print(f"Khoảng thời gian: {raw['Date'].min().date():,} -> {raw['Date'].max().date():,}")
display(raw.head())
na = raw[need].isna().sum(); print("Giá trị thiếu theo cột:"); display(na.to_frame("Số ô thiếu").T)
print("Đồng trùng (Name, Date):", raw.duplicated(["Name", "Date"]).sum())
```

Output

```
Tổng số dòng: 93,612 | Số mã cổ phiếu: 31
Khoảng thời gian: 2006-01-03 -> 2017-12-29
```

	Date	Open	High	Low	Close	Volume	Name
0	2006-01-03	39.6900	41.2200	38.7900	40.9100	24232729	AABA
1	2006-01-04	41.2200	41.9000	40.7700	40.9700	20553479	AABA
2	2006-01-05	40.9300	41.7300	40.8500	41.5300	12829610	AABA
3	2006-01-06	42.8800	43.5700	42.8000	43.2100	29422828	AABA
4	2006-01-09	43.1000	43.6600	42.8200	43.4200	16268338	AABA

Giá trị thiếu theo cột:

	Date	Open	High	Low	Close	Volume	Name
Số ô thiếu	0	25	10	20	0	0	0
Dòng trùng (Name, Date):	0						

Giải thích code

Đọc dữ liệu OHLCV của tất cả mã, kiểm tra cột, chuẩn hóa ngày và thống kê thiếu/trùng.

Kiểm tra bảy cột bắt buộc (Date, Open, High, Low, Close, Volume, Name), chuyển Date sang datetime, bỏ dòng ngày lỗi, sắp xếp theo (Name, Date). Sau đó in số dòng, số mã, khoảng thời gian, xem 5 dòng đầu, đếm ô thiếu theo cột và đếm dòng trùng theo cặp (Name, Date).

Giải thích output

Tổng cộng 93.612 dòng, 31 mã cổ phiếu (30 mã DJIA cộng thêm AABA), từ 03/01/2006 đến 29/12/2017. Năm dòng đầu thuộc AABA (Yahoo/Altaba) với giá đóng cửa khoảng 40 đô la.

Giá trị thiếu: Open 25, High 10, Low 20, Close/Volume/Name/Date bằng 0. Số thiếu rất ít so với 93.612 dòng nên sẽ xử lý bằng điền tiếp giá trị trước (ffill) ở cell sau. Không có dòng trùng (Name, Date).

c) Phân tích khám phá (EDA)

2.1 Tổng quan 30 mã và chọn mã phân tích

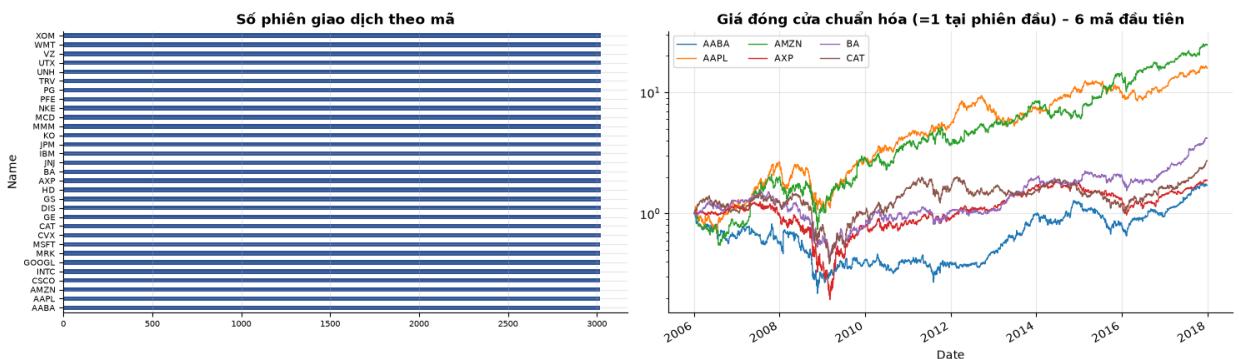
Cell 9: Tổng quan 30 mã và chọn mã phân tích IBM

Code

```
cnt = raw.groupby("Name").size().sort_values()
fig, axes = plt.subplots(1, 2, figsize=(15, 4.5))
cnt.plot.barh(ax=axes[0], color="#2F5496", fontsize=7); axes[0].set_title("Số phiên giao dịch theo mã")
tick_show = list(raw["Name"].unique()[:6])
for t in tick_show:
    s = raw[raw["Name"] == t].set_index("Date")["Close"].dropna()
    (s / s.iloc[0]).plot(ax=axes[1], lw=1, label=t)
axes[1].set_title("Giá đóng cửa chuẩn hóa (=1 tại phiên đầu) - 6 mã đầu tiên"); axes[1].legend(ncol=3,
    fontsize=8)
axes[1].set_yscale("log"); plt.tight_layout(); plt.show()
print("Các mã có ít phiên nhất (niêm yết muộn hoặc thiếu dữ liệu):", cnt.head(3).to_dict())

if TICKER not in raw["Name"].unique():
    print(f"Không có mã '{TICKER}', chuyển sang mã đầu tiên: {raw['Name'].iloc[0]}")
    TICKER = raw["Name"].iloc[0]
df_t = raw[raw["Name"] == TICKER].set_index("Date")[["Open", "High", "Low", "Close", "Volume"]].sort_index()
df_t = df_t[~df_t.index.duplicated()].ffill()
print(f"\n>>> Phân tích chi tiết mã {TICKER}: {len(df_t)} phiên, {df_t.index.min().date()} ->
    {df_t.index.max().date()}")
display(df_t.describe().T)
```

Output



Các mã có ít phiên nhất (niêm yết muộn hoặc thiếu dữ liệu): {'AABA': 3019, 'AAPL': 3019, 'AMZN': 3019}

>>> Phân tích chi tiết mã IBM: 3020 phiên, 2006-01-03 -> 2017-12-29

	count	mean	std	min	25%	50%	75%	max
Open	3,020.0000	145.5153	37.5487	72.7400	116.4075	149.6050	178.4375	215.3800
High	3,020.0000	146.6817	37.6134	73.9400	117.7650	150.3300	179.7625	215.9000
Low	3,020.0000	144.4714	37.4714	69.5000	115.5000	148.4250	177.3200	214.3000
Close	3,020.0000	145.6173	37.5294	71.7400	116.5250	149.3150	178.6850	215.8000
Volume	3,020.0000	5,773,300.6225	3,192,830.5401	254,256.0000	3,622,681.2500	4,928,851.5000	6,965,014.0000	30,774,276.0000

Giải thích code

Vẽ số phiên theo mã và giá chuẩn hóa của 6 mã đầu, sau đó lọc riêng mã IBM.

cnt là số phiên mỗi mã. Panel phải chia giá đóng cửa của 6 mã đầu cho giá phiên đầu (=1) và dùng trục log để so sánh tăng trưởng tương đối. Nếu TICKER không có trong dữ liệu thì chuyển sang mã đầu tiên. df_t giữ cột Open/High/Low/Close/Volume của mã được chọn, loại chỉ mục trùng và ffill để lấp ô thiếu. Cuối cùng in mô tả thống kê.

Giải thích output

Biểu đồ trái: hầu hết mã có gần cùng số phiên (khoảng 3.019 đến 3.020). Biểu đồ phải: sau chuẩn hóa, AMZN và AAPL tăng gấp hơn 10 lần, còn các mã khác dao động thấp hơn nhiều; cú sứt 2008 đến 2009 thấy rõ ở mọi mã.

Các mã ít phiên nhất (AABA, AAPL, AMZN) vẫn có 3.019 phiên, vậy không có mã nào bị thiếu dữ liệu lớn. IBM có 3.020 phiên (03/01/2006 đến 29/12/2017). Thống kê của IBM: giá đóng cửa trung bình 145,6 (nhỏ nhất 71,74, lớn nhất 215,80), độ lệch chuẩn 37,5; khối lượng trung bình khoảng 5,77 triệu cổ phiếu/phiên, đỉnh 30,8 triệu.

2.2 Đặc điểm chuỗi giá của mã được chọn

Giá, đường trung bình động, khối lượng, phân phối lợi suất, biến động, tự tương quan, tương quan giữa các cột, lợi suất theo năm và drawdown.

Cell 11: EDA chi tiết chuỗi giá IBM (3 hình)

Code

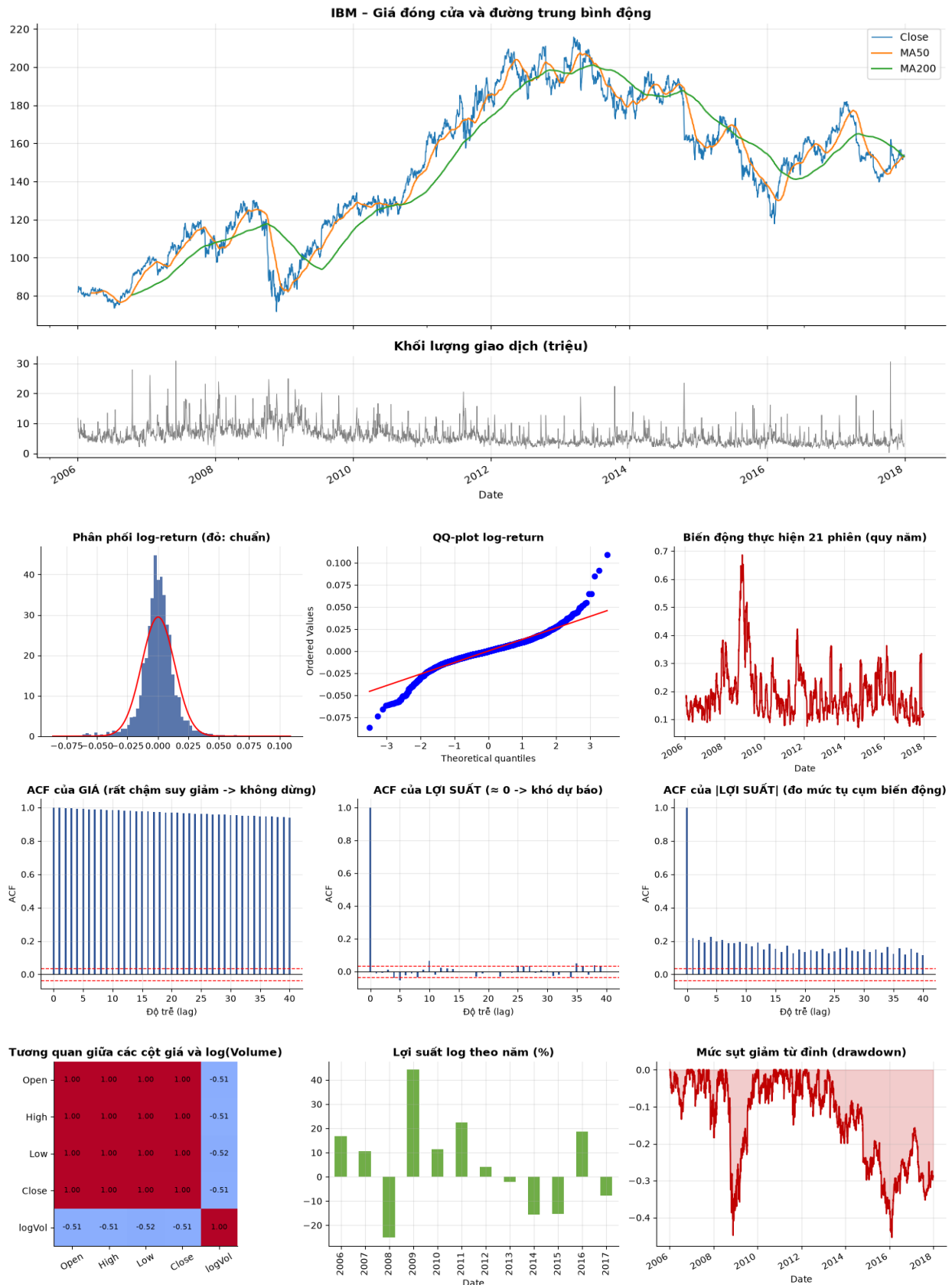
```
from scipy import stats
ret = np.log(df_t["Close"]).diff().dropna()

fig, axes = plt.subplots(2, 1, figsize=(13, 7), sharex=True, gridspec_kw={"height_ratios": [3, 1]})
df_t["Close"].plot(ax=axes[0], lw=1, label="Close")
df_t["Close"].rolling(50).mean().plot(ax=axes[0], lw=1.4, label="MA50")
df_t["Close"].rolling(200).mean().plot(ax=axes[0], lw=1.4, label="MA200")
axes[0].set_title(f"{TICKER} - Giá đóng cửa và đường trung bình động"); axes[0].legend()
(df_t["Volume"] / 1e6).plot(ax=axes[1], lw=0.6, c="#7F7F7F"); axes[1].set_title("Khối lượng giao dịch (triệu)")
plt.tight_layout(); plt.show()

fig, axes = plt.subplots(2, 3, figsize=(15, 8))
# 1) phân phối lợi suất so với phân phối chuẩn
axes[0, 0].hist(ret, bins=80, density=True, color="#2F5496", alpha=0.8)
xs = np.linspace(ret.min(), ret.max(), 300); axes[0, 0].plot(xs, stats.norm.pdf(xs, ret.mean(), ret.std()), "r")
axes[0, 0].set_title("Phân phối log-return (đó: chuẩn)")
# 2) QQ-plot
stats.probplot(ret, dist="norm", plot=axes[0, 1]); axes[0, 1].set_title("QQ-plot log-return")
# 3) biến động
(ret.rolling(21).std() * np.sqrt(252)).plot(ax=axes[0, 2], c="#C00000"); axes[0, 2].set_title("Biến động thực  
hiện 21 phiên (quy năm)")
# 4) ACF giá, lợi suất, |lợi suất|
plot_acf(axes[1, 0], df_t["Close"], 40, "ACF của GIÁ (rất chậm suy giảm -> không dừng)")
plot_acf(axes[1, 1], ret, 40, "ACF của LỢI SUẤT (≈ 0 -> khó dự báo)")
plot_acf(axes[1, 2], ret.abs(), 40, "ACF của |LỢI SUẤT| (đo mức tự cụm biến động)")
plt.tight_layout(); plt.show()

fig, axes = plt.subplots(1, 3, figsize=(15, 4))
cm = df_t.assign(logVol=np.log1p(df_t["Volume"])).drop(columns="Volume").corr()
im = axes[0].imshow(cm, cmap="coolwarm", vmin=-1, vmax=1); axes[0].grid(False)
axes[0].set_xticks(range(len(cm))); axes[0].set_xticklabels(cm.columns, rotation=30);
axes[0].set_yticks(range(len(cm))); axes[0].set_yticklabels(cm.columns)
for i in range(len(cm)):
    for j in range(len(cm)):
        axes[0].text(j, i, f"{cm.iloc[i, j]:.2f}", ha="center", va="center", fontsize=8)
axes[0].set_title("Tương quan giữa các cột giá và log(Volume)")
(ret.groupby(ret.index.year).sum() * 100).plot.bar(ax=axes[1], color="#70AD47"); axes[1].set_title("Lợi suất  
log theo năm (%)")
dd = df_t["Close"] / df_t["Close"].cummax() - 1
dd.plot(ax=axes[2], c="#C00000"); axes[2].fill_between(dd.index, dd.values, 0, color="#C00000", alpha=0.2);
axes[2].set_title("Mức sụt giảm từ đỉnh (drawdown)")
plt.tight_layout(); plt.show()
```

Output



Giải thích code

Ba hình khám phá chuỗi giá và lợi suất của IBM.

$ret = \log(gi\grave{a}_t / gi\grave{a}_{t-1})$ là lợi suất log hằng ngày. Hình 1: giá đóng cửa với MA50, MA200 và khối lượng. Hình 2 (2×3): histogram log-return so với đường phân phối chuẩn, QQ-plot, biến động thực hiện 21 phiên quy năm ($\text{độ lệch chuẩn trượt} \times \sqrt{252}$), ACF của giá, của lợi suất và của |lợi suất|. Hình 3: ma trận tương quan giữa Open/High/Low/Close và $\log(\text{Volume})$, lợi suất log theo từng năm (%) và mức sụt giảm từ đỉnh (drawdown).

Giải thích output

Hình 1: IBM tăng từ khoảng 80 (2006) lên đỉnh gần 215 (2013), sau đó giảm về khoảng 120 (đầu 2016) rồi phục hồi quanh 150 đến 155 vào cuối 2017; giá có xu hướng dài hạn rõ rệt nên không dừng. Khối lượng có xu hướng giảm dần với vài đỉnh đột biến (tới ~30 triệu).

Hình 2: log-return có đỉnh nhọn và đuôi dày hơn phân phối chuẩn (đường đỏ); QQ-plot cong lệch ở hai đuôi (đặc biệt đuôi âm). Biến động 21 phiên thường ở 0,1 đến 0,3 nhưng vọt lên gần 0,7 vào cuối 2008 đến 2009 (khủng hoảng tài chính). ACF của giá gần 1 và suy giảm rất chậm (chuỗi không dừng); ACF của lợi suất bằng 0 ở hầu hết các lag (rất khó dự báo); ACF của |lợi suất| dương và suy giảm chậm (biến động tụ cụm).

Hình 3: bốn cột giá tương quan gần 1,00 với nhau (các mức giá cùng ngày gần như trùng) và tương quan -0,51 với $\log(\text{Volume})$. Lợi suất theo năm: dương ở 2006 đến 2007, 2009 đến 2012 và 2016 (cao nhất 2009 khoảng +44%), âm ở 2008 (khoảng -25%), 2013 đến 2015 và 2017. Drawdown sâu nhất khoảng -45% (2009 và 2016).

2.3 Nhận định tự động về tính chất thống kê

Cell 13: Nhận định tự động về tính chất thống kê

Code

```
a_p = acf(df_t["Close"], 20); a_r = acf(ret, 20); a_v = acf(ret.abs(), 20)
conf = 1.96 / np.sqrt(len(ret))
print(f"- Lợi suất hằng ngày: TB = {ret.mean() * 100:.3f}%, độ lệch chuẩn = {ret.std() * 100:.2f}%, "
      f"skew = {stats.skew(ret):.2f}, kurtosis dư = {stats.kurtosis(ret):.1f} (kurtosis dư > 0: đuôi dày hơn "
      f"phân phối chuẩn)")
print(f"- ACF của giá ở lag 20 = {a_p[20]:.2f} -> " + ("giá tự tương quan rất mạnh, chuỗi không dừng." if
a_p[20] > 0.5 else "tự tương quan của giá không quá mạnh."))
print(f"- ACF của lợi suất: lag-1 = {a_r[1]:.3f}, lag-5 = {a_r[5]:.3f} (ngưỡng ±{conf:.3f}) -> " + ("hầu như "
không có tín hiệu tuyến tính." if max(abs(a_r[1:6])) < 2 * conf else "có một vài tự tương quan đáng chú "
ý."))
print(f"- ACF của |lợi suất| ở lag 1 = {a_v[1]:.2f} -> " + ("biến động tụ cụm (giai đoạn dao động mạnh đi liên "
nhau)." if a_v[1] > conf else "chưa thấy hiện tượng tụ cụm biến động rõ rệt."))
try:
    from statsmodels.tsa.stattools import adfuller
    print(f"- Kiểm định ADF: p-value(giá) = {adfuller(df_t['Close'].dropna())[1]:.3f} | p-value(lợi suất) = "
          {adfuller(ret)[1]:.4f}")
    print(" (p > 0.05: không bác bỏ nghiệm đơn vị -> giá không dừng; lợi suất dừng)")
except Exception:
    print("- (Bỏ qua ADF vì chưa cài statsmodels)")
print("=> Vì vậy ta dự báo LỢI SUẤT (dùng) thay vì giá thô, rồi quy đổi lại thành giá: giá(t+1) = giá(t) × "
      exp(lợi suất dự báo).")
```


Output

```
- Lợi suất hằng ngày: TB = 0.021%, độ lệch chuẩn = 1.35%, skew = -0.12, kurtosis dư = 6.1 (kurtosis dư > 0: đuôi  
i dày hơn phân phối chuẩn)  
- ACF của giá ở lag 20 = 0.97 -> giá tự tương quan rất mạnh, chuỗi không dừng.  
- ACF của lợi suất: lag-1 = -0.011, lag-5 = -0.054 (ngưỡng ±0.036) -> hầu như không có tín hiệu tuyến tính.  
- ACF của |lợi suất| ở lag 1 = 0.22 -> biến động tự cụm (giai đoạn dao động mạnh đi liền nhau).  
- (Bỏ qua ADF vì chưa cài statsmodels)  
=> Vì vậy ta dự báo LỢI SUẤT (dùng) thay vì giá thô, rồi quy đổi lại thành giá:  $giá(t+1) = giá(t) \times \exp(\text{lợi suất dự báo})$ .
```

Giải thích code

Tính các thống kê bằng số để chứng minh: dự báo lợi suất hợp lý hơn dự báo giá.

a_p , a_r , a_v là ACF của giá, lợi suất và $|lợi suất|$. $conf = 1,96/\sqrt{n}$. Code in trung bình, độ lệch chuẩn, độ lệch (skew) và kurtosis dư của lợi suất; ACF giá lag 20; ACF lợi suất lag 1 và lag 5; ACF $|lợi suất|$ lag 1; thử chạy kiểm định ADF (nghiệm đơn vị) nếu có statsmodels. Dòng cuối nêu công thức quy đổi $giá(t+1) = giá(t) \times \exp(\text{lợi suất dự báo})$.

Giải thích output

Lợi suất hằng ngày trung bình 0,021%, độ lệch chuẩn 1,35%, skew -0,12, kurtosis dư 6,1 (đuôi dày). ACF giá ở lag 20 = 0,97 nên giá tự tương quan cực mạnh và không dừng. ACF lợi suất: lag-1 = -0,011, lag-5 = -0,054, ngưỡng ±0,036; lag-5 vượt ngưỡng một chút nhưng dưới hai lần ngưỡng nên code kết luận 'hầu như không có tín hiệu tuyến tính'. ACF $|lợi suất|$ lag-1 = 0,22, biến động tự cụm.

Dòng 'Bỏ qua ADF' xuất hiện vì máy chưa cài statsmodels nên kiểm định ADF không chạy; các kết luận về tính không dừng vẫn được hỗ trợ bởi ACF. Từ đó notebook chọn dự báo lợi suất (chuỗi dừng) thay vì giá thô.

d) Tạo đặc trưng, chia dữ liệu theo thời gian, chuẩn hóa và tạo cửa sổ

Bảng đặc trưng đều **dừng** (không phụ thuộc mức giá tuyệt đối): lợi suất, khoảng nhảy mở cửa, biên độ trong phiên, khối lượng tương đối, độ lệch so với MA5 và MA20, biến động 10 phiên. Nhấn là lợi suất phiên kế tiếp. Các bộ chuẩn hóa chỉ fit trên tập huấn luyện. Mỗi mẫu có dạng (WINDOW, 7).

Cell 15: Tạo đặc trưng, chia thời gian, chuẩn hóa, tạo cửa sổ

Code

```
FEATS = ["ret", "gap", "range", "vol_z", "ma5", "ma20", "vol10"]
N_FEATURES, OUT_DIM = len(FEATS), 1

def make_features(d):
    """Tạo đặc trưng từ OHLCV. Dữ liệu được cho dự báo quy ở phần 9."""
    o, h, l, c, v = d["Open"], d["High"], d["Low"], d["Close"], d["Volume"]
    prev_c, lv = c.shift(1), np.log1p(v)
    f = pd.DataFrame(index=d.index)
    f["ret"] = np.log(c / prev_c) # lợi suất log hôm nay
    f["gap"] = np.log(o / prev_c) # khoảng nhảy giá lúc mở cửa
    f["range"] = (h - l) / c # biên độ trong phiên
    f["vol_z"] = lv - lv.rolling(20).mean() # khối lượng so với TB 20 phiên (log)
    f["ma5"] = c / c.rolling(5).mean() - 1 # lệch so với MA5
    f["ma20"] = c / c.rolling(20).mean() - 1 # lệch so với MA20
    f["vol10"] = f["ret"].rolling(10).std() # biến động 10 phiên
    f["Close"] = c
    return f.replace([np.inf, -np.inf], np.nan)

feat = make_features(df_t)
feat["target"] = feat["ret"].shift(-1) # NHÃN: lợi suất của phiên kế tiếp
feat = feat.dropna(subset=FEATS) # hàng cuối có target=NaN được giữ lại cho dự báo thật
display(feat.tail(3))

F = feat[FEATS].values.astype("float32")
T = feat["target"].values
C = feat["Close"].values
idx = feat.index
n = len(feat); n_lab = n - 1 # số hàng có nhãn
i1, i2 = int(n_lab * TRAIN_RATIO), int(n_lab * (TRAIN_RATIO + VAL_RATIO))

scaler_x = StandardScaler().fit(F[:i1])
scaler_y = StandardScaler().fit(T[:i1].reshape(-1, 1))
F_s = scaler_x.transform(F).astype("float32")

end_idx = np.arange(WINDOW - 1, n_lab) # chỉ số phiên cuối của mỗi cửa sổ
X_all = np.stack([F_s[t - WINDOW + 1:t + 1] for t in end_idx]).astype("float32")
y_all = scaler_y.transform(T[end_idx].reshape(-1, 1)).astype("float32")
m_tr, m_va, m_te = end_idx < i1, (end_idx >= i1) & (end_idx < i2), end_idx >= i2
X_train, y_train = X_all[m_tr], y_all[m_tr]
X_val, y_val = X_all[m_va], y_all[m_va]
X_test, y_test = X_all[m_te], y_all[m_te]

print(f"Train: {X_train.shape} | Val: {X_val.shape} | Test: {X_test.shape}")
print(f"Train: {idx[WINDOW]:%Y-%m-%d} -> {idx[i1]:%Y-%m-%d} | Val: {idx[i1 + 1]:%Y-%m-%d} -> {idx[i2]:%Y-%m-%d} | Test: {idx[i2 + 1]:%Y-%m-%d} -> {idx[-1]:%Y-%m-%d}")

fig, ax = plt.subplots(figsize=(13, 3.5))
ax.plot(idx, C, lw=0.8, c="gray")
for lo, hi, c_, lb in [(0, i1, "#2F5496", "Train"), (i1, i2, "#ED7D31", "Validation"), (i2, n, "#70AD47", "Test")]:
    ax.axvspan(idx[lo], idx[hi - 1], color=c_, alpha=0.15, label=lb)
ax.set_title("Chia dữ liệu theo trình tự thời gian (không xáo trộn)"); ax.legend(); plt.show()
```

Output

```
      ret    gap  range  vol_z   ma5   ma20  vol10   Close \
Date
2017-12-27  0.0020 0.0008 0.0037 -0.7202 0.0036 -0.0052 0.0078 153.1300
2017-12-28  0.0059 0.0005 0.0060 -0.4647 0.0081  0.0006 0.0058 154.0400
2017-12-29 -0.0040 0.0008 0.0085 -0.2177 0.0015 -0.0033 0.0060 153.4200

      target
Date
2017-12-27  0.0059
2017-12-28 -0.0040
2017-12-29   NaN
Train: (2041, 60, 7) | Val: (450, 60, 7) | Test: (450, 60, 7)
Train: 2006-04-27 -> 2014-06-05 | Val: 2014-06-06 -> 2016-03-18 | Test: 2016-03-21 -> 2017-12-29
```



Giải thích code

Sinh 7 đặc trưng dùng từ OHLCV, chia dữ liệu theo thời gian, chuẩn hóa và tạo cửa sổ 60 phiên.

make_features tạo: ret (lợi suất log hôm nay), gap (log giá mở cửa so với đóng cửa hôm trước), range ((High – Low)/Close, biên độ trong phiên), vol_z (log khối lượng trừ trung bình 20 phiên), ma5 và ma20 (độ lệch của giá so với MA5/MA20), vol10 (độ lệch chuẩn lợi suất 10 phiên). Cột Close được giữ lại để quy đổi sang giá. Nhãn target là ret của phiên kế tiếp (shift(-1)); hàng cuối cùng không có nhãn được giữ để dự báo thật.

Chuẩn hóa: scaler_x (đặc trưng) và scaler_y (nhãn) chỉ fit trên phần huấn luyện. end_idx là chỉ số phiên cuối của mỗi cửa sổ; X_all ghép các đoạn 60 phiên, y_all là nhãn đã chuẩn hóa. Ba mặt nạ tách train/val/test theo thứ tự thời gian, rồi vẽ đồ thị giá tô màu ba vùng.

Giải thích output

Bảng 3 dòng cuối: ngày 29/12/2017 có Close 153,42 và target = NaN (chưa biết lợi suất ngày sau), đúng như thiết kế.

Kích thước tập: Train (2.041, 60, 7), Val (450, 60, 7), Test (450, 60, 7). Train từ 27/04/2006 đến 05/06/2014, Val từ 06/06/2014 đến 18/03/2016, Test từ 21/03/2016 đến 29/12/2017.

Đồ thị: vùng train chứa giá lên tới đỉnh 215, vùng val chứa giai đoạn giảm mạnh xuống khoảng 120, vùng test chứa phục hồi và đi ngang 140 đến 180. Mức giá trong test nằm trong phạm vi đã thấy ở train.

e) Mô hình cơ sở (baseline)

**Naive* dự báo lợi suất = 0 (giá ngày mai = giá hôm nay) và *Drift* dự báo bằng lợi suất trung bình của tập train. Với dữ liệu này, baseline Naive rất khó đánh bại.*

Cell 17: Mô hình cơ sở (baseline) và hàm đánh giá

Code

```
def eval_split(mask, ret_hat):
    """Quy đổi lợi suất dự báo -> giá và tính các chỉ số."""
    t = end_idx[mask]
    close_t, ret_true = C[t], T[t]
    p_true, p_hat = close_t * np.exp(ret_true), close_t * np.exp(ret_hat)
    sse, sse0 = np.sum((ret_true - ret_hat) ** 2), np.sum(ret_true ** 2)
    nz = np.any(ret_hat != 0)
    return {
        "RMSE giá": float(np.sqrt(np.mean((p_true - p_hat) ** 2))),
        "MAE giá": float(np.mean(np.abs(p_true - p_hat))),
        "MAPE giá (%)": float(np.mean(np.abs((p_true - p_hat) / p_true)) * 100),
        "RMSE lợi suất": float(np.sqrt(sse / len(t))),
        "R² so với dự báo 0": float(1 - sse / sse0),
        "Đúng hướng (%)": float(np.mean(np.sign(ret_hat) == np.sign(ret_true)) * 100) if nz else np.nan,
    }

masks = {"val": m_va, "test": m_te}
ret_true = {k: T[end_idx[m]] for k, m in masks.items()}
drift = float(T[:11].mean()) # lợi suất trung bình của tập train
ret_hats = {k: {"Naive (giá hôm nay)": np.zeros(m.sum()), "Drift (TB train)": np.full(m.sum(), drift)} for k, m
in masks.items()}

up_rate = float(np.mean(ret_true["test"] > 0) * 100)
print(f"Tỷ lệ phiên tăng giá trong tập test: {up_rate:.1f}% (mốc so sánh cho độ chính xác hướng: luôn đoán 'tăng' đạt ≈ 51.3%)")
display(pd.DataFrame({k: eval_split(masks["test"], v) for k, v in ret_hats["test"].items()}))
```

Output

Tỷ lệ phiên tăng giá trong tập test: 51.3% (mốc so sánh cho độ chính xác hướng: luôn đoán 'tăng' đạt ≈ 51.3%)

	RMSE giá	MAE giá	MAPE giá (%)	RMSE lợi suất \
Naive (giá hôm nay)	1.6180	1.0723	0.6870	0.0104
Drift (TB train)	1.6190	1.0723	0.6870	0.0104

	R² so với dự báo 0	Đúng hướng (%)
Naive (giá hôm nay)	0.0000	NaN
Drift (TB train)	-0.0008	51.3333

Giải thích code

Định nghĩa `eval_split` và dựng hai baseline: Naive và Drift.

`eval_split(mask, ret_hat)` quy lợi suất dự báo thành giá dự báo: $p_hat = Close_t \times \exp(ret_hat)$, rồi tính RMSE/MAE/MAPE của giá, RMSE lợi suất, R^2 so với dự báo 0 ($1 - SSE/SSE_0$, dương nghĩa là tốt hơn dự báo 'lợi suất = 0') và tỷ lệ đúng hướng. Baseline Naive dự báo lợi suất bằng 0; baseline Drift dự báo bằng lợi suất trung bình tập train. `up_rate` là tỷ lệ phiên tăng giá của tập test, mốc để đánh giá độ chính xác hướng.

Giải thích output

Trong tập test có 51,3% phiên tăng giá, nên chiến lược luôn đoán 'tăng' đã đạt 51,3% (mốc cần vượt).

Naive: RMSE giá 1,618, MAE 1,072, MAPE 0,687%, RMSE lợi suất 0,0104; $R^2 = 0$ theo định nghĩa. Drift: RMSE giá 1,619, $R^2 = -0,0008$, đúng hướng 51,33%. Độ chính xác hướng của Naive là NaN vì dự báo bằng 0 không có dấu. Hai baseline gần như trùng nhau nên mô hình học sâu phải cải thiện dự báo 0 mới có ý nghĩa; sai số giá 1,6 chỉ ~1% mức giá 155.

f) Xây dựng và huấn luyện RNN / LSTM / GRU (Keras (TensorFlow))

Ba kiến trúc dùng cùng siêu tham số để so sánh công bằng.

Cell 19: Xây dựng mô hình Keras và xem cấu trúc

Code

```
def build_model(kind, input_shape, out_dim):
    """2 lớp RNN/LSTM/GRU xếp chồng + Dropout + đầu ra Dense."""
    Cell = {"RNN": layers.SimpleRNN, "LSTM": layers.LSTM, "GRU": layers.GRU}[kind]
    model = keras.Sequential([
        layers.Input(shape=input_shape),
        Cell(UNITS, return_sequences=True),
        layers.Dropout(DROPOUT),
        Cell(UNITS),
        layers.Dropout(DROPOUT),
        layers.Dense(32, activation="relu"),
        layers.Dense(out_dim),
    ], name=kind)
    model.compile(optimizer=keras.optimizers.Adam(LR), loss="mse")
    return model

def train_model(kind, X_tr, y_tr, X_va, y_va):
    keras.utils.set_random_seed(SEED)
    model = build_model(kind, X_tr.shape[1:], y_tr.shape[1])
    callbacks = [
        keras.callbacks.EarlyStopping(monitor="val_loss", patience=PATIENCE, restore_best_weights=True),
        keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=max(3, PATIENCE // 3),
                                          min_lr=1e-5),
    ]
    h = model.fit(X_tr, y_tr, validation_data=(X_va, y_va), epochs=EPOCHS,
                  batch_size=BATCH_SIZE, callbacks=callbacks, verbose=1)
    n_ep = len(h.history["loss"])
    print(f"({kind:5s}) | dừng sau {n_ep:3d} epoch | val_loss tốt nhất = {min(h.history['val_loss']):.4f} "
          f"| số tham số = {model.count_params():,}")
    return model, {"loss": h.history["loss"], "val_loss": h.history["val_loss"]}

def predict_scaled(model, X):
    """Dự đoán trên không gian đã chuẩn hóa, trả về mảng (n_mẫu, out_dim)."""
    return model.predict(X, batch_size=256, verbose=1).reshape(len(X), -1)

# Xem cấu trúc một mô hình mẫu
build_model("LSTM", (WINDOW, N_FEATURES), OUT_DIM).summary()
```

Output

Model: "LSTM"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 60, 64)	18,432
dropout (Dropout)	(None, 60, 64)	0
lstm_1 (LSTM)	(None, 64)	33,024
dropout_1 (Dropout)	(None, 64)	0
dense (Dense)	(None, 32)	2,080
dense_1 (Dense)	(None, 1)	33

Total params: 53,569 (209.25 KB)

Trainable params: 53,569 (209.25 KB)

Non-trainable params: 0 (0.00 B)

Giải thích code

Giống cell 20 của Bài 1 nhưng với đầu vào (60, 7) và một đầu ra (lợi suất chuẩn hóa).

build_model xếp hai lớp SimpleRNN/LSTM/GRU 64 đơn vị (lớp đầu return_sequences=True) với Dropout, Dense(32, relu) và Dense(1). train_model

dùng EarlyStopping (patience 12, restore_best_weights) và ReduceLROnPlateau; predict_scaled bọc model.predict. Cuối cell in summary của mô hình LSTM.

Giải thích output

LSTM: lớp 1 có 18.432 tham số ($= 4 \times (64 \times (7 + 64) + 64)$), lớp 2 có 33.024, Dense(32) có 2.080, Dense(1) có 33; tổng 53.569 tham số (209,25 KB). Ít hơn Bài 1 (53.924) vì đầu vào 7 đặc trưng thay vì 8 và đầu ra 1 thay vì 4.

Cell 20: Huấn luyện ba mô hình RNN, LSTM, GRU

Code

```
KINDS = ["RNN", "LSTM", "GRU"]
models, histories = {}, {}
for kind in KINDS:
    models[kind], histories[kind] = train_model(kind, X_train, y_train, X_val, y_val)
```

Output

```
Epoch 1/100
64/64 _____ 5s 28ms/step - loss: 1.1412 - val_loss: 0.8961 - learning_rate: 0.0010
Epoch 2/100
64/64 _____ 2s 18ms/step - loss: 1.0757 - val_loss: 0.9088 - learning_rate: 0.0010
Epoch 3/100
64/64 _____ 1s 16ms/step - loss: 1.0252 - val_loss: 0.8939 - learning_rate: 0.0010
Epoch 4/100
64/64 _____ 1s 18ms/step - loss: 1.0199 - val_loss: 0.9058 - learning_rate: 0.0010
Epoch 5/100
64/64 _____ 1s 18ms/step - loss: 0.9909 - val_loss: 0.9003 - learning_rate: 0.0010
Epoch 6/100
64/64 _____ 1s 19ms/step - loss: 0.9690 - val_loss: 0.9369 - learning_rate: 0.0010
Epoch 7/100
64/64 _____ 1s 18ms/step - loss: 0.9514 - val_loss: 0.9518 - learning_rate: 0.0010
Epoch 8/100
64/64 _____ 2s 27ms/step - loss: 0.9150 - val_loss: 0.9265 - learning_rate: 5.0000e-04
Epoch 9/100
64/64 _____ 3s 27ms/step - loss: 0.8878 - val_loss: 0.9487 - learning_rate: 5.0000e-04
Epoch 10/100
64/64 _____ 3s 26ms/step - loss: 0.8613 - val_loss: 0.9669 - learning_rate: 5.0000e-04
Epoch 11/100
64/64 _____ 1s 16ms/step - loss: 0.8832 - val_loss: 0.9822 - learning_rate: 5.0000e-04
... (đã rút gọn 57 dòng ở giữa) ...
Epoch 11/100
64/64 _____ 5s 52ms/step - loss: 0.9752 - val_loss: 0.9165 - learning_rate: 5.0000e-04
Epoch 11/100
64/64 _____ 3s 46ms/step - loss: 0.9625 - val_loss: 0.9245 - learning_rate: 5.0000e-04
Epoch 12/100
64/64 _____ 5s 46ms/step - loss: 0.9576 - val_loss: 0.9231 - learning_rate: 5.0000e-04
Epoch 13/100
64/64 _____ 3s 45ms/step - loss: 0.9516 - val_loss: 0.9258 - learning_rate: 5.0000e-04
Epoch 14/100
64/64 _____ 6s 50ms/step - loss: 0.9371 - val_loss: 0.9163 - learning_rate: 2.5000e-04
Epoch 15/100
64/64 _____ 3s 47ms/step - loss: 0.9279 - val_loss: 0.9238 - learning_rate: 2.5000e-04
Epoch 16/100
64/64 _____ 5s 45ms/step - loss: 0.9251 - val_loss: 0.9299 - learning_rate: 2.5000e-04
Epoch 17/100
64/64 _____ 3s 51ms/step - loss: 0.9188 - val_loss: 0.9282 - learning_rate: 2.5000e-04
GRU | dừng sau 17 epoch | val_loss tốt nhất = 0.8887 | số tham số = 41,089
```

Giải thích code

Vòng lặp huấn luyện RNN, LSTM, GRU với cùng tham số (giống Bài 1).

Giải thích output

Log các epoch (đã rút gọn trong ảnh). Kết quả: RNN dừng sau 15 epoch, val_loss tốt nhất 0,8939, 14.977 tham số; LSTM dừng sau 14 epoch, 0,8809, 53.569 tham số; GRU dừng sau 17 epoch, 0,8887, 41.089 tham số.

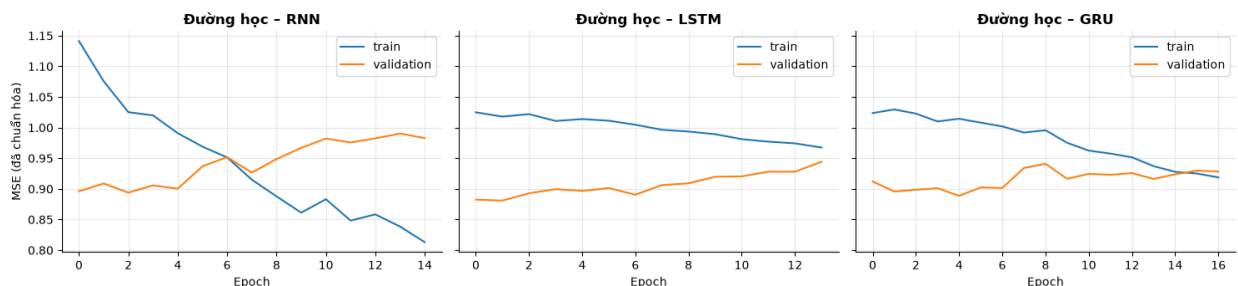
$\text{val_loss} < 1$ (khoảng 0,88) có thể do nhãn được chuẩn hóa theo độ lệch chuẩn của tập train, vốn chứa cả cuộc khủng hoảng 2008 đến 2009 với biến động rất cao, nên độ lệch chuẩn dùng để chuẩn hóa lớn hơn của giai đoạn kiểm định và đoán 0 cũng cho MSE dưới 1. Do đó không nên coi 0,88 là bằng chứng học được; cần so với baseline ở cell 23.

Cell 21: Đường học

Code

```
fig, axes = plt.subplots(1, 3, figsize=(15, 3.6), sharey=True)
for ax, kind in zip(axes, KINDS):
    ax.plot(histories[kind]["loss"], label="train"); ax.plot(histories[kind]["val_loss"], label="validation")
    ax.set_title(f"Đường học - {kind}"); ax.set_xlabel("Epoch"); ax.legend()
axes[0].set_ylabel("MSE (đã chuẩn hóa)")
plt.tight_layout(); plt.show()
```

Output



Giải thích code

Vẽ loss huấn luyện và validation theo epoch cho 3 mô hình (giống Bài 1).

Giải thích output

Train loss giảm (RNN mạnh nhất, từ khoảng 1,14 xuống 0,81) trong khi validation loss gần như đi ngang rồi tăng (RNN từ 0,90 lên 0,98; LSTM từ 0,88 lên khoảng 0,945; GRU quanh 0,89 đến 0,93). Đó là dấu hiệu quá khớp điển hình khi mô hình ghi nhớ nhiều; early stopping khôi phục trọng số ở epoch có val_loss thấp nhất, tức các epoch đầu.

g) Đánh giá và trực quan hóa dự đoán

Các chỉ số được tính ở cả mức **giá** (dễ đọc, nhưng bị lạc quan giả tạo do tự tương quan) và mức **lợi suất** (phản ánh khả năng dự báo thật), cùng độ chính xác dự đoán hướng.

Cell 23: Đánh giá: bảng chỉ số và độ chính xác hướng

Code

```
inv_y = lambda z: scaler_y.inverse_transform(np.asarray(z).reshape(-1, 1)).ravel()
for kind in KINDS:
    for k, X in [("val", X_val), ("test", X_test)]:
        ret_hats[k][kind] = inv_y(predict_scaled(models[kind], X)[: , 0])

tab = {name: {**{"f{m}" (val)": v for m, v in eval_split(masks["val"], ret_hats["val"][name]).items() if m in
["RMSE lợi suất"]},
**eval_split(masks["test"], ret_hats["test"][name])} for name in ret_hats["test"]}
results = pd.DataFrame(tab).T
display(results[["RMSE giá", "MAE giá", "MAPE giá (%)", "RMSE lợi suất", "R2 so với dự báo 0", "Đúng hướng
(%)"]].sort_values("RMSE giá"))

best_kind = min(KINDS, key=lambda k: results.loc[k, "RMSE lợi suất (val)"])
print(f"Mô hình tốt nhất trên tập kiểm định (theo RMSE lợi suất): {best_kind}")

fig, axes = plt.subplots(1, 2, figsize=(14, 3.8))
s = results[["RMSE giá"]].sort_values()
s.plot.barh(ax=axes[0], color=["#2F5496" if i in KINDS else "#A6A6A6" for i in s.index]);
axes[0].set_title("RMSE giá - tập test (xanh: RNN, xám: baseline)")
d = results.loc[KINDS, "Đúng hướng (%)"]
d.plot.bar(ax=axes[1], color="#70AD47"); axes[1].axhline(50, c="k", ls=":", label="50%");
axes[1].axhline(up_rate, c="r", ls="--", label=f"luôn đoán tăng = {up_rate:.1f}%")
axes[1].set_ylim(30, 70); axes[1].set_title("Độ chính xác dự đoán hướng (test)"); axes[1].legend();
axes[1].tick_params(axis="x", rotation=0)
plt.tight_layout(); plt.show()
```

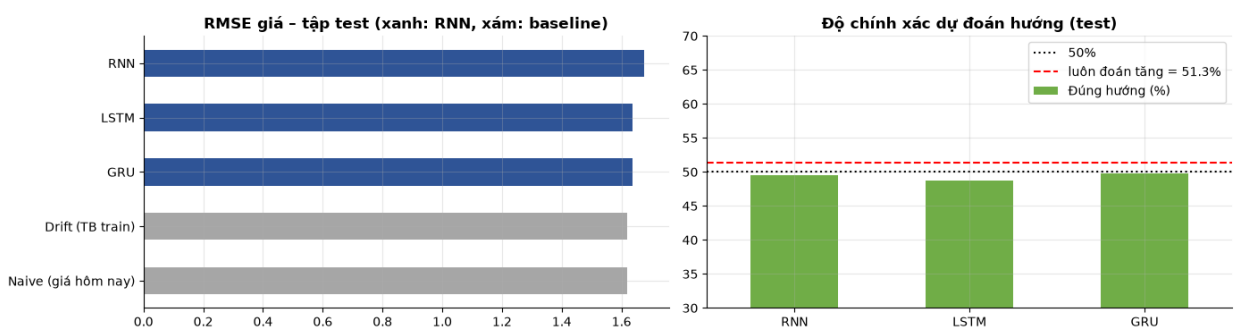
Output

```
2/2 ----- 1s 364ms/step
2/2 ----- 0s 39ms/step
2/2 ----- 1s 429ms/step
2/2 ----- 0s 100ms/step
WARNING:tensorflow:5 out of the last 9 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_s
tep_on_data_distributed at 0x000001F7D35BE980> triggered tf.function retracing. Tracing is expensive and the exc
essive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors wi
th different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function out
side of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. F
or (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorfl
ow.org/api_docs/python/tf/function for more details.
1/2 ----- 0s 439ms/stepWARNING:tensorflow:6 out of the last 10 calls to <function TensorFlowTrain
er.make_predict_function.<locals>.one_step_on_data_distributed at 0x000001F7D35BE980> triggered tf.function retr
acing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repea
tedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (
1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option
that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#control
ling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.
2/2 ----- 1s 408ms/step
2/2 ----- 0s 74ms/step
```

	RMSE giá	MAE giá	MAPE giá (%)	RMSE lợi suất \
Naive (giá hôm nay)	1.6180	1.0723	0.6870	0.0104
Drift (TB train)	1.6190	1.0723	0.6870	0.0104
GRU	1.6354	1.0912	0.6993	0.0105
LSTM	1.6368	1.0866	0.6961	0.0105
RNN	1.6742	1.1316	0.7246	0.0108

	R ² so với dự báo 0	Đúng hướng (%)
Naive (giá hôm nay)	0.0000	NaN
Drift (TB train)	-0.0008	51.3333
GRU	-0.0215	49.7778
LSTM	-0.0219	48.6667
RNN	-0.0670	49.5556

Mô hình tốt nhất trên tập kiểm định (theo RMSE lợi suất): LSTM



Giải thích code

Dự đoán val/test, đảo chuẩn hóa nhãn, tổng hợp bảng kết quả và chọn mô hình tốt nhất.

inv_y đảo chuẩn hóa nhãn bằng scaler_y. Bảng results gồm cả 3 RNN và 2 baseline với RMSE giá, MAE giá, MAPE giá, RMSE lợi suất, R^2 so với dự báo 0 và tỷ lệ đúng hướng. best_kind chọn theo RMSE lợi suất trên tập val. Hai biểu đồ: RMSE giá (xanh: RNN, xám: baseline) và độ chính xác hướng so với 50% và mốc 'luôn đoán tăng' 51,3%.

Giải thích output

Dòng cảnh báo tf.function retracing chỉ là thông báo hiệu năng của TensorFlow (gọi predict nhiều lần với kích thước batch khác nhau), không ảnh hưởng kết quả.

Bảng: Naive RMSE giá 1,6180; Drift 1,6190; GRU 1,6354 (R^2 -0,0215, đúng hướng 49,78%); LSTM 1,6368 (R^2 -0,0219, 48,67%); RNN 1,6742 (R^2 -0,0670, 49,56%). Không mạng nào đánh bại Naive: RMSE cao hơn và R^2 âm. Độ chính xác hướng của cả ba (48,7 đến 49,8%) đều dưới mốc 50% và dưới mốc 51,3% của việc 'luôn đoán tăng'. Mô hình tốt nhất trên val: LSTM.

Cell 24: Đồ thị dự đoán giá và phân tích ở mức lợi suất

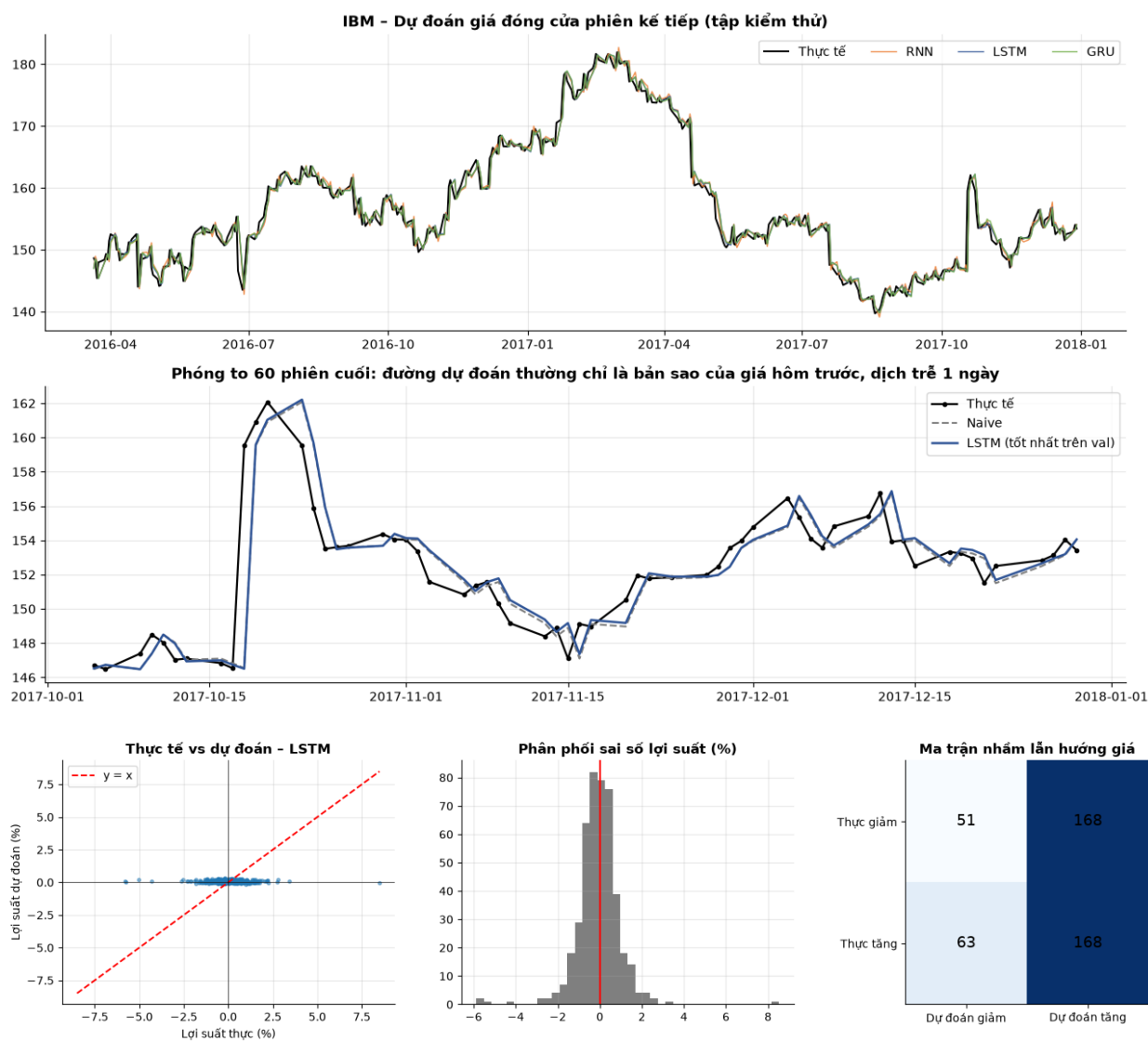
Code

```
t_te = end_idx[m_te]
d_te = idx[t_te + 1] # ngày của giá được dự đoán
price_true = C[t_te + 1]
price_hat = {k: C[t_te] * np.exp(ret_hats["test"][k]) for k in ret_hats["test"]}
colors = {"RNN": "#ED7D31", "LSTM": "#2F5496", "GRU": "#70AD47"}

fig, axes = plt.subplots(2, 1, figsize=(13, 8))
axes[0].plot(d_te, price_true, c="k", lw=1.4, label="Thực tế")
for k in KINDS: axes[0].plot(d_te, price_hat[k], c=colors[k], lw=1, alpha=0.85, label=k)
axes[0].set_title(f"{TICKER} - Dự đoán giá đóng cửa phiên kế tiếp (tập kiểm thử)"); axes[0].legend(ncol=4)
z = slice(-60, None)
axes[1].plot(d_te[z], price_true[z], c="k", lw=1.6, marker="o", ms=3, label="Thực tế")
axes[1].plot(d_te[z], price_hat["Naive (giá hôm nay)"][z], c="gray", ls="--", label="Naive")
axes[1].plot(d_te[z], price_hat[best_kind][z], c=colors[best_kind], lw=1.8, label=f"{best_kind} (tốt nhất trên val)")
axes[1].set_title("Phóng to 60 phiên cuối: đường dự đoán thường chỉ là bản sao của giá hôm trước, dịch trễ 1 ngày"); axes[1].legend()
plt.tight_layout(); plt.show()

# --- Phân tích ở mức lợi suất (nơi thật sự đo được khả năng dự báo) ---
rt, rh = ret_true["test"], ret_hats["test"][best_kind]
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
axes[0].scatter(rt * 100, rh * 100, s=8, alpha=0.5); lim = max(abs(rt).max(), abs(rh).max()) * 100
axes[0].plot([-lim, lim], [-lim, lim], "r--", label="y = x"); axes[0].axhline(0, c="k", lw=0.5);
axes[0].axvline(0, c="k", lw=0.5)
axes[0].set_xlabel("Lợi suất thực (%)"); axes[0].set_ylabel("Lợi suất dự đoán (%)"); axes[0].set_title(f"Thực tế vs dự đoán - {best_kind}"); axes[0].legend()
axes[1].hist((rt - rh) * 100, bins=40, color="#7F7F7F"); axes[1].axvline(0, c="r"); axes[1].set_title("Phân phối sai số lợi suất (%)")
cmx = pd.crosstab(np.sign(rt) > 0, np.sign(rh) > 0).reindex(index=[False, True], columns=[False, True], fill_value=0)
axes[2].imshow(cmx.values, cmap="Blues"); axes[2].grid(False)
axes[2].set_xticks([0, 1]); axes[2].set_xticklabels(["Dự đoán giảm", "Dự đoán tăng"]); axes[2].set_yticks([0, 1]); axes[2].set_yticklabels(["Thực giảm", "Thực tăng"])
for i in range(2):
    for j in range(2): axes[2].text(j, i, int(cmx.values[i, j]), ha="center", va="center", fontsize=14)
axes[2].set_title("Ma trận nhầm lẫn hướng giá")
plt.tight_layout(); plt.show()
print(f"Độ lệch chuẩn lợi suất thực = {rt.std() * 100:.2f}% | dự đoán = {rh.std() * 100:.3f}% "
      f"(dự đoán co cụm quanh 0 nghĩa là mô hình rất thận trọng, đúng với bản chất nhiều của lợi suất)")
```

Output



Độ lệch chuẩn lợi suất thực = 1.04% | dự đoán = 0.091% (dự đoán co cụm quanh 0 nghĩa là mô hình rất thận trọng, đúng với bản chất nhiễu của lợi suất)

Giải thích code

Quy đổi ngược sang giá và vẽ; phân tích lợi suất dự đoán của mô hình tốt nhất.

d_te là ngày của giá được dự đoán; $price_hat = Close_t \times \exp(ret_hat)$. Hình 1: giá thực tế và dự đoán của 3 mô hình trên toàn tập test và phóng to 60 phiên cuối so với Naive. Hình 2: phân tán lợi suất thực–dự đoán (với $y = x$), histogram sai số lợi suất, ma trận nhầm lẫn hướng (tăng/giảm). Dòng in cuối so sánh độ lệch chuẩn của lợi suất thực và dự đoán.

Giải thích output

Hình 1: đường dự đoán gần như chồng khít giá thực. Nhưng hình phóng to cho thấy đường LSTM/Naive chỉ là bản sao của giá hôm trước, trễ một phiên (ví dụ cú nhảy giá giữa 10/2017 được 'phát hiện' chậm một ngày). Vì vậy độ khớp trên đồ thị giá không chứng minh khả năng dự báo.

Hình 2: lợi suất dự đoán co cụm sát 0 (dải ngang hẹp), lệch hẳn đường $y = x$. Ma trận nhầm lẫn: thực giảm – dự đoán giảm 51, thực giảm – dự đoán tăng 168, thực tăng – dự đoán giảm 63, thực tăng – dự đoán tăng 168; tổng đúng $219/450 = 48,7\%$. Mô hình đoán 'tăng' 336/450 lần (75%).

Độ lệch chuẩn lợi suất thực 1,04% nhưng dự đoán chỉ 0,091%: mô hình gần như luôn đưa ra dự báo rất sát 0, đúng với bản chất nhiễu của lợi suất.

Cell 25: Minh họa chiến lược giao dịch theo dấu dự đoán

Code

```
# ---- Minh họa: chiến lược mua/bán không theo dấu dự đoán so với mua và giữ (KHÔNG tính phí giao dịch) ----
pos_ = np.sign(ret_hats["test"][best_kind])
strat = pos_ * ret_true["test"]
fig, ax = plt.subplots(figsize=(12, 3.8))
ax.plot(d_te, np.exp(np.cumsum(ret_true["test"])), label="Mua và giữ", c="gray")
ax.plot(d_te, np.exp(np.cumsum(strat)), label=f"Theo dấu dự đoán {best_kind}", c=colors[best_kind])
ax.set_title("Giá trị tích lũy của 1 đơn vị vốn (chưa tính chi phí)"); ax.legend(); plt.show()
print(f"Sharpe (quy năm) chiến lược: {strat.mean() / (strat.std() + 1e-12) * np.sqrt(252):.2f} | mua-giữ: {ret_true['test'].mean() / ret_true['test'].std() * np.sqrt(252):.2f}")
print("Chỉ mang tính minh họa: kết quả không đủ để kết luận về khả năng sinh lời thực tế.")
```

Output



Sharpe (quy năm) chiến lược: -1.51 | mua-giữ: 0.14

Chỉ mang tính minh họa: kết quả không đủ để kết luận về khả năng sinh lời thực tế.

Giải thích code

Kiểm tra nhanh nếu giao dịch theo dấu dự đoán thì lãi lỗ ra sao (không tính phí).

$pos_ = \text{sign}(\text{lợi suất dự đoán})$: +1 mua, -1 bán không. Lợi suất chiến lược mỗi phiên = $pos_ \times \text{lợi suất thực}$. Giá trị tích lũy của 1 đơn vị vốn là $\exp(\text{cumsum}(\text{lợi suất}))$; so với 'mua và giữ'. Sharpe quy năm = trung bình / độ lệch chuẩn $\times \sqrt{252}$.

Giải thích output

Đường chiến lược (xanh) đi xuống khoảng 0,64 vào cuối kỳ, trong khi mua và giữ (xám) kết thúc khoảng 1,05. Sharpe chiến lược -1,51, mua-giữ +0,14: theo tín hiệu này sẽ lỗ khoảng 36% vốn. Điều này khớp với độ chính xác hướng < 50%. Notebook nhắc rõ kết quả chỉ mang tính minh họa.

h) Phân tích đặc trưng nào quan trọng

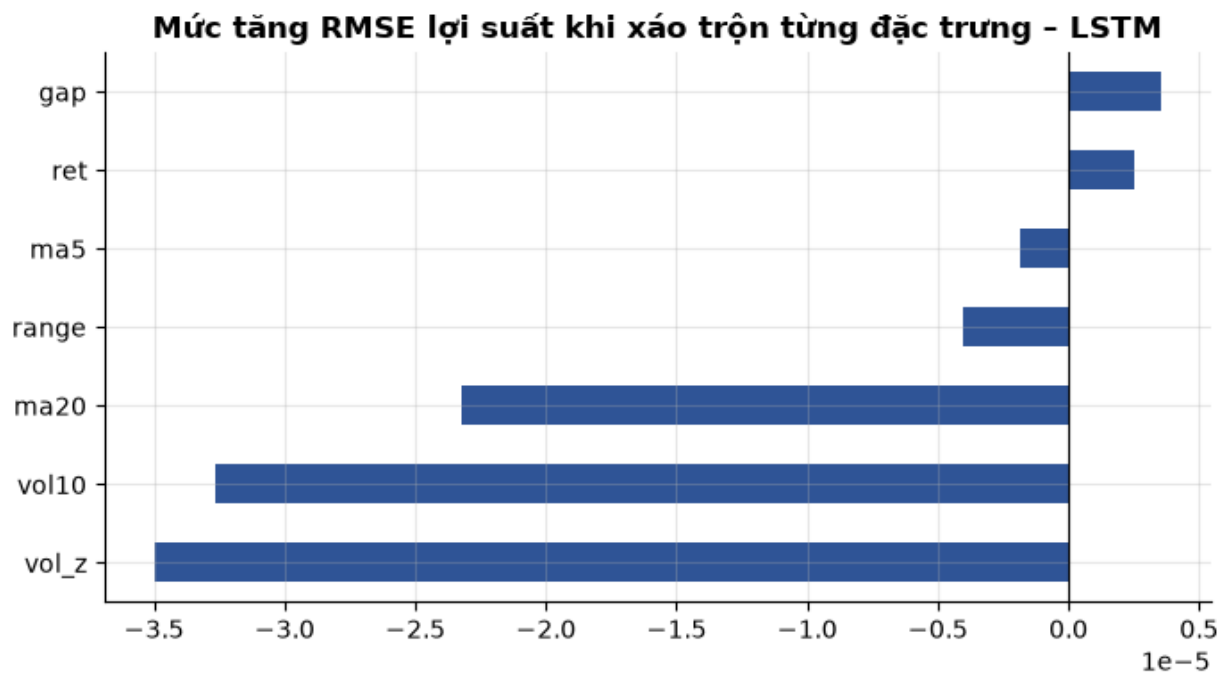
Cell 27: Độ quan trọng đặc trưng bằng hoán vị

Code

```
base = eval_split(masks["test"], ret_hats["test"][best_kind])["RMSE lợi suất"]
rng = np.random.default_rng(SEED)
imp = {}
for j, name in enumerate(FEATS):
    ds = []
    for _ in range(5):
        Xp = X_test.copy(); Xp[:, :, j] = Xp[rng.permutation(len(Xp))][:, :, j]
        ds.append(eval_split(masks["test"], inv_y(predict_scaled(models[best_kind], Xp[:, 0])))["RMSE lợi suất"] - base)
    imp[name] = np.mean(ds)
imp = pd.Series(imp).sort_values()
fig, ax = plt.subplots(figsize=(8, 4)); imp.plot.barh(ax=ax, color="#2F5496"); ax.axvline(0, c="k", lw=0.8)
ax.set_title(f"Mức tăng RMSE lợi suất khi xáo trộn từng đặc trưng - {best_kind}"); plt.show()
print("Đặc trưng mô hình dựa vào nhiều nhất:", imp.idxmax())
```

Output

```
2/2 _____ 0s 81ms/step
2/2 _____ 0s 78ms/step
2/2 _____ 0s 70ms/step
2/2 _____ 0s 81ms/step
2/2 _____ 0s 70ms/step
2/2 _____ 0s 75ms/step
2/2 _____ 0s 72ms/step
2/2 _____ 0s 98ms/step
2/2 _____ 0s 86ms/step
2/2 _____ 0s 116ms/step
2/2 _____ 0s 100ms/step
2/2 _____ 0s 66ms/step
2/2 _____ 0s 80ms/step
2/2 _____ 0s 77ms/step
2/2 _____ 0s 80ms/step
2/2 _____ 0s 83ms/step
2/2 _____ 0s 64ms/step
2/2 _____ 0s 79ms/step
2/2 _____ 0s 78ms/step
2/2 _____ 0s 69ms/step
2/2 _____ 0s 71ms/step
2/2 _____ 0s 68ms/step
2/2 _____ 0s 69ms/step
2/2 _____ 0s 71ms/step
2/2 _____ 0s 107ms/step
2/2 _____ 0s 89ms/step
2/2 _____ 0s 72ms/step
2/2 _____ 0s 66ms/step
2/2 _____ 0s 76ms/step
2/2 _____ 0s 108ms/step
2/2 _____ 0s 70ms/step
2/2 _____ 0s 83ms/step
2/2 _____ 0s 70ms/step
2/2 _____ 0s 78ms/step
2/2 _____ 0s 78ms/step
```



Đặc trưng mô hình dựa vào nhiều nhất: gap

Giải thích code

Như Bài 1: xáo trộn từng đặc trưng trong tập test và đo mức tăng RMSE lợi suất. Với mỗi đặc trưng trong FEATS lặp 5 lần hoán vị, dự đoán lại rồi tính RMSE lợi suất (nhờ eval_split) trừ RMSE gốc.

Giải thích output

Đặc trưng 'dựa nhiều nhất' là gap, nhưng trục đồ thị có thang 1e-5: mức tăng cao nhất chỉ khoảng $3,5 \times 10^{-5}$ trên RMSE lợi suất 0,0105 (0,0003%), còn vol_z, vol10, ma20 cho giá trị âm nhẹ (khoảng -2 đến $-3,5 \times 10^{-5}$). Toàn bộ chênh lệch đều nhỏ đến mức vô nghĩa, xác nhận mô hình gần như không dùng đặc trưng nào.

i) Dự báo tương lai

Dự báo đệ quy HORIZON phiên sau phiên cuối cùng của dữ liệu.

Cell 29: Dự báo đệ quy 5 phiên tiếp theo

Code

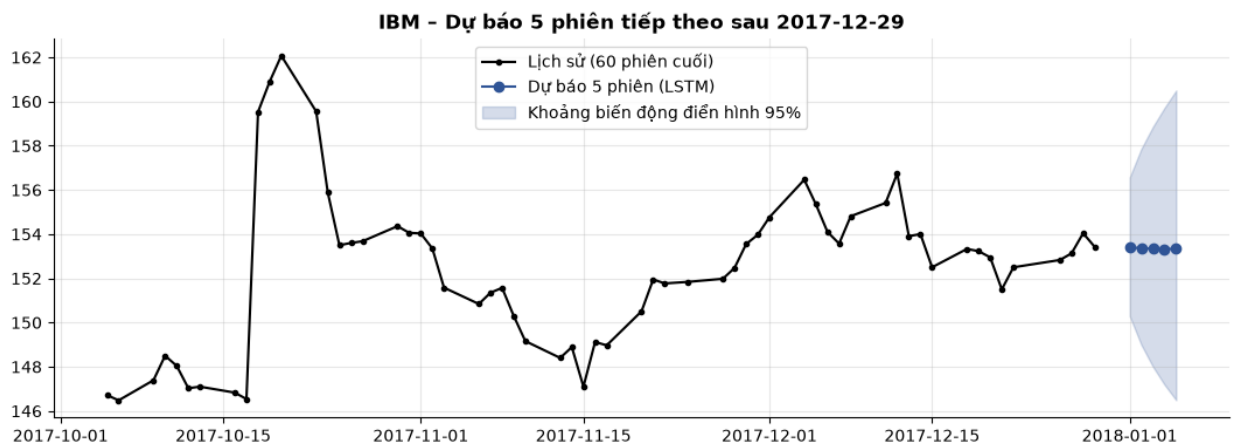
```
def forecast_future(model, horizon):
    """Dự báo đệ quy: giá dự báo được thêm vào lịch sử như một phiên mới (0=H=L=C, Volume giữ nguyên), rồi tính lại đặc trưng."""
    hist = df_t[["Open", "High", "Low", "Close", "Volume"]].tail(WINDOW + 60).copy()
    rows = []
    for h in range(1, horizon + 1):
        f = make_features(hist)[FEATS].dropna().tail(WINDOW)
        x = scaler_x.transform(f.values).astype("float32")[None, ...]
        r = float(inv_y(predict_scaled(model, x)[0, 0]))
        new_close = hist["Close"].iloc[-1] * np.exp(r)
        d = hist.index[-1] + pd.offsets.BDay(1)
        hist.loc[d] = [new_close, new_close, new_close, new_close, hist["Volume"].iloc[-1]]
        rows.append((d, new_close, r))
    return pd.DataFrame(rows, columns=["Ngày", "Giá dự báo", "Lợi suất dự báo"]).set_index("Ngày")

fc = forecast_future(models[best_kind], HORIZON)
s_ret = float(ret_true["test"].std())
steps = np.arange(1, HORIZON + 1)
fc["Cận dưới (95%)"] = df_t["Close"].iloc[-1] * np.exp(np.cumsum(fc["Lợi suất dự báo"].values) - 1.96 * s_ret * np.sqrt(steps))
fc["Cận trên (95%)"] = df_t["Close"].iloc[-1] * np.exp(np.cumsum(fc["Lợi suất dự báo"].values) + 1.96 * s_ret * np.sqrt(steps))

h_ = df_t["Close"].iloc[-60:]
fig, ax = plt.subplots(figsize=(13, 4.2))
ax.plot(h_.index, h_.values, c="k", marker="o", ms=3, label="Lịch sử (60 phiên cuối)")
ax.plot(fc.index, fc["Giá dự báo"], c=colors[best_kind], marker="o", label=f"Dự báo {HORIZON} phiên ({best_kind})")
ax.fill_between(fc.index, fc["Cận dưới (95%)"], fc["Cận trên (95%)"], color=colors[best_kind], alpha=0.2, label="Khoảng biến động điển hình 95%")
ax.set_title(f"{TICKER} - Dự báo {HORIZON} phiên tiếp theo sau {df_t.index[-1].%Y-%m-%d}"); ax.legend();
plt.show()
display(fc.round(3))
print("Khoảng 95% được dựng từ độ biến động lịch sử của lợi suất, phản ánh mức bất định thực tế lớn hơn nhiều so với độ dịch chuyển của đường dự báo.")
```

Output

```
1/1 _____ 0s 62ms/step
1/1 _____ 0s 46ms/step
1/1 _____ 0s 51ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 45ms/step
```



Ngày	Giá dự báo	Lợi suất dự báo	Cận dưới (95%)	Cận trên (95%)
2018-01-01	153.3980	-0.0000	150.2980	156.5610
2018-01-02	153.3640	-0.0000	149.0000	157.8560
2018-01-03	153.3360	-0.0000	148.0090	158.8550
2018-01-04	153.3250	-0.0000	147.1910	159.7140
2018-01-05	153.3360	0.0000	146.4940	160.4970

Khoảng 95% được dựng từ độ biến động lịch sử của lợi suất, phản ánh mức bất định thực tế lớn hơn nhiều so với độ dịch chuyển của đường dự báo.

Giải thích code

Dự báo lợi suất 5 phiên sau 29/12/2017 và quy đổi thành giá.

Mỗi bước: lấy lịch sử OHLCV (WINDOW + 60 phiên cuối), tính lại đặc trưng bằng make_features, lấy 60 dòng cuối, chuẩn hóa bằng scaler_x, dự đoán lợi suất r, tính giá mới = giá cuối $\times \exp(r)$, thêm một phiên giả (O=H=L=C=giá mới, giữ Volume) ở ngày làm việc kế tiếp (BDay). Khoảng 95% dựng từ độ lệch chuẩn lợi suất thực trên test nhân $\sqrt{\text{số bước}}$, cộng dồn lợi suất dự báo.

Giải thích output

Giá dự báo giảm rất nhẹ từ 153,398 (01/01/2018) xuống 153,336 (05/01/2018); lợi suất dự báo ≈ 0 (làm tròn $-0,0000$). So với giá cuối 153,42, độ dịch chuyển chỉ vài phần trăm của đô la. Khoảng 95% mở rộng theo $\sqrt{\text{số bước}}$: từ 150,30 đến 156,56 ở bước 1, và 146,49 đến 160,50 ở bước 5, rất lớn so với đường dự báo. Ngày 01/01/2018 là ngày nghỉ lễ nhưng code dùng BDay (ngày làm việc theo lịch) nên vẫn gán ngày này, chỉ là nhãn nên không ảnh hưởng đến kết quả.

j) Kết luận

Cell 31: Kết luận tự động

Code

```
r = results
b = r.loc[best_kind]; n0 = r.loc["Naive (giá hôm nay)"]
print(f"RMSE giá test - {best_kind}: {b['RMSE giá']:.3f} | Naive: {n0['RMSE giá']:.3f} | thay đổi so với Naive: {(1 - b['RMSE giá'] / n0['RMSE giá']) * 100:+.2f}%")
print(f"R² so với dự báo 0 (lợi suất): {b['R² so với dự báo 0']:.4f} | Đúng hướng: {b['Đúng hướng (%)']:.1f}% (mức 'luôn đoán tăng' = {up_rate:.1f}%)")
if b["R² so với dự báo 0"] > 0.01 and b["Đúng hướng (%)"] > up_rate + 2:
    print("=> Có bằng chứng (yếu) cho thấy mô hình bắt được chút tín hiệu; cần kiểm chứng thêm qua nhiều mã và nhiều giai đoạn.")
else:
    print("=> Mô hình không vượt rõ rệt dự báo 'giá không đổi'. Đồ thị giá khớp chủ yếu nhờ giá hôm nay đã chứa gần hết thông tin, "
          "phù hợp với tính chất gần-ngẫu-nhiên của giá cổ phiếu.")
```

Output

```
RMSE giá test - LSTM: 1.637 | Naive: 1.618 | thay đổi so với Naive: -1.16%
R² so với dự báo 0 (lợi suất): -0.0219 | Đúng hướng: 48.7% (mức 'luôn đoán tăng' = 51.3%)
=> Mô hình không vượt rõ rệt dự báo 'giá không đổi'. Đồ thị giá khớp chủ yếu nhờ giá hôm nay đã chứa gần hết thông tin, phù hợp với tính chất gần-ngẫu-nhiên của giá cổ phiếu.
```

Giải thích code

So sánh mô hình tốt nhất với Naive và in nhận định có điều kiện.

In RMSE giá của mô hình tốt nhất, của Naive và thay đổi %; R^2 so với dự báo 0 và độ chính xác hướng so với mức 'luôn đoán tăng'. Nếu $R^2 > 0,01$ và độ chính xác hướng $>$ mức $+ 2\%$ thì kết luận có tín hiệu yếu; ngược lại kết luận không vượt được dự báo 'giá không đổi'.

Giải thích output

LSTM: RMSE giá 1,637 so với Naive 1,618 (kém 1,16%); $R^2 = -0,0219$; đúng hướng 48,7% so với mức 51,3%. Không thỏa điều kiện nên notebook kết luận mô hình không vượt rõ rệt dự báo 'giá không đổi', phù hợp với tính gần-ngẫu-nhiên của giá cổ phiếu. Đây là một kết quả trung thực và hợp lý.

2.2. Phiên bản PyTorch

Dữ liệu: **DJIA 30 Stock Time Series** (Kaggle) – tệp `all_stocks_2006-01-01_to_2018-01-01.csv` gồm OHLCV hằng ngày của 30 cổ phiếu, 2006–2017.

Bài toán: với một mã cổ phiếu, dùng WINDOW phiên gần nhất để dự đoán **lợi suất log của phiên kế tiếp**, rồi quy đổi thành **giá đóng cửa dự đoán**. So sánh RNN, LSTM, GRU với hai baseline (giá không đổi, drift).

Vì sao dự báo lợi suất mà không phải giá thô? Giá là chuỗi không dừng: tập kiểm thử (2016–2017) thường có giá cao hơn hẳn phạm vi của tập huấn luyện nên mô hình huấn luyện trên giá thô sẽ ngoại suy kém. Lợi suất là chuỗi gần dừng, và là đại lượng phản ánh đúng khả năng dự báo.

Quy trình: 1) Nạp và kiểm tra → 2) EDA → 3) Đặc trưng và cửa sổ trượt → 4) Baseline → 5) Huấn luyện 3 mô hình → 6) Đánh giá, trực quan hóa → 7) Phân tích đặc trưng → 8) Dự báo tương lai → 9) Kết luận.

Cảnh báo: đây là bài tập học thuật. Kết quả không phải khuyến nghị đầu tư và không tính chi phí giao dịch.

a) Thư viện và cấu hình

Cell 2: Nhập thư viện và thiết lập hiển thị

Code

```
import os, glob, math, random, warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display
from sklearn.preprocessing import StandardScaler

warnings.filterwarnings("ignore")
plt.rcParams.update({
    "figure.dpi": 100, "figure.figsize": (11, 4),
    "axes.grid": True, "grid.alpha": 0.3,
    "axes.spines.top": False, "axes.spines.right": False,
    "axes.titlesize": 12, "axes.titleweight": "bold",
})
pd.set_option("display.float_format", lambda v: f"{v:,.4f}")
pd.set_option("display.max_columns", 30)
```

Cell này không tạo output hiển thị.

Giải thích code

Code giống hệt cell 2 của bản Keras (mục 2.1). Giống cell 2 của Bài 1: nhập `os`, `glob`, `math`, `random`, `warnings`, `numpy`, `pandas`, `matplotlib`, `display` và

StandardScaler; đặt style biểu đồ (lưới mờ, bỏ viền trên/phải, tiêu đề đậm) và định dạng số của pandas (dấu phân cách nghìn, 4 chữ số thập phân).

Cell 3: Khai báo tham số cấu hình

Code

```
# ===== CẤU HÌNH (có thể chỉnh) =====
DATA_PATH = "../data/all_stocks_2006-01-01_to_2018-01-01.csv" # <-- đường dẫn tới tệp CSV đã tải về
TICKER = "IBM" # mã cổ phiếu cần phân tích (AAPL, MSFT, GE, JPM, ...)
WINDOW = 60 # số phiên giao dịch lịch sử đưa vào RNN
HORIZON = 5 # số phiên dự báo tương lai
TRAIN_RATIO = 0.70
VAL_RATIO = 0.15

UNITS = 64
DROPOUT = 0.2
LR = 1e-3
BATCH_SIZE = 32
EPOCHS = int(os.environ.get("NB_EPOCHS", 100))
PATIENCE = 12
SEED = 42
```

Cell này không tạo output hiển thị.

Giải thích code

Code giống hệt cell 3 của bản Keras (mục 2.1). Gom tham số của bài toán cổ phiếu.

Cell 4: Nạp PyTorch và chọn thiết bị

Code

```
import copy
import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader

random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("PyTorch:", torch.__version__, "| Thiết bị:", device)
```

Output

```
PyTorch: 2.14.0+cu126 | Thiết bị: cuda
```

Giải thích code

Nạp torch, torch.nn, TensorDataset, DataLoader và copy; cố định seed; chọn thiết bị cuda nếu có.

Giải thích output

PyTorch 2.14.0+cu126, thiết bị cuda: huấn luyện chạy trên GPU (khác bản Keras chạy CPU).

Cell 5: Hàm tiện ích: tìm tệp, ACF, vẽ ACF

Code

```
def find_data_file(path, pattern):
    """Trả về đường dẫn tệp dữ liệu. Nếu DATA_PATH không tồn tại thì tự tìm theo mẫu tên trong thư mục hiện tại."""
    if os.path.exists(path):
        return path
    found = glob.glob(pattern) + glob.glob(os.path.join("...", pattern), recursive=True)
    found = sorted(set(found), key=lambda p: ("large" not in p.lower(), len(p), p))
    if not found:
        raise FileNotFoundError(
            f"Không tìm thấy '{path}'. Hãy sửa biến DATA_PATH ở ô cấu hình thành đường dẫn tới tệp CSV đã tải về.")
    print(f"Không thấy '{path}', tự động dùng: {found[0]}")
    return found[0]

def acf(x, nlags):
    """Hàm tự tương quan (ACF) tính trực tiếp bằng numpy."""
    x = np.asarray(x, dtype=float)
    x = x - x.mean()
    d = np.dot(x, x)
    return np.array([np.dot(x[:len(x) - k], x[k:]) / d for k in range(nlags + 1)])

def plot_acf(ax, x, nlags, title):
    a = acf(x, nlags)
    conf = 1.96 / np.sqrt(len(x))
    ax.bar(range(nlags + 1), a, width=0.3, color="#2F5496")
    ax.axhline(conf, ls="--", c="r", lw=1); ax.axhline(-conf, ls="--", c="r", lw=1)
    ax.axhline(0, c="k", lw=0.8)
    ax.set_title(title); ax.set_xlabel("Độ trễ (lag)"); ax.set_ylabel("ACF")
    return a, conf
```

Cell này không tạo output hiển thị.

Giải thích code

Code giống hệt cell 5 của bản Keras (mục 2.1). Giống cell 5 của Bài 1: find_data_file tìm tệp CSV theo mẫu tên khi đường dẫn sai; acf tính tự tương quan bằng numpy; plot_acf vẽ ACF kèm ngưỡng tin cậy $\pm 1,96/\sqrt{n}$ và trả về (mảng ACF, ngưỡng).

b) Nạp dữ liệu và kiểm tra chất lượng

Cell 7: Nạp dữ liệu và kiểm tra chất lượng

Code

```
DATA_PATH = find_data_file(DATA_PATH, "all_stocks*.csv")
raw = pd.read_csv(DATA_PATH)
raw.columns = raw.columns.str.strip()
need = ["Date", "Open", "High", "Low", "Close", "Volume", "Name"]
assert all(c in raw.columns for c in need), f"Thiếu cột. Cần {need}, hiện có {list(raw.columns)}"
raw["Date"] = pd.to_datetime(raw["Date"], errors="coerce")
raw = raw.dropna(subset=["Date"]).sort_values(["Name", "Date"]).reset_index(drop=True)

print(f"Tổng số dòng: {len(raw):,} | Số mã cổ phiếu: {raw['Name'].nunique():,}")
print(f"Khoảng thời gian: {raw['Date'].min().date()} -> {raw['Date'].max().date()}")
display(raw.head())
na = raw[need].isna().sum(); print("Giá trị thiếu theo cột:"); display(na.to_frame("Số ô thiếu").T)
print("Đông trùng (Name, Date):", raw.duplicated(["Name", "Date"]).sum())
```

Output

Tổng số dòng: 93,612 | Số mã cổ phiếu: 31
Khoảng thời gian: 2006-01-03 -> 2017-12-29

	Date	Open	High	Low	Close	Volume	Name
0	2006-01-03	39.6900	41.2200	38.7900	40.9100	24232729	AABA
1	2006-01-04	41.2200	41.9000	40.7700	40.9700	20553479	AABA
2	2006-01-05	40.9300	41.7300	40.8500	41.5300	12829610	AABA
3	2006-01-06	42.8800	43.5700	42.8000	43.2100	29422828	AABA
4	2006-01-09	43.1000	43.6600	42.8200	43.4200	16268338	AABA

Giá trị thiếu theo cột:

	Date	Open	High	Low	Close	Volume	Name
Số ô thiếu	0	25	10	20	0	0	0
Dòng trùng (Name, Date):	0						

Giải thích code

Code giống hệt cell 7 của bản Keras (mục 2.1). Đọc dữ liệu OHLCV của tất cả mã, kiểm tra cột, chuẩn hóa ngày và thống kê thiếu/trùng.

Giải thích output

Output giống hệt bản Keras (mục 2.1, cell 7). Tổng cộng 93.612 dòng, 31 mã cổ phiếu (30 mã DJIA cộng thêm AABA), từ 03/01/2006 đến 29/12/2017. Năm dòng đầu thuộc AABA (Yahoo/Altaba) với giá đóng cửa khoảng 40 đô la. Chi tiết đầy đủ xem giải thích ở mục 2.1.

c) Phân tích khám phá (EDA)

2.1 Tổng quan 30 mã và chọn mã phân tích

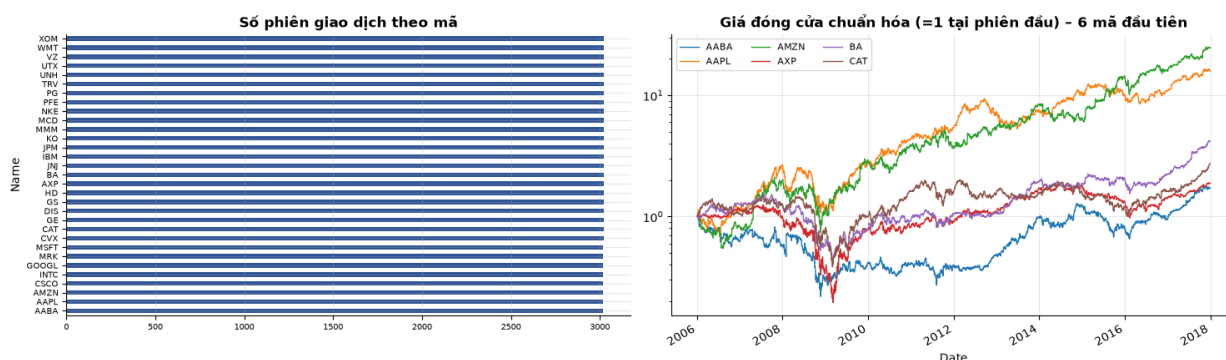
Cell 9: Tổng quan 30 mã và chọn mã phân tích IBM

Code

```
cnt = raw.groupby("Name").size().sort_values()
fig, axes = plt.subplots(1, 2, figsize=(15, 4.5))
cnt.plot.barh(ax=axes[0], color="#2F5496", fontsize=7); axes[0].set_title("Số phiên giao dịch theo mã")
tick_show = list(raw["Name"].unique()[:6])
for t in tick_show:
    s = raw[raw["Name"] == t].set_index("Date")["Close"].dropna()
    (s / s.iloc[0]).plot(ax=axes[1], lw=1, label=t)
axes[1].set_title("Giá đóng cửa chuẩn hóa (=1 tại phiên đầu) - 6 mã đầu tiên"); axes[1].legend(ncol=3,
    fontsize=8)
axes[1].set_yscale("log"); plt.tight_layout(); plt.show()
print("Các mã có ít phiên nhất (niêm yết muộn hoặc thiếu dữ liệu):", cnt.head(3).to_dict())

if TICKER not in raw["Name"].unique():
    print(f"Không có mã '{TICKER}', chuyển sang mã đầu tiên: {raw['Name'].iloc[0]}")
    TICKER = raw["Name"].iloc[0]
df_t = raw[raw["Name"] == TICKER].set_index("Date")[["Open", "High", "Low", "Close", "Volume"]].sort_index()
df_t = df_t[~df_t.index.duplicated()].ffill()
print(f"\n>> Phân tích chi tiết mã {TICKER}: {len(df_t)} phiên, {df_t.index.min().date()} -> {df_t.index.max().date()}")
display(df_t.describe().T)
```

Output



Các mã có ít phiên nhất (niêm yết muộn hoặc thiếu dữ liệu): {'AABA': 3019, 'AAPL': 3019, 'AMZN': 3019}

>>> Phân tích chi tiết mã IBM: 3020 phiên, 2006-01-03 -> 2017-12-29

	count	mean	std	min	25%	\
Open	3,020.0000	145.5153	37.5487	72.7400	116.4075	
High	3,020.0000	146.6817	37.6134	73.9400	117.7650	
Low	3,020.0000	144.4714	37.4714	69.5000	115.5000	
Close	3,020.0000	145.6173	37.5294	71.7400	116.5250	
Volume	3,020.0000	5,773,300.6225	3,192,830.5401	254,256.0000	3,622,681.2500	
	50%	75%	max			
Open	149.6050	178.4375	215.3800			
High	150.3300	179.7625	215.9000			
Low	148.4250	177.3200	214.3000			
Close	149.3150	178.6850	215.8000			
Volume	4,928,851.5000	6,965,014.0000	30,774,276.0000			

Giải thích code

Code giống hệt cell 9 của bản Keras (mục 2.1). Vẽ số phiên theo mã và giá chuẩn hóa của 6 mã đầu, sau đó lọc riêng mã IBM.

Giải thích output

Output giống hệt bản Keras (mục 2.1, cell 9). Biểu đồ trái: hầu hết mã có gần cùng số phiên (khoảng 3.019 đến 3.020). Biểu đồ phải: sau chuẩn hóa, AMZN và AAPL tăng gấp hơn 10 lần, còn các mã khác dao động thấp hơn nhiều; cú sứt 2008 đến 2009 thấy rõ ở mọi mã. Chi tiết đầy đủ xem giải thích ở mục 2.1.

2.2 Đặc điểm chuỗi giá của mã được chọn

Giá, đường trung bình động, khối lượng, phân phối lợi suất, biến động, tự tương quan, tương quan giữa các cột, lợi suất theo năm và drawdown.

Cell 11: EDA chi tiết chuỗi giá IBM (3 hình)

Code

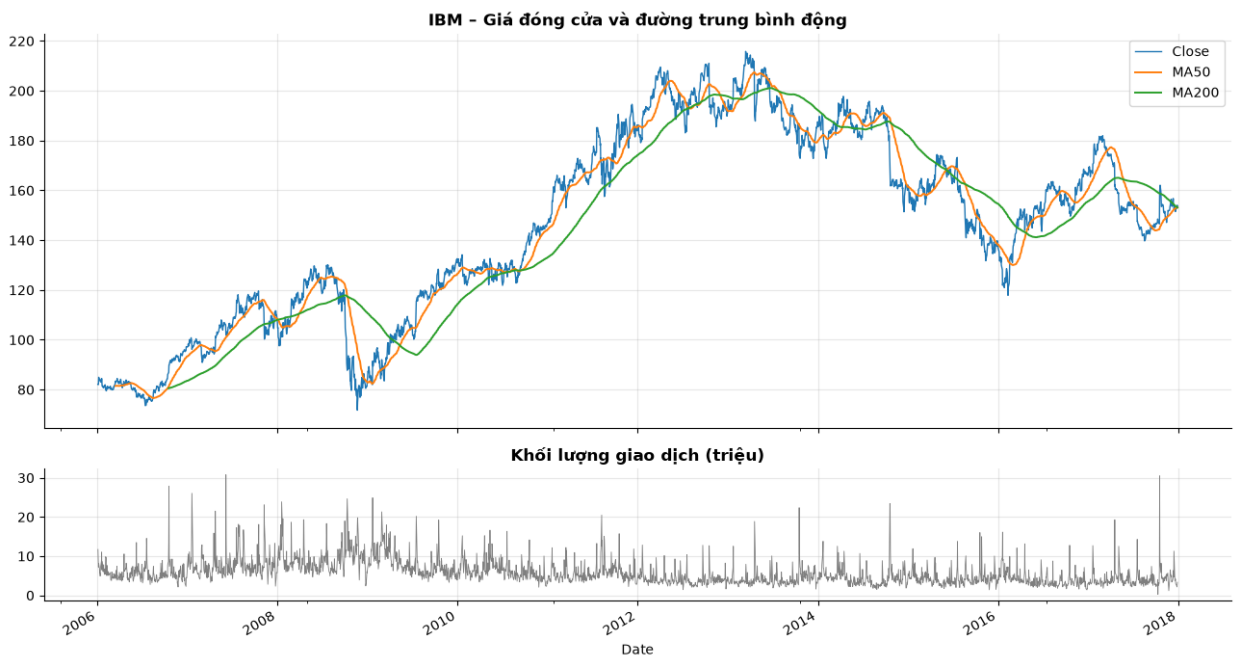
```
from scipy import stats
ret = np.log(df_t["Close"]).diff().dropna()

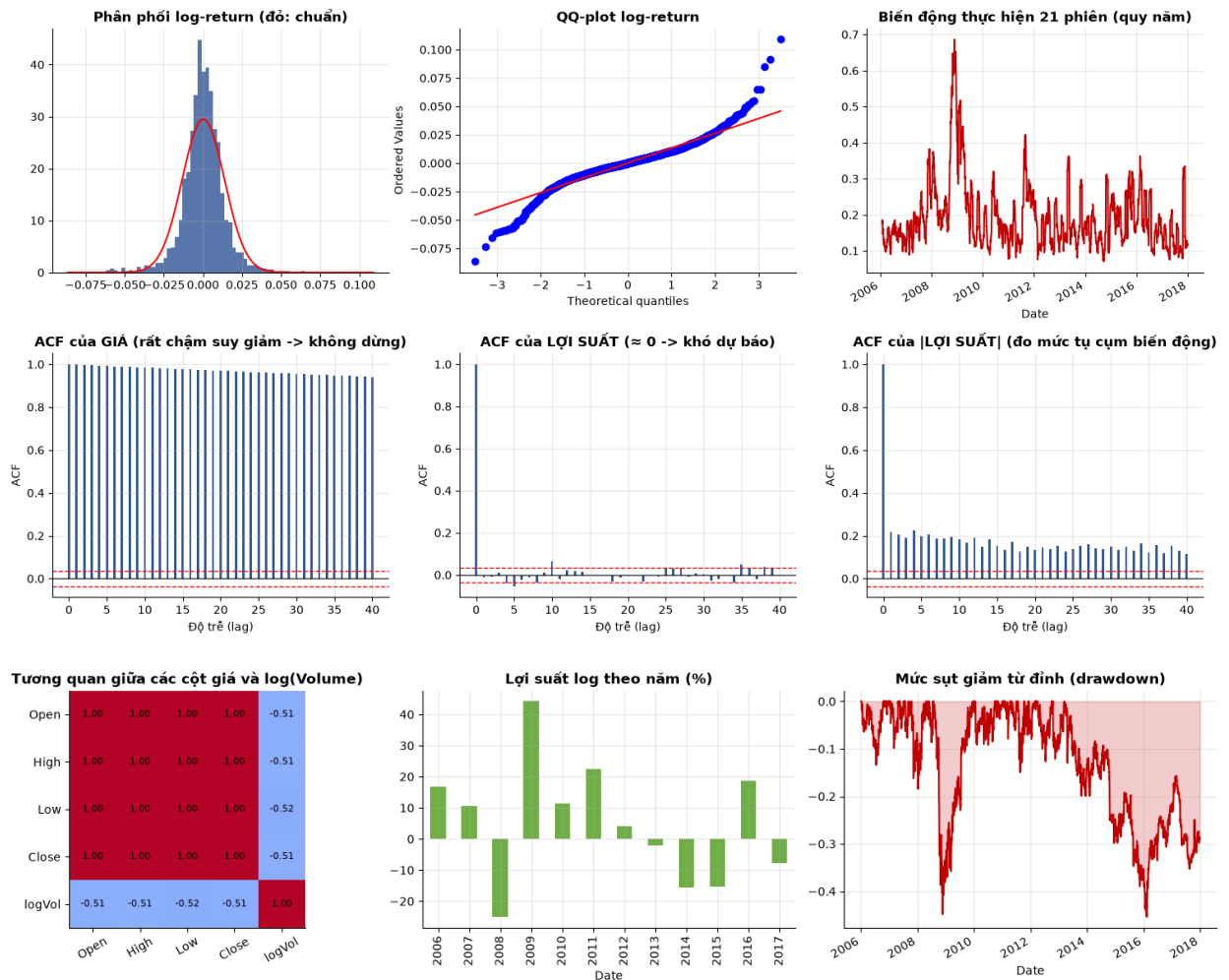
fig, axes = plt.subplots(2, 1, figsize=(13, 7), sharex=True, gridspec_kw={"height_ratios": [3, 1]})
df_t["Close"].plot(ax=axes[0], lw=1, label="Close")
df_t["Close"].rolling(50).mean().plot(ax=axes[0], lw=1.4, label="MA50")
df_t["Close"].rolling(200).mean().plot(ax=axes[0], lw=1.4, label="MA200")
axes[0].set_title(f"{TICKER} - Giá đóng cửa và đường trung bình động"); axes[0].legend()
(df_t["Volume"] / 1e6).plot(ax=axes[1], lw=0.6, c="#7F7F7F"); axes[1].set_title("Khối lượng giao dịch (triệu)")
plt.tight_layout(); plt.show()

fig, axes = plt.subplots(2, 3, figsize=(15, 8))
# 1) phân phối lợi suất so với phân phối chuẩn
axes[0, 0].hist(ret, bins=80, density=True, color="#2F5496", alpha=0.8)
xs = np.linspace(ret.min(), ret.max(), 300); axes[0, 0].plot(xs, stats.norm.pdf(xs, ret.mean(), ret.std()), "r")
axes[0, 0].set_title("Phân phối log-return (đồ: chuẩn)")
# 2) QQ-plot
stats.probplot(ret, dist="norm", plot=axes[0, 1]); axes[0, 1].set_title("QQ-plot log-return")
# 3) biến động
(ret.rolling(21).std() * np.sqrt(252)).plot(ax=axes[0, 2], c="#C00000"); axes[0, 2].set_title("Biến động thực
hiện 21 phiên (quy năm)")
# 4) ACF giá, lợi suất, |lợi suất|
plot_acf(axes[1, 0], df_t["Close"], 40, "ACF của GIÁ (rất chậm suy giảm -> không dừng)")
plot_acf(axes[1, 1], ret, 40, "ACF của LỢI SUẤT (= 0 -> khó dự báo)")
plot_acf(axes[1, 2], ret.abs(), 40, "ACF của |LỢI SUẤT| (đo mức tụ cụm biến động)")
plt.tight_layout(); plt.show()

fig, axes = plt.subplots(1, 3, figsize=(15, 4))
cm = df_t.assign(logVol=np.log1p(df_t["Volume"])).drop(columns="Volume").corr()
im = axes[0].imshow(cm, cmap="coolwarm", vmin=-1, vmax=1); axes[0].grid(False)
axes[0].set_xticks(range(len(cm))); axes[0].set_xticklabels(cm.columns, rotation=30);
axes[0].set_yticks(range(len(cm))); axes[0].set_yticklabels(cm.columns)
for i in range(len(cm)):
    for j in range(len(cm)):
        axes[0].text(j, i, f"{cm.iloc[i, j]:.2f}", ha="center", va="center", fontsize=8)
axes[0].set_title("Tương quan giữa các cột giá và log(Volume)")
(ret.groupby(ret.index.year).sum() * 100).plot.bar(ax=axes[1], color="#70AD47"); axes[1].set_title("Lợi suất
log theo năm (%)")
dd = df_t["Close"] / df_t["Close"].cummax() - 1
dd.plot(ax=axes[2], c="#C00000"); axes[2].fill_between(dd.index, dd.values, 0, color="#C00000", alpha=0.2);
axes[2].set_title("Mức sụt giảm từ đỉnh (drawdown)")
plt.tight_layout(); plt.show()
```

Output





Giải thích code

Code giống hệt cell 11 của bản Keras (mục 2.1). Ba hình khám phá chuỗi giá và lợi suất của IBM.

Giải thích output

Output giống hệt bản Keras (mục 2.1, cell 11). Hình 1: IBM tăng từ khoảng 80 (2006) lên đỉnh gần 215 (2013), sau đó giảm về khoảng 120 (đầu 2016) rồi phục hồi quanh 150 đến 155 vào cuối 2017; giá có xu hướng dài hạn rõ rệt nên không dừng. Khối lượng có xu hướng giảm dần với vài đỉnh đột biến (tới ~30 triệu). Chi tiết đầy đủ xem giải thích ở mục 2.1.

2.3 Nhận định tự động về tính chất thống kê

Cell 13: Nhận định tự động về tính chất thống kê

Code

```
a_p = acf(df_t["Close"], 20); a_r = acf(ret, 20); a_v = acf(ret.abs(), 20)
conf = 1.96 / np.sqrt(len(ret))
print(f"- Lợi suất hằng ngày: TB = {ret.mean() * 100:.3f}%, độ lệch chuẩn = {ret.std() * 100:.2f}%, "
      f"skew = {stats.skew(ret):.2f}, kurtosis dư = {stats.kurtosis(ret):.1f} (kurtosis dư > 0: đuôi dày hơn "
      f"phân phối chuẩn)")
print(f"- ACF của giá ở lag 20 = {a_p[20]:.2f} -> " + ("giá tự tương quan rất mạnh, chuỗi không dừng." if
      a_p[20] > 0.5 else "tự tương quan của giá không quá mạnh."))
print(f"- ACF của lợi suất: lag-1 = {a_r[1]:.3f}, lag-5 = {a_r[5]:.3f} (ngưỡng ±{conf:.3f}) -> " + ("hầu như "
      f"không có tín hiệu tuyến tính." if max(abs(a_r[1:6])) < 2 * conf else "có một vài tự tương quan đáng chú "
      f"ý."))
print(f"- ACF của |lợi suất| ở lag 1 = {a_v[1]:.2f} -> " + ("biến động tự cụm (giai đoạn dao động mạnh đi liền "
      f"nhau)." if a_v[1] > conf else "chưa thấy hiện tượng tự cụm biến động rõ rệt."))
try:
    from statsmodels.tsa.stattools import adfuller
    print(f"- Kiểm định ADF: p-value(giá) = {adfuller(df_t['Close'].dropna())[1]:.3f} | p-value(lợi suất) = "
          f"{adfuller(ret)[1]:.4f}")
    print(" (p > 0.05: không bác bỏ nghiệm đơn vị -> giá không dừng; lợi suất dừng)")
except Exception:
    print("- (Bỏ qua ADF vì chưa cài statsmodels)")
print("=> Vì vậy ta dự báo LỢI SUẤT (dùng) thay vì giá thô, rồi quy đổi lại thành giá: giá(t+1) = giá(t) × "
      f"exp(lợi suất dự báo).")
```

Output

```
- Lợi suất hằng ngày: TB = 0.021%, độ lệch chuẩn = 1.35%, skew = -0.12, kurtosis dư = 6.1 (kurtosis dư > 0: đũa
i dày hơn phân phối chuẩn)
- ACF của giá ở lag 20 = 0.97 -> giá tự tương quan rất mạnh, chuỗi không dừng.
- ACF của lợi suất: lag-1 = -0.011, lag-5 = -0.054 (ngưỡng ±0.036) -> hầu như không có tín hiệu tuyến tính.
- ACF của |lợi suất| ở lag 1 = 0.22 -> biến động tự cụm (giai đoạn dao động mạnh đi liền nhau).
- (Bỏ qua ADF vì chưa cài statsmodels)
=> Vì vậy ta dự báo LỢI SUẤT (dùng) thay vì giá thô, rồi quy đổi lại thành giá: giá(t+1) = giá(t) × exp(lợi suất
dự báo).
```

Giải thích code

Code giống hệt cell 13 của bản Keras (mục 2.1). Tính các thống kê bằng số để chứng minh: dự báo lợi suất hợp lý hơn dự báo giá.

Giải thích output

Output giống hệt bản Keras (mục 2.1, cell 13). Lợi suất hằng ngày trung bình 0,021%, độ lệch chuẩn 1,35%, skew -0,12, kurtosis dư 6,1 (đuôi dày). ACF giá ở lag 20 = 0,97 nên giá tự tương quan cực mạnh và không dừng. ACF lợi suất: lag-1 = -0,011, lag-5 = -0,054, ngưỡng ±0,036; lag-5 vượt ngưỡng một chút nhưng dưới hai lần ngưỡng nên code kết luận 'hầu như không có tín hiệu tuyến tính'. ACF |lợi suất| lag-1 = 0,22, biến động tự cụm. Chi tiết đầy đủ xem giải thích ở mục 2.1.

d) Tạo đặc trưng, chia dữ liệu theo thời gian, chuẩn hóa và tạo cửa sổ

Bảy đặc trưng đều **dừng** (không phụ thuộc mức giá tuyệt đối): lợi suất, khoảng nhảy mở cửa, biên độ trong phiên, khối lượng tương đối, độ lệch so với MA5 và MA20, biến động 10 phiên. Nhấn là lợi suất phiên kế tiếp. Các bộ chuẩn hóa chỉ fit trên tập huấn luyện. Mỗi mẫu có dạng (WINDOW, 7).

Cell 15: Tạo đặc trưng, chia thời gian, chuẩn hóa, tạo cửa sổ

Code

```
FEATS = ["ret", "gap", "range", "vol_z", "ma5", "ma20", "vol10"]
N_FEATURES, OUT_DIM = len(FEATS), 1

def make_features(d):
    """Tạo đặc trưng từ OHLCV. Dữ liệu được cho dự báo quy ở phần 9."""
    o, h, l, c, v = d["Open"], d["High"], d["Low"], d["Close"], d["Volume"]
    prev_c, lv = c.shift(1), np.log1p(v)
    f = pd.DataFrame(index=d.index)
    f["ret"] = np.log(c / prev_c) # lợi suất log hôm nay
    f["gap"] = np.log(o / prev_c) # khoảng nhảy giá lúc mở cửa
    f["range"] = (h - l) / c # biên độ trong phiên
    f["vol_z"] = lv - lv.rolling(20).mean() # khối lượng so với TB 20 phiên (log)
    f["ma5"] = c / c.rolling(5).mean() - 1 # lệch so với MA5
    f["ma20"] = c / c.rolling(20).mean() - 1 # lệch so với MA20
    f["vol10"] = f["ret"].rolling(10).std() # biến động 10 phiên
    f["Close"] = c
    return f.replace([np.inf, -np.inf], np.nan)

feat = make_features(df_t)
feat["target"] = feat["ret"].shift(-1) # NHÃN: lợi suất của phiên kế tiếp
feat = feat.dropna(subset=FEATS) # hàng cuối có target=NaN được giữ lại cho dự báo thật
display(feat.tail(3))

F = feat[FEATS].values.astype("float32")
T = feat["target"].values
C = feat["Close"].values
idx = feat.index
n = len(feat); n_lab = n - 1 # số hàng có nhãn
i1, i2 = int(n_lab * TRAIN_RATIO), int(n_lab * (TRAIN_RATIO + VAL_RATIO))

scaler_x = StandardScaler().fit(F[:i1])
scaler_y = StandardScaler().fit(T[:i1].reshape(-1, 1))
F_s = scaler_x.transform(F).astype("float32")

end_idx = np.arange(WINDOW - 1, n_lab) # chỉ số phiên cuối của mỗi cửa sổ
X_all = np.stack([F_s[t - WINDOW + 1:t + 1] for t in end_idx]).astype("float32")
y_all = scaler_y.transform(T[end_idx].reshape(-1, 1)).astype("float32")
m_tr, m_va, m_te = end_idx < i1, (end_idx >= i1) & (end_idx < i2), end_idx >= i2
X_train, y_train = X_all[m_tr], y_all[m_tr]
X_val, y_val = X_all[m_va], y_all[m_va]
X_test, y_test = X_all[m_te], y_all[m_te]

print(f"Train: {X_train.shape} | Val: {X_val.shape} | Test: {X_test.shape}")
print(f"Train: {idx[WINDOW]:%Y-%m-%d} -> {idx[i1]:%Y-%m-%d} | Val: {idx[i1 + 1]:%Y-%m-%d} -> {idx[i2]:%Y-%m-%d} | Test: {idx[i2 + 1]:%Y-%m-%d} -> {idx[-1]:%Y-%m-%d}")

fig, ax = plt.subplots(figsize=(13, 3.5))
ax.plot(idx, C, lw=0.8, c="gray")
for lo, hi, c_, lb in [(0, i1, "#2F5496", "Train"), (i1, i2, "#ED7D31", "Validation"), (i2, n, "#70AD47", "Test")]:
    ax.axvspan(idx[lo], idx[hi - 1], color=c_, alpha=0.15, label=lb)
ax.set_title("Chia dữ liệu theo trình tự thời gian (không xáo trộn)"); ax.legend(); plt.show()
```

Output

```
      ret    gap  range  vol_z   ma5   ma20  vol10   Close \
Date
2017-12-27  0.0020 0.0008 0.0037 -0.7202 0.0036 -0.0052 0.0078 153.1300
2017-12-28  0.0059 0.0005 0.0060 -0.4647 0.0081  0.0006 0.0058 154.0400
2017-12-29 -0.0040 0.0008 0.0085 -0.2177 0.0015 -0.0033 0.0060 153.4200

      target
Date
2017-12-27  0.0059
2017-12-28 -0.0040
2017-12-29   NaN
Train: (2041, 60, 7) | Val: (450, 60, 7) | Test: (450, 60, 7)
Train: 2006-04-27 -> 2014-06-05 | Val: 2014-06-06 -> 2016-03-18 | Test: 2016-03-21 -> 2017-12-29
```



Giải thích code

Code giống hệt cell 15 của bản Keras (mục 2.1). Sinh 7 đặc trưng dùng từ OHLCV, chia dữ liệu theo thời gian, chuẩn hóa và tạo cửa sổ 60 phiên.

Giải thích output

Output giống hệt bản Keras (mục 2.1, cell 15). Bảng 3 dòng cuối: ngày 29/12/2017 có Close 153,42 và target = NaN (chưa biết lợi suất ngày sau), đúng như thiết kế. Chi tiết đầy đủ xem giải thích ở mục 2.1.

e) Mô hình cơ sở (baseline)

**Naive* dự báo lợi suất = 0 (giá ngày mai = giá hôm nay) và *Drift* dự báo bằng lợi suất trung bình của tập train. Với dữ liệu này, baseline Naive rất khó đánh bại.*

Cell 17: Mô hình cơ sở (baseline) và hàm đánh giá

Code

```
def eval_split(mask, ret_hat):
    """Quy đổi lợi suất dự báo -> giá và tính các chỉ số."""
    t = end_idx[mask]
    close_t, ret_true = C[t], T[t]
    p_true, p_hat = close_t * np.exp(ret_true), close_t * np.exp(ret_hat)
    sse, sse0 = np.sum((ret_true - ret_hat) ** 2), np.sum(ret_true ** 2)
    nz = np.any(ret_hat != 0)
    return {
        "RMSE giá": float(np.sqrt(np.mean((p_true - p_hat) ** 2))),
        "MAE giá": float(np.mean(np.abs(p_true - p_hat))),
        "MAPE giá (%)": float(np.mean(np.abs((p_true - p_hat) / p_true)) * 100),
        "RMSE lợi suất": float(np.sqrt(sse / len(t))),
        "R² so với dự báo 0": float(1 - sse / sse0),
        "Đúng hướng (%)": float(np.mean(np.sign(ret_hat) == np.sign(ret_true)) * 100) if nz else np.nan,
    }

masks = {"val": m_va, "test": m_te}
ret_true = {k: T[end_idx[m]] for k, m in masks.items()}
drift = float(T[:1].mean()) # lợi suất trung bình của tập train
ret_hats = {k: {"Naive (giá hôm nay)": np.zeros(m.sum()), "Drift (TB train)": np.full(m.sum(), drift)} for k, m
             in masks.items()}

up_rate = float(np.mean(ret_true["test"] > 0) * 100)
print(f"Tỷ lệ phiên tăng giá trong tập test: {up_rate:.1f}% (mốc so sánh cho độ chính xác hướng: luôn đoán 'tăng' đạt ≈ 51.3%)")
display(pd.DataFrame({k: eval_split(masks["test"], v) for k, v in ret_hats["test"].items()}))
```

Output

```
Tỷ lệ phiên tăng giá trong tập test: 51.3% (mốc so sánh cho độ chính xác hướng: luôn đoán 'tăng' đạt ≈ 51.3%)
```

	RMSE giá	MAE giá	MAPE giá (%)	RMSE lợi suất	\
Naive (giá hôm nay)	1.6180	1.0723	0.6870	0.0104	
Drift (TB train)	1.6190	1.0723	0.6870	0.0104	

	R² so với dự báo 0	Đúng hướng (%)
Naive (giá hôm nay)	0.0000	NaN
Drift (TB train)	-0.0008	51.3333

Giải thích code

Code giống hệt cell 17 của bản Keras (mục 2.1). Định nghĩa eval_split và dựng hai baseline: Naive và Drift.

Giải thích output

Output giống hệt bản Keras (mục 2.1, cell 17). Trong tập test có 51,3% phiên tăng giá, nên chiến lược luôn đoán 'tăng' đã đạt 51,3% (mốc cần vượt). Chi tiết đầy đủ xem giải thích ở mục 2.1.

f) Xây dựng và huấn luyện RNN / LSTM / GRU (PyTorch)

Ba kiến trúc dùng cùng siêu tham số để so sánh công bằng.

Cell 19: Mô hình PyTorch và vòng huấn luyện có nhật ký chi tiết

Code

```
class RNNRegressor(nn.Module):
    """2 lớp RNN/LSTM/GRU xếp chồng, lấy trạng thái ẩn của bước cuối -> Dropout -> MLP nhỏ."""
    def __init__(self, kind, n_features, out_dim, hidden_UNITS, n_layers=2, dropout=DROPOUT):
        super().__init__()
        cell = ("RNN": nn.RNN, "LSTM": nn.LSTM, "GRU": nn.GRU)[kind]
        self.rnn = cell(n_features, hidden, num_layers=n_layers, batch_first=True, dropout=dropout)
        self.drop = nn.Dropout(dropout)
        self.head = nn.Sequential(nn.Linear(hidden, 32), nn.ReLU(), nn.Linear(32, out_dim))

    def forward(self, x):
        out, _ = self.rnn(x)
        return self.head(self.drop(out[:, -1, :]))

def train_model(kind, X_tr, y_tr, X_va, y_va):
    torch.manual_seed(SEED)

    model = RNNRegressor(
        kind,
        X_tr.shape[2],
        y_tr.shape[1]
    ).to(device)

    opt = torch.optim.Adam(model.parameters(), lr=LR)

    sched = torch.optim.lr_scheduler.ReduceLROnPlateau(
        opt,
        factor=0.5,
        patience=max(3, PATIENCE // 3),
        min_lr=1e-5
    )

    loss_fn = nn.MSELoss()

    loader = DataLoader(
        TensorDataset(
            torch.tensor(X_tr, dtype=torch.float32),
            torch.tensor(y_tr, dtype=torch.float32)
        ),
        batch_size=BATCH_SIZE,
        shuffle=True
    )

    Xv = torch.tensor(X_va, dtype=torch.float32).to(device)
    yv = torch.tensor(y_va, dtype=torch.float32).to(device)

    hist = {
        "loss": [],
        "val_loss": []
    }

    best = float("inf")
    best_state = None
    wait = 0

    print(f"\n(' '*70)")
    print(f"Training {kind}")
    print(f"(' '*70)")

    for epoch in range(EPOCHS):
        model.train()
        total = 0.0

        for xb, yb in loader:
            xb, yb = xb.to(device), yb.to(device)

            opt.zero_grad()

            pred = model(xb)
            loss = loss_fn(pred, yb)

            loss.backward()

            # Chống exploding gradient
            nn.utils.clip_grad_norm_(
                model.parameters(),
                1.0
            )

            opt.step()

            total += loss.item() * len(xb)

        tr_loss = total / len(loader.dataset)
        model.eval()

        with torch.no_grad():
            val_pred = model(Xv)
            va_loss = loss_fn(val_pred, yv).item()
            sched.step(val_loss)
            hist["loss"].append(tr_loss)
            hist["val_loss"].append(va_loss)

        # Learning rate hiện tại
        current_lr = opt.param_groups[0]["lr"]

        # =====
        # IN TỪNG EPOCH
        # =====
        print(
            f"Epoch (epoch + 1:3d)/(EPOCHS) | "
            f"Loss: {tr_loss:.6f} | "
            f"Val Loss: {va_loss:.6f} | "
            f"LR: {current_lr:.2e}"
        )

        # =====
        # EARLY STOPPING
        # =====
        if va_loss < best - 1e-6:
            best = va_loss
            best_state = copy.deepcopy(
                model.state_dict()
            )
            wait = 0

            print(
                f"
                o Best model "
                f"val_loss = {best:.6f}"
            )
        else:
            wait += 1

            if wait >= PATIENCE:
                print(
                    f"
                    o Early stopping "
                    f"(patience = {PATIENCE})"
                )
                break

    model.load_state_dict(best_state)

    n_params = sum(
        p.numel()
        for p in model.parameters()
    )

    print(f"(' '*70)")
    print(
        f"(' '*50) | "
        f"dùng sau (len(hist['loss']):3d) epoch | "
        f"val_loss tốt nhất = {best:.6f} | "
        f"số tham số = {n_params:,}"
    )
    print(f"(' '*70)")

    return model, hist

def predict_scaled(model, X):
    """Dự đoán trên không gian đã chuẩn hóa, trả về mảng (n_mẫu, out_dim)."""
    model.eval(); outs = []
    with torch.no_grad():
        for i in range(0, len(X), 256):
            xb = torch.tensor(X[i:i + 256]).to(device)
            outs.append(model(xb).cpu().numpy())
    return np.concatenate(outs, axis=0)

print(RNNRegressor("LSTM", N_FEATURES, OUT_DIM))
```

Output

```
RNNRegressor(  
  (rnn): LSTM(7, 64, num_layers=2, batch_first=True, dropout=0.2)  
  (drop): Dropout(p=0.2, inplace=False)  
  (head): Sequential(  
    (0): Linear(in_features=64, out_features=32, bias=True)  
    (1): ReLU()  
    (2): Linear(in_features=32, out_features=1, bias=True)  
  )  
)
```

Giải thích code

Tương tự Bài 1 PyTorch: lớp RNNRegressor (RNN/LSTM/GRU 2 lớp, dropout, đầu Linear(64→32)–ReLU–Linear(32→1)), vòng huấn luyện tự viết với Adam, ReduceLROnPlateau, MSELoss, DataLoader(shuffle=True), cắt gradient (clip_grad_norm_ = 1,0) và early stopping bằng deepcopy state_dict.

Điểm khác so với Bài 1: hàm train_model in nhật ký phong phú hơn, gồm tiêu đề 'Training kind', mỗi epoch in Loss, Val Loss và LR hiện tại (opt.param_groups[0]['lr']), in dòng '↳ Best model' mỗi khi val_loss cải thiện và '↳ Early stopping' khi dừng, cuối cùng in tóm tắt với val_loss 6 chữ số thập phân. Dữ liệu chuyển tensor float32 tường minh.

Giải thích output

print(model): LSTM(7, 64, num_layers=2, batch_first=True, dropout=0.2), Dropout(0.2) và đầu Linear(64→32), ReLU, Linear(32→1): khớp với 7 đặc trưng và 1 đầu ra của bài này.

Cell 20: Huấn luyện ba mô hình (PyTorch)

Code

```
KINDS = ["RNN", "LSTM", "GRU"]  
models, histories = {}, {}  
for kind in KINDS:  
    models[kind], histories[kind] = train_model(kind, X_train, y_train, X_val, y_val)
```

Output

```
=====
Training RNN
=====
Epoch 1/100 | Loss: 1.023451 | Val Loss: 0.874845 | LR: 1.00e-03
           □ Best model (val_loss = 0.874845)
Epoch 2/100 | Loss: 1.016810 | Val Loss: 0.870675 | LR: 1.00e-03
           □ Best model (val_loss = 0.870675)
Epoch 3/100 | Loss: 1.017809 | Val Loss: 0.884746 | LR: 1.00e-03
Epoch 4/100 | Loss: 1.010511 | Val Loss: 0.885190 | LR: 1.00e-03
Epoch 5/100 | Loss: 1.007613 | Val Loss: 0.870915 | LR: 1.00e-03
Epoch 6/100 | Loss: 0.999160 | Val Loss: 0.866731 | LR: 1.00e-03
           □ Best model (val_loss = 0.866731)
Epoch 7/100 | Loss: 0.996461 | Val Loss: 0.888452 | LR: 1.00e-03
Epoch 8/100 | Loss: 0.986766 | Val Loss: 0.862688 | LR: 1.00e-03
           □ Best model (val_loss = 0.862688)
Epoch 9/100 | Loss: 0.974969 | Val Loss: 0.864622 | LR: 1.00e-03
Epoch 10/100 | Loss: 0.959963 | Val Loss: 0.880085 | LR: 1.00e-03
Epoch 11/100 | Loss: 0.955346 | Val Loss: 0.885528 | LR: 1.00e-03
Epoch 12/100 | Loss: 0.930827 | Val Loss: 0.891657 | LR: 1.00e-03
Epoch 13/100 | Loss: 0.934208 | Val Loss: 0.890581 | LR: 5.00e-04
Epoch 14/100 | Loss: 0.899845 | Val Loss: 0.883518 | LR: 5.00e-04
Epoch 15/100 | Loss: 0.876221 | Val Loss: 0.904103 | LR: 5.00e-04
... (đã rút gọn 37 dòng ở giữa) ...
Epoch 2/100 | Loss: 1.020720 | Val Loss: 0.876346 | LR: 1.00e-03
Epoch 3/100 | Loss: 1.015882 | Val Loss: 0.886246 | LR: 1.00e-03
Epoch 4/100 | Loss: 1.019454 | Val Loss: 0.876477 | LR: 1.00e-03
Epoch 5/100 | Loss: 1.013070 | Val Loss: 0.904847 | LR: 1.00e-03
Epoch 6/100 | Loss: 1.010715 | Val Loss: 0.878076 | LR: 5.00e-04
Epoch 7/100 | Loss: 1.004103 | Val Loss: 0.892294 | LR: 5.00e-04
Epoch 8/100 | Loss: 1.004538 | Val Loss: 0.885859 | LR: 5.00e-04
Epoch 9/100 | Loss: 1.003906 | Val Loss: 0.880361 | LR: 5.00e-04
Epoch 10/100 | Loss: 1.002014 | Val Loss: 0.882289 | LR: 5.00e-04
Epoch 11/100 | Loss: 0.997154 | Val Loss: 0.892916 | LR: 2.50e-04
Epoch 12/100 | Loss: 0.993616 | Val Loss: 0.895149 | LR: 2.50e-04
Epoch 13/100 | Loss: 0.987233 | Val Loss: 0.895986 | LR: 2.50e-04
           □ Early stopping (patience = 12)
-----
GRU | dừng sau 13 epoch | val_loss tốt nhất = 0.874932 | số tham số = 41,089
=====
```

Giải thích code

Code giống hệt bản Keras: gọi `train_model` cho RNN, LSTM, GRU.

Giải thích output

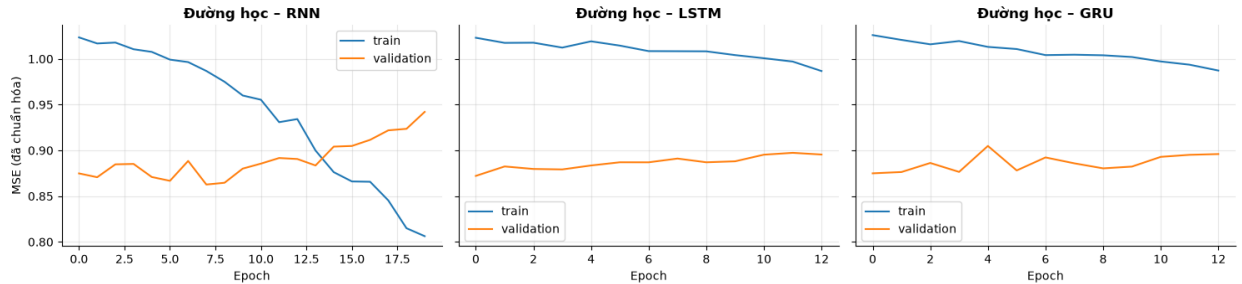
Nhật ký có LR giảm dần khi val_loss chững (ví dụ GRU: $1,00e-3 \rightarrow 5,00e-4 \rightarrow 2,50e-4$) và dòng '□ Early stopping'. Kết quả: RNN dừng sau 20 epoch, val_loss tốt nhất 0,862688, 15.105 tham số; LSTM 13 epoch, 0,872121, 54.081 tham số; GRU 13 epoch, 0,874932, 41.089 tham số. Số tham số của RNN và LSTM lớn hơn Keras một chút (15.105 so với 14.977; 54.081 so với 53.569) vì PyTorch dùng hai bias mỗi lớp.

Cell 21: Đường học (PyTorch)

Code

```
fig, axes = plt.subplots(1, 3, figsize=(15, 3.6), sharey=True)
for ax, kind in zip(axes, KINDS):
    ax.plot(histories[kind]["loss"], label="train"); ax.plot(histories[kind]["val_loss"], label="validation")
    ax.set_title(f"Đường học - {kind}"); ax.set_xlabel("Epoch"); ax.legend()
axes[0].set_ylabel("MSE (đã chuẩn hóa)")
plt.tight_layout(); plt.show()
```

Output



Giải thích code

Code giống hệt bản Keras.

Giải thích output

RNN có train loss giảm từ khoảng 1,02 xuống 0,80 trong 19 epoch trong khi validation loss tăng từ 0,875 lên khoảng 0,94: quá khớp mạnh. LSTM và GRU có train loss giảm chậm (1,02 xuống 0,985) và val loss tăng nhẹ (0,875 lên 0,895). Dạng đường học tương tự bản Keras.

g) Đánh giá và trực quan hóa dự đoán

Các chỉ số được tính ở cả mức giá (dễ đọc, nhưng bị lạc quan giả tạo do tự tương quan) và mức lợi suất (phản ánh khả năng dự báo thật), cùng độ chính xác dự đoán hướng.

Cell 23: Đánh giá: bảng chỉ số (PyTorch)

Code

```
inv_y = lambda z: scaler_y.inverse_transform(np.asarray(z).reshape(-1, 1)).ravel()
for kind in KINDS:
    for k, X in [("val", X_val), ("test", X_test)]:
        ret_hats[k][kind] = inv_y(predict_scaled(models[kind], X)[: , 0])

tab = {name: {f"{m} (val)": v for m, v in eval_split(masks["val"], ret_hats["val"][name]).items() if m in ["RMSE lợi suất"]},
        **eval_split(masks["test"], ret_hats["test"][name])} for name in ret_hats["test"]}
results = pd.DataFrame(tab).T
display(results[["RMSE giá", "MAE giá", "MAPE giá (%)", "RMSE lợi suất", "R² so với dự báo 0", "Đúng hướng (%)"]].sort_values("RMSE giá"))

best_kind = min(KINDS, key=lambda k: results.loc[k, "RMSE lợi suất (val)"])
print(f"Mô hình tốt nhất trên tập kiểm định (theo RMSE lợi suất): {best_kind}")

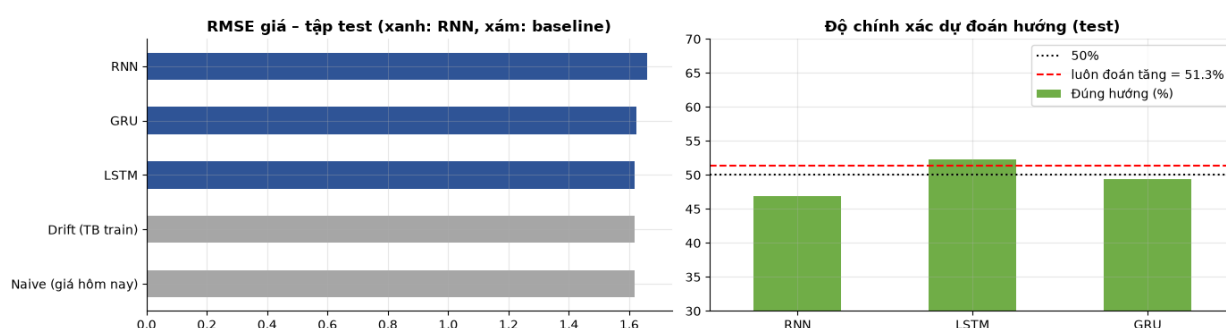
fig, axes = plt.subplots(1, 2, figsize=(14, 3.8))
s = results["RMSE giá"].sort_values()
s.plot.barh(ax=axes[0], color=["#2F5496" if i in KINDS else "#A6A6A6" for i in s.index]);
axes[0].set_title("RMSE giá - tập test (xanh: RNN, xám: baseline)")
d = results.loc[KINDS, "Đúng hướng (%)"]
d.plot.bar(ax=axes[1], color="#70AD47"); axes[1].axhline(50, c="k", ls=":", label="50%");
axes[1].axhline(up_rate, c="r", ls="--", label=f"luôn đoán tăng = {up_rate:1f}%");
axes[1].set_ylim(30, 70); axes[1].set_title("Độ chính xác dự đoán hướng (test)"); axes[1].legend();
axes[1].tick_params(axis="x", rotation=0)
plt.tight_layout(); plt.show()
```

Output

	RMSE giá	MAE giá	MAPE giá (%)	RMSE lợi suất \
Naive (giá hôm nay)	1.6180	1.0723	0.6870	0.0104
Drift (TB train)	1.6190	1.0723	0.6870	0.0104
LSTM	1.6192	1.0723	0.6870	0.0104
GRU	1.6238	1.0759	0.6892	0.0105
RNN	1.6580	1.1194	0.7167	0.0107

	R ² so với dự báo 0	Đúng hướng (%)
Naive (giá hôm nay)	0.0000	NaN
Drift (TB train)	-0.0008	51.3333
LSTM	-0.0011	52.2222
GRU	-0.0068	49.3333
RNN	-0.0492	46.8889

Mô hình tốt nhất trên tập kiểm định (theo RMSE lợi suất): RNN



Giải thích code

Code giống hệt bản Keras.

Giải thích output

Bảng (sắp theo RMSE giá): Naive 1,6180, Drift 1,6190, LSTM 1,6192 (R^2 -0,0011; đúng hướng 52,22%), GRU 1,6238 (R^2 -0,0068; 49,33%), RNN 1,6580 (R^2 -0,0492; 46,89%). LSTM PyTorch gần như trùng Naive (dự đoán rất sát 0) và là mô hình duy nhất có độ chính xác hướng vượt mốc 51,3% (52,2%), nhưng chênh lệch 0,9 điểm phần trăm trên 450 mẫu nằm trong sai số thống kê.

Mô hình tốt nhất trên tập kiểm định (theo RMSE lợi suất) là RNN, vì RNN có val_loss thấp nhất (0,8627) ở cell 20. Trên test, RNN lại là mô hình tệ nhất. Đây là minh họa của việc chọn mô hình theo một tập kiểm định nhiều.

Cell 24: Đồ thị dự đoán và phân tích lợi suất (PyTorch)

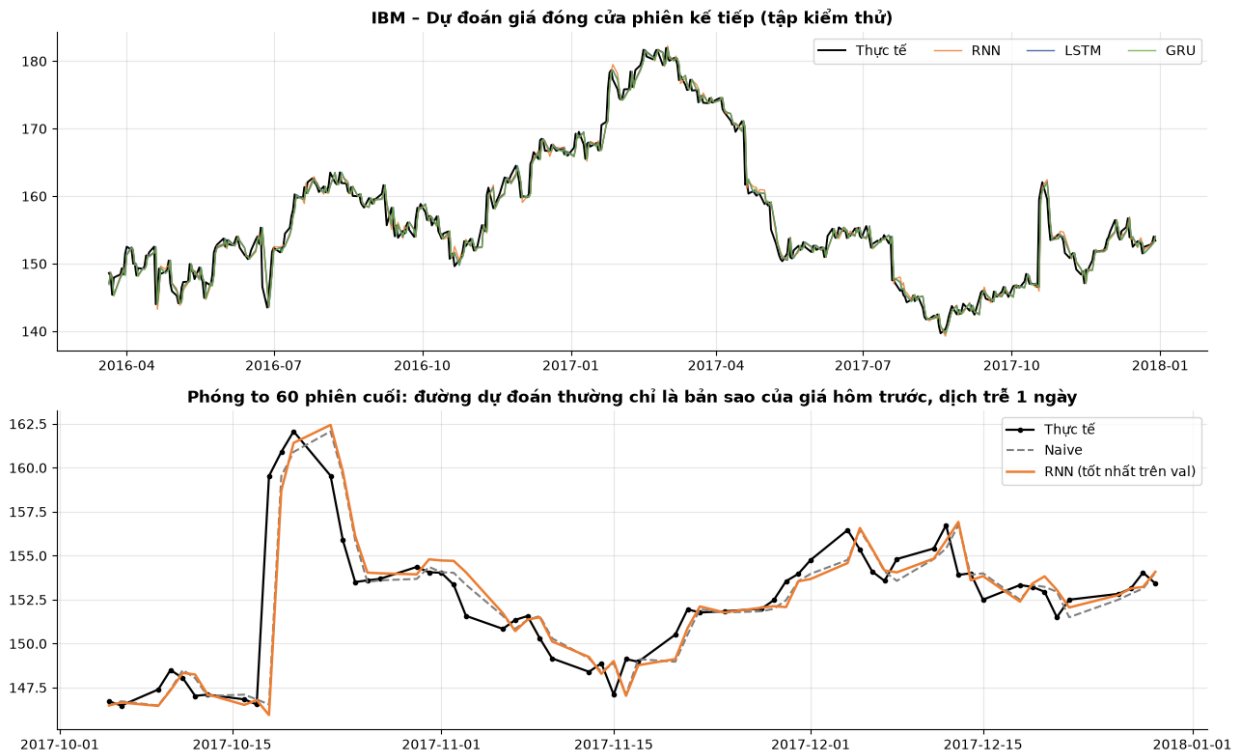
Code

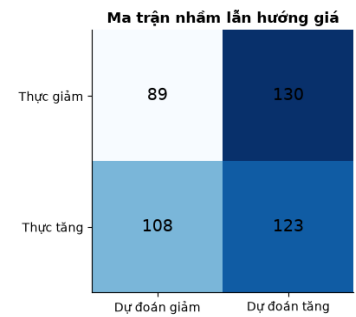
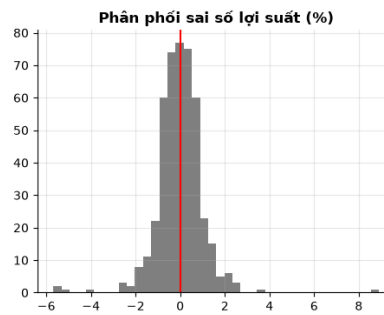
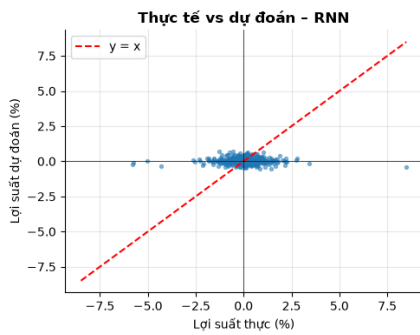
```
t_te = end_idx[m_te]
d_te = idx[t_te + 1] # ngày của giá được dự đoán
price_true = C[t_te + 1]
price_hat = {k: C[t_te] * np.exp(ret_hats["test"][k]) for k in ret_hats["test"]}
colors = {"RNN": "#ED7D31", "LSTM": "#2F5496", "GRU": "#70AD47"}

fig, axes = plt.subplots(2, 1, figsize=(13, 8))
axes[0].plot(d_te, price_true, c="k", lw=1.4, label="Thực tế")
for k in KINDS: axes[0].plot(d_te, price_hat[k], c=colors[k], lw=1, alpha=0.85, label=k)
axes[0].set_title(f"{TICKER} - Dự đoán giá đóng cửa phiên kế tiếp (tập kiểm thử)"); axes[0].legend(ncol=4)
z = slice(-60, None)
axes[1].plot(d_te[z], price_true[z], c="k", lw=1.6, marker="o", ms=3, label="Thực tế")
axes[1].plot(d_te[z], price_hat["Naive (giá hôm nay)"][z], c="gray", ls="--", label="Naive")
axes[1].plot(d_te[z], price_hat[best_kind][z], c=colors[best_kind], lw=1.8, label=f"{best_kind} (tốt nhất trên val)")
axes[1].set_title("Phóng to 60 phiên cuối: đường dự đoán thường chỉ là bản sao của giá hôm trước, dịch trễ 1 ngày"); axes[1].legend()
plt.tight_layout(); plt.show()

# --- Phân tích ở mức lợi suất (nơi thật sự đo được khả năng dự báo) ---
rt, rh = ret_true["test"], ret_hats["test"][best_kind]
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
axes[0].scatter(rt * 100, rh * 100, s=8, alpha=0.5); lim = max(abs(rt).max(), abs(rh).max()) * 100
axes[0].plot([-lim, lim], [-lim, lim], "r--", label="y = x"); axes[0].axhline(0, c="k", lw=0.5)
axes[0].set_xlabel("Lợi suất thực (%)"); axes[0].set_ylabel("Lợi suất dự đoán (%)"); axes[0].set_title(f"Thực tế vs dự đoán - {best_kind}")
axes[1].hist((rt - rh) * 100, bins=40, color="#7F7F7F"); axes[1].axvline(0, c="r"); axes[1].set_title("Phân phối sai số lợi suất (%)")
cmx = pd.crosstab(np.sign(rt) > 0, np.sign(rh) > 0).reindex(index=[False, True], columns=[False, True], fill_value=0)
axes[2].imshow(cmx.values, cmap="Blues"); axes[2].grid(False)
axes[2].set_xticks([0, 1]); axes[2].set_xticklabels(["Dự đoán giảm", "Dự đoán tăng"]); axes[2].set_yticks([0, 1]); axes[2].set_yticklabels(["Thực giảm", "Thực tăng"])
for i in range(2):
    for j in range(2): axes[2].text(j, i, int(cmx.values[i, j]), ha="center", va="center", fontsize=14)
axes[2].set_title("Ma trận nhầm lẫn hướng giá")
plt.tight_layout(); plt.show()
print(f"Độ lệch chuẩn lợi suất thực = {rt.std() * 100:.2f}% | dự đoán = {rh.std() * 100:.3f}% "
      "(dự đoán co cụm quanh 0 nghĩa là mô hình rất thận trọng, đúng với bản chất nhiều của lợi suất)")
```

Output





Độ lệch chuẩn lợi suất thực = 1.04% | dự đoán = 0.198% (dự đoán co cụm quanh 0 nghĩa là mô hình rất thận trọng, đúng với bản chất nhiều của lợi suất)

Giải thích code

Code giống hệt bản Keras, áp dụng cho mô hình tốt nhất là RNN.

Giải thích output

Cấu trúc ba hình như bản Keras. Độ lệch chuẩn lợi suất thực 1,04%, lợi suất dự đoán của RNN 0,198% (gấp đôi 0,091% của LSTM Keras): RNN 'dám' dự đoán lớn hơn nhưng không chính xác hơn, dẫn tới RMSE và độ chính xác hướng tệ hơn. Vì lợi suất dự đoán vẫn rất nhỏ so với 1,04%, đường giá dự đoán về cơ bản vẫn bám giá phiên hôm trước.

Cell 25: Chiến lược giao dịch minh họa (PyTorch)

Code

```
# ---- Minh họa: chiến lược mua/bán không theo dấu dự đoán so với mua và giữ (KHÔNG tính phí giao dịch) ----
pos_ = np.sign(ret_hats["test"][best_kind])
strat = pos_ * ret_true["test"]
fig, ax = plt.subplots(figsize=(12, 3.8))
ax.plot(d_te, np.exp(np.cumsum(ret_true["test"])), label="Mua và giữ", c="gray")
ax.plot(d_te, np.exp(np.cumsum(strat)), label=f"Theo dấu dự đoán {best_kind}", c=colors[best_kind])
ax.set_title("Giá trị tích lũy của 1 đơn vị vốn (chưa tính chi phí)"); ax.legend(); plt.show()
print(f"Sharpe (quy năm) chiến lược: {strat.mean() / (strat.std() + 1e-12) * np.sqrt(252):.2f} | mua-giữ: {ret_true['test'].mean() / ret_true['test'].std() * np.sqrt(252):.2f}")
print("Chỉ mang tính minh họa: kết quả không đủ để kết luận về khả năng sinh lời thực tế.")
```

Output



Sharpe (quy năm) chiến lược: -0.22 | mua-giữ: 0.14

Chỉ mang tính minh họa: kết quả không đủ để kết luận về khả năng sinh lời thực tế.

Giải thích code

Code giống hệt bản Keras.

Giải thích output

Sharpe chiến lược theo RNN $-0,22$, mua-giữ $+0,14$ (cùng số vì cùng tập test). Kém hơn mua-giữ nhưng đỡ tệ hơn nhiều so với $-1,51$ của LSTM Keras; cả hai đều là kết quả không có ý nghĩa thống kê. Kết quả thay đổi mạnh theo framework/seed là dấu hiệu của nhiễu.

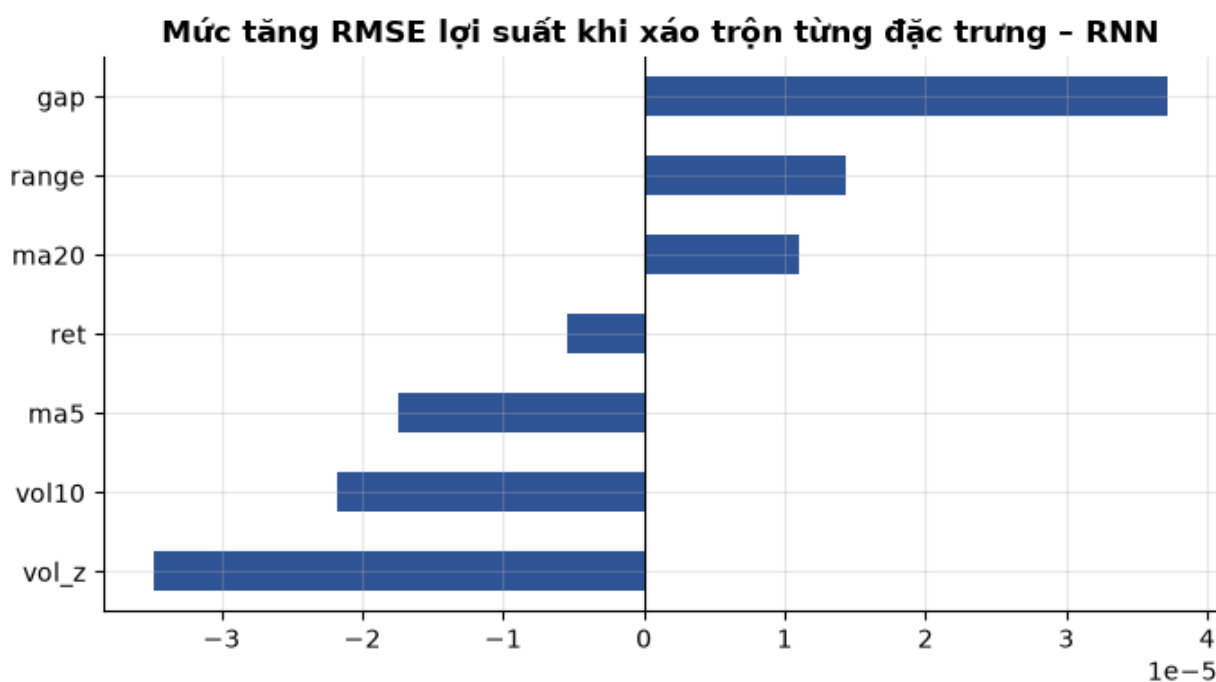
h) Phân tích đặc trưng nào quan trọng

Cell 27: Độ quan trọng đặc trưng (PyTorch)

Code

```
base = eval_split(masks["test"], ret_hats["test"][best_kind])["RMSE lợi suất"]
rng = np.random.default_rng(SEED)
imp = {}
for j, name in enumerate(FEATS):
    ds = []
    for _ in range(5):
        Xp = X_test.copy(); Xp[:, :, j] = Xp[rng.permutation(len(Xp))][:, :, j]
        ds.append(eval_split(masks["test"], inv_y(predict_scaled(models[best_kind], Xp[:, 0])))["RMSE lợi suất"] - base)
    imp[name] = np.mean(ds)
imp = pd.Series(imp).sort_values()
fig, ax = plt.subplots(figsize=(8, 4)); imp.plot.barh(ax=ax, color="#2F5496"); ax.axvline(0, c="k", lw=0.8)
ax.set_title(f"Mức tăng RMSE lợi suất khi xáo trộn từng đặc trưng - {best_kind}"); plt.show()
print("Đặc trưng mô hình dựa vào nhiều nhất:", imp.idxmax())
```

Output



Đặc trưng mô hình dựa vào nhiều nhất: gap

Giải thích code

Code giống hệt bản Keras, chạy trên RNN.

Giải thích output

Đặc trưng 'quan trọng nhất' cũng là gap. Như ở bản Keras, các mức thay đổi RMSE lợi suất cực nhỏ (thang rất bé so với RMSE 0,0107), nên không nên diễn giải như bằng chứng đặc trưng có sức dự báo.

i) Dự báo tương lai

Dự báo đệ quy HORIZON phiên sau phiên cuối cùng của dữ liệu.

Cell 29: Dự báo đệ quy 5 phiên (PyTorch)

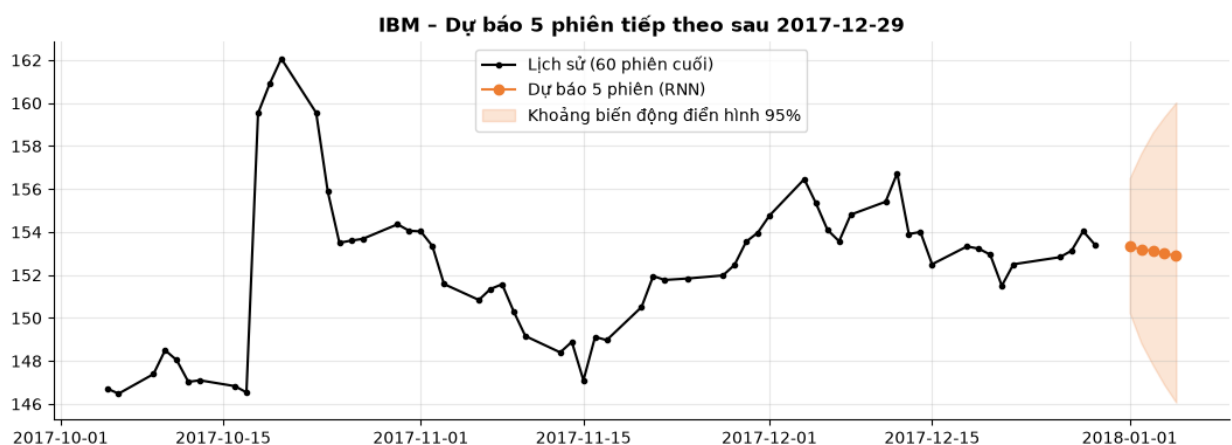
Code

```
def forecast_future(model, horizon):
    """Dự báo đệ quy: giá dự báo được thêm vào lịch sử như một phiên mới (0=H=L=C, Volume giữ nguyên), rồi tính lại đặc trưng."""
    hist = df_t[["Open", "High", "Low", "Close", "Volume"]].tail(WINDOW + 60).copy()
    rows = []
    for h in range(1, horizon + 1):
        f = make_features(hist)[FEATS].dropna().tail(WINDOW)
        x = scaler_x.transform(f.values).astype("float32")[None, ...]
        r = float(inv_y(predict_scaled(model, x)[:0][0]))
        new_close = hist["Close"].iloc[-1] * np.exp(r)
        d = hist.index[-1] + pd.offsets.BDay(1)
        hist.loc[d] = [new_close, new_close, new_close, new_close, hist["Volume"].iloc[-1]]
        rows.append((d, new_close, r))
    return pd.DataFrame(rows, columns=["Ngày", "Giá dự báo", "Lợi suất dự báo"]).set_index("Ngày")

fc = forecast_future(models[best_kind], HORIZON)
s_ret = float(ret_true["test"].std())
steps = np.arange(1, HORIZON + 1)
fc["Cận dưới (95%)"] = df_t["Close"].iloc[-1] * np.exp(np.cumsum(fc["Lợi suất dự báo"].values) - 1.96 * s_ret * np.sqrt(steps))
fc["Cận trên (95%)"] = df_t["Close"].iloc[-1] * np.exp(np.cumsum(fc["Lợi suất dự báo"].values) + 1.96 * s_ret * np.sqrt(steps))

h_ = df_t["Close"].iloc[-60:]
fig, ax = plt.subplots(figsize=(13, 4.2))
ax.plot(h_.index, h_.values, c="k", marker="o", ms=3, label="Lịch sử (60 phiên cuối)")
ax.plot(fc.index, fc["Giá dự báo"], c=colors[best_kind], marker="o", label=f"Dự báo {HORIZON} phiên ({best_kind})")
ax.fill_between(fc.index, fc["Cận dưới (95%)"], fc["Cận trên (95%)"], color=colors[best_kind], alpha=0.2, label="Khoảng biến động điển hình 95%")
ax.set_title(f"{TICKER} - Dự báo {HORIZON} phiên tiếp theo sau {df_t.index[-1].%Y-%m-%d}"); ax.legend();
plt.show()
display(fc.round(3))
print("Khoảng 95% được dựng từ độ biến động lịch sử của lợi suất, phản ánh mức bất định thực tế lớn hơn nhiều so với độ dịch chuyển của đường dự báo.")
```

Output



Ngày	Giá dự báo	Lợi suất dự báo	Cận dưới (95%)	Cận trên (95%)
2018-01-01	153.3440	-0.0000	150.2460	156.5070
2018-01-02	153.1840	-0.0010	148.8250	157.6710
2018-01-03	153.1280	-0.0000	147.8080	158.6390
2018-01-04	152.9960	-0.0010	146.8750	159.3720
2018-01-05	152.8900	-0.0010	146.0680	160.0310

Khoảng 95% được dựng từ độ biến động lịch sử của lợi suất, phản ánh mức bất định thực tế lớn hơn nhiều so với độ dịch chuyển của đường dự báo.

Giải thích code

Code giống hệt bản Keras, dùng RNN.

Giải thích output

Giá dự báo giảm dần từ 153,344 xuống 152,890 (lợi suất khoảng $-0,001$ mỗi phiên), dốc hơn bản Keras (153,398 xuống 153,336) vì RNN đưa ra dự báo lớn hơn. Khoảng 95% từ 150,25 đến 156,51 ở bước 1 và từ 146,07 đến 160,03 ở bước 5; giá cuối 153,42 vẫn nằm sâu trong khoảng, nên độ dịch chuyển chỉ là chi tiết nhỏ so với bất định.

j) Kết luận

Cell 31: Kết luận tự động (PyTorch)

Code

```
r = results
b = r.loc[best_kind]; n0 = r.loc["Naive (giá hôm nay)"]
print(f"RMSE giá test - {best_kind}: {b['RMSE giá']:.3f} | Naive: {n0['RMSE giá']:.3f} | thay đổi so với Naive: {(1 - b['RMSE giá'] / n0['RMSE giá']) * 100:+.2f}%")
print(f"R² so với dự báo 0 (lợi suất): {b['R² so với dự báo 0']:.4f} | Đúng hướng: {b['Đúng hướng (%)']:.1f}% (mức 'luôn đoán tăng' = {up_rate:.1f}%)")
if b["R² so với dự báo 0"] > 0.01 and b["Đúng hướng (%)"] > up_rate + 2:
    print("=> Có bằng chứng (yếu) cho thấy mô hình bắt được chút tín hiệu; cần kiểm chứng thêm qua nhiều mã và nhiều giai đoạn.")
else:
    print("=> Mô hình không vượt rõ rệt dự báo 'giá không đổi'. Đồ thị giá khớp chủ yếu nhờ giá hôm nay đã chứa gần hết thông tin, "
          "phù hợp với tính chất gần-ngẫu-nhiên của giá cổ phiếu.")
```

Output

```
RMSE giá test - RNN: 1.658 | Naive: 1.618 | thay đổi so với Naive: -2.47%
R² so với dự báo 0 (lợi suất): -0.0492 | Đúng hướng: 46.9% (mức 'luôn đoán tăng' = 51.3%)
=> Mô hình không vượt rõ rệt dự báo 'giá không đổi'. Đồ thị giá khớp chủ yếu nhờ giá hôm nay đã chứa gần hết thông tin, phù hợp với tính chất gần-ngẫu-nhiên của giá cổ phiếu.
```

Giải thích code

Code giống hệt bản Keras.

Giải thích output

RNN: RMSE giá 1,658 so với Naive 1,618 (kém 2,47%); $R^2 = -0,0492$; đúng hướng 46,9% so với mức 51,3%. Kết luận giống bản Keras: mô hình không vượt dự báo 'giá không đổi'.

Phần IV. So sánh và kết luận

1. Kết quả Bài 1 (RMSE doanh thu, tập test)

Mô hình	Keras	PyTorch	R ² Keras	R ² PyTorch
Naive (hôm qua)	55.658	55.658	−1,0458	−1,0458
TB 7 ngày	41.844	41.844	−0,1563	−0,1563
RNN	40.875	39.497	−0,1034	−0,0303
LSTM	39.318	39.358	−0,0209	−0,0230
GRU	39.684	39.230	−0,0400	−0,0164
Tốt nhất theo val	LSTM	GRU		

2. Kết quả Bài 2 (tập test)

Mô hình	RMSE giá Keras	RMSE giá PyTorch	Đúng hướng Keras	Đúng hướng PyTorch
Naive	1,6180	1,6180	—	—
Drift	1,6190	1,6190	51,33%	51,33%
RNN	1,6742	1,6580	49,56%	46,89%
LSTM	1,6368	1,6192	48,67%	52,22%
GRU	1,6354	1,6238	49,78%	49,33%
Tốt nhất theo val	LSTM	RNN		

3. Nhận xét

Keras và PyTorch cho cùng bức tranh. Với cùng dữ liệu, kiến trúc (2 lớp × 64 đơn vị, dropout 0,2) và siêu tham số, hai framework đưa ra kết quả cùng bậc nhưng không trùng nhau: khác cách khởi tạo trọng số, khác thiết bị (CPU và GPU), và khác chi tiết cài đặt (PyTorch có thêm cắt gradient và dùng hai vector bias nên số tham số lớn hơn một chút). Mô hình được chọn là tốt nhất theo tập kiểm định cũng khác nhau giữa hai bản (Bài 1: LSTM và GRU; Bài 2: LSTM và RNN), cho thấy chênh lệch giữa RNN, LSTM, GRU nhỏ hơn mức nhiều do khởi tạo.

Bài 1 (doanh thu): dữ liệu tổng hợp ($\text{Total} \neq \text{Price} \times \text{Quantity}$ ở 99,98% dòng, phân phối mọi biến gần đều), chuỗi theo ngày gần nhiều trắng: ACF lag-1 chỉ 0,024, không xu hướng, không mùa vụ tuần. Cả 6 mạng đều đạt RMSE khoảng 39 đến 41 nghìn, thấp hơn baseline khoảng 2 đến 6%, nhưng $R^2 \approx 0$ và dự báo gần như một đường phẳng quanh mức trung bình (khoảng 503 đến 505 nghìn). Cải thiện so với baseline chỉ vì dự đoán trung bình dài hạn tốt hơn Naive hay trung bình 7 ngày, không phải do học được quy luật.

Bài 2 (cổ phiếu IBM): giá không dừng (ACF lag 20 = 0,97) nhưng lợi suất gần như không tự tương quan; chỉ độ lớn biến động ($|\text{lợi suất}|$) có tính tụ cụm. Không mạng nào đánh bại Naive: RMSE giá cao hơn 1 đến 3%, R^2 so với dự báo 0 âm, độ chính xác hướng quanh 47 đến 52% (mốc luôn đoán tăng là 51,3%). Chiến lược giao dịch minh họa lỗ (Sharpe -1,51 với LSTM Keras và -0,22 với RNN PyTorch so với +0,14 của mua-giữ). Đồ thị giá khớp thực tế chỉ vì giá hôm nay gần bằng giá ngày mai.

Bài học chung: (1) luôn so mô hình học sâu với baseline đơn giản; (2) R^2 và độ chính xác hướng đáng tin hơn đồ thị khớp giá; (3) khi val_loss không giảm và train loss vẫn giảm là dấu hiệu quá khớp, cần early stopping; (4) kết luận tự động dựa trên ngưỡng (ví dụ cải thiện > 3% nên kết luận có cấu trúc) cần được kiểm tra thêm bằng R^2 và hình dạng dự đoán.

Hạn chế và hướng mở rộng: mỗi cấu hình chỉ chạy một seed và một mã cổ phiếu; nên lập nhiều seed, thử nhiều mã, walk-forward validation. Với Bài 1, dữ liệu tổng hợp nên không có gì để dự báo; có thể chuyển sang bài toán phân loại churn. Với Bài 2, có thể dự báo biến động (volatility), vốn có cấu trúc rõ hơn lợi suất.